UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

STRUCTURAL CHARACTERISATION OF AI MODELS: PROFILING AND HARDWARE PERFORMANCE ANALYSIS

JAVIER BEIRO PINON

Thesis supervisor
PETAR RADOJKOVIC (Barcelona Supercomputing Center (BSC))

Tutor: EDUARD AYGUADÉ PARRA (Department of Computer Architecture)

Degree
Master's Degree in Innovation and Research in Informatics (High Performance Computing)

Master's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Abstract

This thesis presents a theoretical and experimental analysis of modern AI models, focusing on their structure and hardware performance characteristics. The work is divided into three main parts.

First, we **analyze the most widely used AI models** in the industry, including Deep Neural Networks (**DNNs**), Deep Learning Recommendation Systems (**DLRMs**), Recurrent Neural Networks (**RNNs**) and **Transformers**. We study their layer structures, compute and memory requirements, data types, and execution patterns. The goal is to identify common operations, such as matrix-vector multiplications or attention layers, that are frequently used across models and have a strong impact on hardware performance. This analysis is **summarized in the FAiNDER platform** [1], a live web tool that provides structured and interactive access to model details and their hardware demands. FAiNDER has received the HiPEAC Technology Transfer Award for its contribution to power technologies, which will support the development of the next generation of high-performance AI applications. This platform helps researchers and developers explore model architectures, compare performance characteristics, and find optimization opportunities based on reliable information from scientific publications.

In the second part of the thesis, we perform over **Llama3**, one of the most widely used LLM open source models, a detailed performance evaluation. We present a methodology to measure the compute and memory behavior of the model under different configurations and execution scenarios. The experimental setup includes analysis across layers and execution phases, helping to understand how models behave in real applications. Finally, we apply this knowledge in a practical case study, exploring how part of the model can be offloaded to an alternative hardware platform. This allows for improved resource utilization and system performance. The proposed methodology and results serve as a guide for future work in AI model **profiling** and hardware optimization.

Finally, we use the insights gained from the analysis to propose **offloading part of the model** to another hardware platform, with the goal of improving hardware utilization. This serves as a real-world example of how the theoretical and experimental analysis can be applied to enhance system performance. The case study demonstrates the potential benefits of offloading and provides a practical reference for future research in this area.

Acknowledgments

I would like to thank the entire team at the Barcelona Supercomputing Center for their support and expertise throughout this project. In particular, I am grateful to Víctor, Pouya, Ashkan, and Valeria for their valuable insights and assistance at every stage of this work. I am especially thankful to Mari for her support and patience during the thesis process. Her support was essential in overcoming challenges and completing this thesis.

Also thanks to my advisors, Petar Radojkovic and Eduard Ayguade, for guiding me in proper research methodology and helping me learn how to focus and extract meaningful results from large amounts of data.

Finally, I extend my gratitude to the entire Micron team—Emanuele Confalonieri, Jason Adlard, Rishabh Dubey, Usman Jamal, and Danilo Caraccio—for their constructive feedback and suggestions, which greatly improved the quality of this work and helped me understanding real industry needs.

This work was funded by the Collaboration Agreement between Micron Technology, Inc. and BSC. The results have been partially funded by the Spanish Ministry of Science and Innovation MCIN/AEI/10.13039/501100011033 (contracts PID2019-107255GB-C21, PCI2022-132935, PCI2021-121958, PID2023-146511NB-I00 and CEX2021-001148-S), the Generalitat of Catalunya (contract 2021-SGR-00763, 2021-SGR00807 and 2021-SGR-01264), and by the Barcelona Zettascale Lab (BZL) project with reference REGAGE22e00058408992 and co-financed by the Ministry for Digital Transformation and Public Services, within the framework of the Resilience and Recovery Fund – and the European Union – NextGenerationEU. This work is part of the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MCIU /AEI /10.13039/501100011033 and EU ERDF.

Contents

Chapter 1

Introduction

Artificial Intelligence has transformed how we solve complex problems, enabling machines to learn from data and make smart decisions. Specially in the last years, the field of AI has seen a rapid growth in research and applications, with a significant increase in the number of papers and models being published, as shown in Figure 1.1.

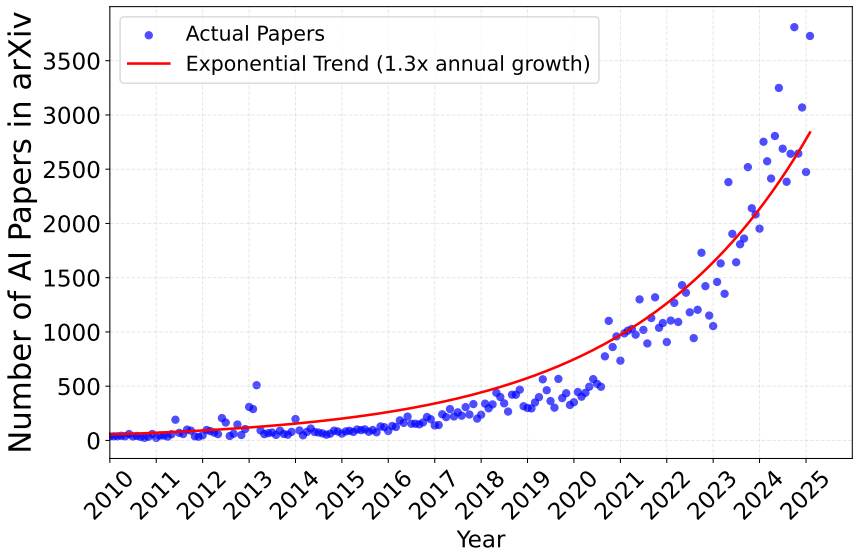


Figure 1.1: Exponential growth in the number of AI papers published on arXiv.

These AI systems now play crucial roles across industries, powering applications from voice assistants and facial recognition to personalized recommendations and language translation [2, 3, 4, 5].

However, this rapid progress brings new challenges. AI models vary significantly in their design and requirements, they use different architectures, process diverse data types, and perform distinct operations. Some models need millions of parameters while others require billions, leading to very different memory and computing needs. Even models with similar layer structures and number of parameters can behave differently depending on their task—for example, generating images requires different resources than summarizing text. These variations directly impact the hardware needed to run them effectively.

The situation becomes more complex when considering practical implementations. A model's requirements change based on whether it's being trained or used for predictions during inference, and new innovations in AI research constantly introduce novel architectures and techniques. This constant evolution makes it difficult for researchers and engineers to track all developments and

understand their hardware implications. Without clear insights into model characteristics and resource needs, optimizing performance becomes challenging, whether selecting suitable hardware, modifying models for efficiency, or adapting systems for better resource utilization.

Prior research has explored these challenges through mainly three approaches: **analyzing specific model families** (e.g., Transformers, RNNs), **studying task-specific implementations** (e.g., machine translation), or **profiling specific hardware performance**. Studies like the Purdue-NVIDIA analysis of BERT-Large on Volta GPUs identified softmax operations as bottlenecks, while others profiled DLRM models on Intel CPUs to highlight memory inefficiencies. However, these efforts **vary widely** in scope—some focus on inference versus training, use **different tools** (e.g., nvprof vs. VTune), or test **parameters** like sequence length and batch size inconsistently. For example, one study achieved 3.4× speedups on V100 GPUs by optimizing attention mechanisms, while another reduced CPU latency by addressing projection layers. This **lack of standardization** complicates comparisons, as metrics range from kernel execution times to cache hit rates across different platforms like proprietary ASIC, GPUs and CPUs.

This way, understanding and optimizing modern AI models presents several challenges. First, existing surveys employ **inconsistent terminology** and varied organization, which makes direct comparisons between models difficult. Second, this surveys have different focus, some of them focus narrowly, some examine broad model families and others target specific tasks or individual components resulting in **fragmented insights** that require considerable effort to integrate. Third, the rapid pace of innovation in AI means that **static surveys quickly become outdated** as new architectures and techniques appear.

Profiling studies add further complexity. Experimental setups differ widely across hardware platforms (such as CPUs, GPUs, and specialized accelerators), model designs (varying depths, attention patterns, layer types, operations...), and test parameters (including batch size, sequence length, input prompt...). Each research group may employ different measurement tools and reporting practices, which makes it difficult to obtain a global picture of the models performance.

1.1 Thesis contribution and structure

This thesis overcomes the problems mentioned through three interconnected contributions, each building on the previous.

The first part delivers a structured survey of industry-standard AI models. We analyze several model's layer structures and operations, data types, memory footprints and production hardware platform during inference and training. To address the limitations of monolithic surveys, we introduce **FAiNDER** [1], an interactive, web-based **platform**, **awarded with the HiPEAC Technology Transfer Award**, that unifies model characteristics and hardware requirements into a live, updatable resource. FAiNDER allows users to **explore per-layer models characteristics**. This enables **side-by-side comparison** and **continuous updates**, and ensures that researchers can track emerging trends and make informed choices when selecting or designing models.

Building on the survey, the second part presents a **standardized profiling methodology**, demonstrated on the Llama3-8B Large Language Model. We instrument the model at multiple granularity levels, to allow users to profile the whole model or specifics layers or operations. In the experiments we vary the parameters in a structures way, including sequence leght, input prompt and output size, allowing to include others parameters like batch size if needed. This allows users, to analyze the behavior of the different components of the model under different workloads and to understand how the model behaves in different parameters configurations. We run our experiments on a NVIDIA A100 GPU, measuring performance metrics such as execution time, memory usage, and kernel behavior. We consistently present results using standard data formats, such as FLOPs and GB/s, and visualize them with well-established methods like the roofline model. This approach enables easier interpretation, comparison, and extrapolation of findings to other models

and hardware platforms. We explore the model's performance across different execution phases (prefill and decode) and layers (self-attention, feed-forward networks, etc.). This way the profiling methodology is designed to be reproducible and extensible, enabling researchers to apply it to other models and hardware platforms and compare results easily.

In the final part, we present a **practical example** that applies the theoretical and profiling insights to system optimization through **layer offloading**. We identify and offload the layer most suitable for execution on secondary hardware, based on its computation and memory requirements.

Together, these three parts form a structure and a cohesive methodology to allow researches to identify limitations and possible improvements on AI models. This structure gives both theoretical foundations and practical tools to analyze, compare, and optimize AI models for real-world hardware environments.

Chapter 2

Structure and requirements of AI models

In this chapter, we focus on analyzing the main industry AI models in order to select the most suitable one for our profiling experiments. We study four of the most important AI models used in the industry: First, we analyze **Deep Neural Networks (DNNs)**, which are the foundation of modern AI models. DNNs are a key component in many complex systems. Next, we examine **Deep Learning Recommendation Systems (DLRS)**, which build upon DNNs by adding complexity. These models are widely used by leading companies such as Amazon, Netflix, and Google [6, 7, 8, 9]. Then, we look into **Recurrent Neural Networks (RNNs)**, which have historically played an important role in language processing tasks [5, 10, 11, 12, 13, 14]. RNNs show structural differences compared to traditional DNNs, which makes them unique. Finally, we study **Transformer models**, with a focus on large language models (LLM). These models share structural similarities with the previous types, both in layer usage and architecture. Transformers have gained great relevance in recent years due to the growing number of models developed in the industry and their increasing number of parameters. For all these model types, we analyze their industry adoption, layer implementations, structural characteristics, and hardware requirements. Our goal is to identify a model that is both widely used and presents practical challenges in real-world deployments. This will allow us to perform detailed profiling and detect possible hardware limitations under realistic usage conditions. All this models' information is **summarized in the FAiNDER** platform [1], a live web tool that has received the HiPEAC Technology Transfer Award that provides structured and interactive access to model details and their hardware demands.

2.1 Deep Neural Networks

2.1.1 Motivation

Deep Neural Networks (DNNs) are used for machine learning related to speech recognition [15], recommender systems [16], anomaly detection [17] and healthcare [18].

They are widely used by industry to predicting user preferences and suggesting relevant items:

- **Google**: Object detection [19] and voice recognition [20]
- **Apple**: Face recognition [2]
- **Amazon**: Custom skills and voice assistants [3]

2.1.2 Overview

Artificial neural networks are popular machine learning techniques that simulate the mechanism of neurons learning in biological organisms [21]. The networks are based on large number of simple computational units referred to as *neurons*. The neurons are organized in *layers*. The neurons have *weighted* connections to the neurons in the successive layer [21].

Figure 2.1 shows the Deep Neural Networks (DNNs) architecture. The DNN input is a vector, (x_1, x_2) in Figure 2.1, which is processed by various hidden layers. The DNN provides a single output vector, (o_{41}, o_{42}) in our example. The weights between the neurons are marked as w_{ij}^k, where e.g. w_{21}^1 corresponds to the weight between the first input neuron (x_1) and the first neuron in the second layer (h_{21}). Each neuron is associated with a *bias*, an parameter which adjusts the neuron output. The number of hidden layers is typically determined empirically through experimentation. It depends mainly on the task and data complexity, and the amount of the available computational resources [22].

2.1.3 Inference

During the inference, the input vector is processed through a pre-trained DNN with a defined architecture: the number of layers, neurons in each layer, biases and weights between all connected neurons. The network produces the prediction as the output vector, a process known as the forward phase [21].

2.1.3.1 Layers

The DNNs are comprised of fully connected layers in which all the neuron in layer j are connected to all neurons in the previous layer $(j-1)$ [23]. The neuron values are computed in two steps. In the first one, we compute the weighted sum of all the neurons from the previous layer and then add the bias value. For example, the value of h_{21} is computed as $[x_1, x_2] \cdot [w_{21}^1, w_{21}^2] + b_{21}$. The vector of all neurons in hidden layer 1 is therefore computed as the product of the dense input vector and the corresponding dense weight matrix, plus the bias vector (see Figure 2.1 bottom). This multiply and add calculation is the main DNN inference operation [24].In the second step, the vector-matrix multiplication output is passed through an activation function, which defines acceptable boundaries for the neuron values (e.g. removes the negative values) and introduces a non-linear transformation of the values within these boundaries [23]. Typical activation functions are Rectified Linear Unit (ReLU), sigmoid or tanh. The motivation for these functions and their implementation exceed the scope of this report. The number of neurons in the output layer varies depending on the network task, with common tasks being classification and regression. In classification, which involves predicting discrete categorical labels, each neuron in the output layer represents

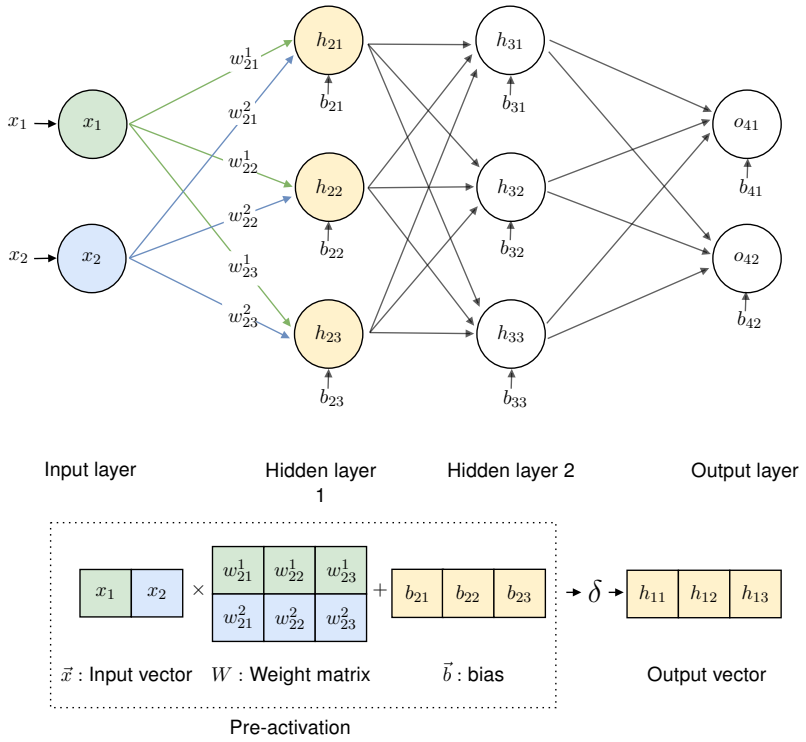


Figure 2.1: DNN architectures example. The input vector (x_1, x_2) is processed by various hidden layers. The prediction is provided as the output vector (o_{41}, o_{42}). Main operations are vector–matrix multiply ($\vec{x} \times W$) , vector–vector add ($\vec{x} \times W + \vec{b}$) and activation functions (δ).

the probability of a specific label. The regression tasks predict continuous numerical values with a single neuron in the output layer [25].

Layer	Sublayers	Operations	Sparsity	Percentage of running time
Neural Network	Weights and bias operations	Matrix-matrix multiply	Dense matrices	—
		Vector-vector add	Dense vectors	
	Activation functions	ReLU	Dense matrix	

Table 2.1: Deep Neural Network inference comprise only one layers: a fully connected deep neural network.

2.1.3.2 Requirements

DNN inference has the following requirements:

- **Real-time response:** The models typically interact with users in real-time, so the latency of processing inferences is required to deliver immediate results [26].
- **Memory bandwidth:** The bandwidth between the processing engines (CPU and GPUs) and the main memory can be the main performance bottleneck [26].
- **Compute intensity:** DNN inference requires significant compute power to perform a large number of multiply and add operations [25, 26].

- **Datatypes**: Initially, DNNs used FP32 matrices and vectors [27, 28]. To reduce the memory footprint and bandwidth and to simplify the computation, the current models use lower precision and range datatypes: FP16 [28], INT8 [28], 8 bits [29], 2 bits [30], and even 1 bit [27].

2.1.3.3 Hardware platforms

- **GPUs**: Inference on Graphics Processing Units (GPUs) benefits from several key factors. Firstly, GPUs provide high operation throughput, facilitating rapid processing of neural network computations [25, 26]. Additionally, specialized hardware features like TensorCores in NVIDIA GPUs are customized for matrix operations commonly found in DNNs [31]. Furthermore, software-accelerated libraries integrated into popular frameworks like TensorFlow, PyTorch, and NVIDIA TensorRT optimize inference workflows, enabling easier deployment and execution of DNN models on GPU platforms [32, 33, 34].
- **CPUs**: Many edge devices and embedded systems rely on CPUs for CNN inference. CPUs typically offer limited parallelism compared to GPUs, which can affect the speed and efficiency of inference tasks. However, CPUs can be more cost-effective when the inference workload is not very large or for running occasional inferences [25, 35].

Data type	Memory footprint	Production hardware platforms	Features
1-bit 2-bit 8-bit INT8 FP16 FP32	<1 GB to 10s GB	CPU + DDRx GPU + HBMx	Real time requirements High memory BW intensive Compute intensive

Table 2.2: Deep Neural Networks inference requirements and hardware platforms.

2.1.4 Training

The DNN training uses datasets of input data with correct outputs to train the network to map input data to accurate predictions or classifications. The process iteratively adjusts the network weights and biases through a series of forward and backward passes. In the forward pass, the network performs the output prediction, i.e. the DNN inference detailed in Section 2.1.3. The in backward pass or backpropagation, the model quantifies the prediction error and updates the network weights and biases in order to decrease it [21]. The DNN training is finished when network can effectively map inputs to outputs.

2.1.4.1 Steps

DNN training comprises four steps: dropout, forward pass, batch normalization and backpropagation. Their main features are presented in Table 2.3.

1. **Dropout**. To prevent the prediction overfit, before the forward pass, the dropout randomly deactivates part of the neurons in the network. The dropout rate determines the fraction of deactivated neurons, typically in the range between 20% and 50%. The output of deactivated neurons are zero, meaning that dropout introduces 20%–50% network sparsity [36].

2. **Forward pass:** Process of passing the input through the network generating the output prediction, described in Section 2.1.3.

3. **Batch normalization:** To improve computational efficiency and convergence speed, DNNs can be trained using batches of multiple input vectors. In this case, multiple vectors are processed as an input matrix that is typically normalized before the activation function. The batch normalization is performed at each DNN layer during forward pass [37].

   Figure 2.2 illustrates a normalization of batch of three 2-feature input vectors, represented as a dense 2×3 matrix. The batch normalization comprises three steps. First, it calculates the mean (μ_i) and variance (σ_i) for each feature (matrix column). Second, each matrix value is subtracted by the corresponding mean and divided by the square root of the variance [38]. Third, the matrix is scaled (multiplied by γ) and shifted (incremented by β). The main operations in the batch normalization are vector–vector add and multiply.

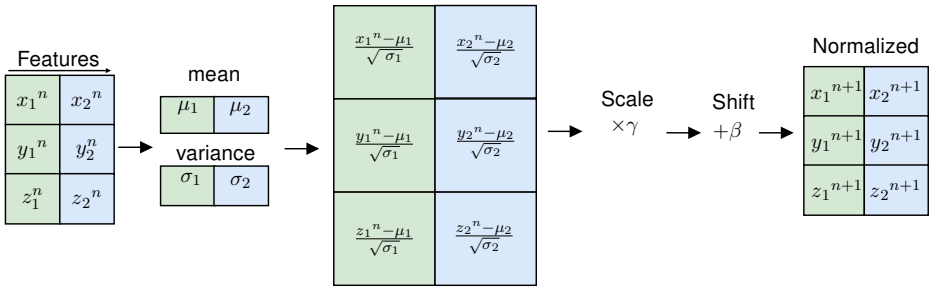


Figure 2.2: Example of a batch normalization. The input matrix is normalized based on the mean and variance of each feature (column), scaling factor γ and shifting factor β. The main operations are vector–vector add and multiply.

4. **Backpropagation:**

   After the forward pass the predictions are evaluated with a loss function, which is a difference between the predicted and actual values, see right part of Figure 2.3.[1] The loss function is also used to adjust the weights and biases in order to improve the model accuracy. We illustrate this with an example of weight w_{21}^1; the process is the same for all weights and biases. The w_{21}^1 is adjusted in two steps. First, we compute the a partial derivate of the loss function with respect to w_{21}^1 $\frac{\partial L}{\partial w_{21}^1}$.[2] Then, this partial derivative is use to update the w_{21}^1 : $w_{21}^1 = prior\_w_{21}^1 - (\frac{\partial L}{\partial w_{21}^1} \times learning\_rate)$.[3] In addition to partial derivatives computation, backpropagation performs add operations between the partial derivatives and the weights and biases vectors [38, 39].

[1] Common loss functions are Mean Squared Error (MSE), Mean Absolute Error (MAE), Huber Loss, Cross-Entropy Loss and Hinge Loss [21].

[2] The exact computation of the loss function partial derivatives exceeds the scope of this report.

[3] Learning rate represents the step size by which the model's parameters are updated during training, influencing the speed and stability of convergence towards an optimal solution [39].

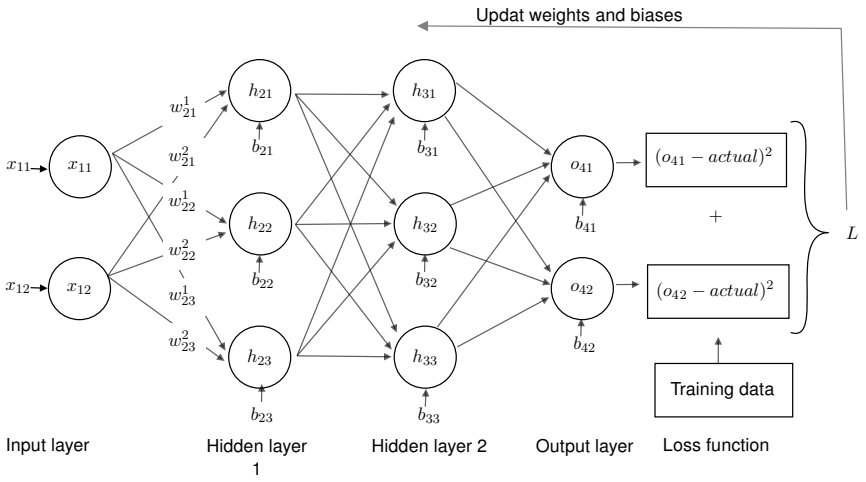


Figure 2.3: DNN backward pass example. The loss function first calculates the prediction error and then it tries to reduce it by updating weights and biases through backpropagation.

Step	Operations	Sparsity
Dropout	Random binary vector generation	Dense vector
	Vector-vector add	Dense vectors
Batch normalization	Vector-vector add	Dense vector
	Vector-vector multiply	Dense vectors
Forward pass	All inference operations	—
Backpropagation	Vector-vector add	Dense vector
	Partial derivatives over a vector	Dense vector

Table 2.3: Deep Neural Networks training comprise four main steps: the dropout, batch normalization, forward pass and backpropagation steps.

2.1.4.2 Requirements

DNN training is a task that often takes days to weeks [40] and has the following requirements:

- **Memory bandwidth:** Bandwidth-demanding task, particularly in fully connected layers, where is necessary to load the output vector from the previous layer along with the weights [26].
- **Memory footprint:** Academic research reports DNN models sizes in the range of hundreds of megabytes. Industrial papers report models that do not fit in the commercial HBM devices, indicating model sizes in the order of tens or hunders of gigabytes [3, 41, 42].
- **Datatypes:** Compared to inference, requires higher precision, such as FP32 and FP16 or BF16 [35, 43, 44]. Mixed precision between, FP32 and FP16 can be used to balance accuracy and computational efficiency [45].

2.1.4.3 Hardware platform

- **Multi-GPU:** GPUs enable highly-parallel processing, which reduces training time and accelerates model convergence [40]. Large production DNNs exceed the memory capacity of a single GPU [46], so they are trained in multi-GPU systems. This leads to large data movements and synchronization overhead and impacts the overall training efficiency and scalability [46].

Data type	Memory footprint	Production hardware platforms	Features
BF16 FP16 FP32	<1 GB to 10 GB	100s GPUs + HBMx	Memory footprint Memory bandwidth Scalability

Table 2.4: Deep Neural Networks training requirements and hardware platforms.

2.2 Deep Learning Recommendation Systems

2.2.1 Motivation

Deep learning recommendation systems (DLRS) are widely used by the industry to provide suggestions or recommendations of items such as advertisements, products, movies, etc., that are most relevant to a certain user:

- **Netflix:** Shows movie options [6, 7, 47]
- **Amazon:** Orders items in the catalog [6, 7, 8]
- **Google:** Provides personalized advertisements [6] and app recommendations [9]
- **Meta:** Ranks news feeds and search results [6]
- **Alibaba:** Provides product recommendations [4, 48, 49]
- **YouTube:** Suggests videos based on user preferences [7, 8, 9, 47]
- **Ebay:** Provides product recommendations [8]
- **LinkedIn:** Ranks news feeds and search results [8]
- **Spotify:** Provides music recommendations [8]
- **Pinterest:** Displays image recommendations [8]

2.2.2 Overview

DLRS are widely used by the industry, representing up to 70% of the machine learning inference cycles in data centers from Meta, Google, Alibaba, or Amazon [47, 50, 51]. Although each implementation is adapted to a specific use-case, they all share a common base which is discussed in this document.

2.2.3 Inference

To provide recommendations, DLRS use descriptions of the products, users, movies, etc. These descriptions can be *dense features*, characteristics represented as numerical values e.g., price, age, dimensions, etc., or *sparse features* characteristics represented as categorical values, e.g., type of device, interaction of users with products, etc. These descriptions are processed to get the probability of a user clicking on a product, advertisement, another user profile, and similar [4, 9, 47, 48, 49, 52, 53, 54].

2.2.3.1 Layers

DLRS inference comprise two layers: an embedding and a neural network layer. Their main features are presented in Table 2.5 and the structure is shown in Figure 2.4.

- **Embedding:** Converts sparse features to numerical vectors that capture their meaning called *embedding vectors*. The vectors values are computed during training and stored in memory in *embedding tables* for access during inference [4, 47, 48, 49].
  - **Embedding lookup:** Each sparse feature is converted into an embedding vector using the embedding tables stored in memory [4, 47, 48, 49].

  - **Lookup aggregation:** The output embedding vectors are combined, performing dense vector–vector add or vector–vector concatenate [4, 47, 48].

- **Neural network** returns the probability of a user clicking on an item, product, advertisement, etc. The neural network processes the dense features and the output of the embedding layer together to capture patterns and relationships, as explained in Section 2.1.2.

  - **Weights and bias operations:** Applying dense matrix–matrix multiply for the weights operations and dense vector–vector add for the bias operations.
  - **Activation:** The result of the weights and bias operation goes through an activation function like ReLU[47].

Layer	Sublayers	Operations	Sparsity	Percentage of running time
Embedding	Embedding lookup	Load form memory	—	<10 % - 80 %
	Lookup aggregation	Vector-vector add or Vector-vector concat	Dense vector	
Neural Network	Weights and bias operations	Matrix-matrix multiply	Dense matrices	<20 % - 90 %
		Vector-vector add	Dense vectors	
	Activation functions	ReLU	Dense matrix	

Table 2.5: Deep Learning Recommendation Systems inference comprise two layers: an embedding and a neural network layer.

The organization of these layers is shown in Figure 2.4.

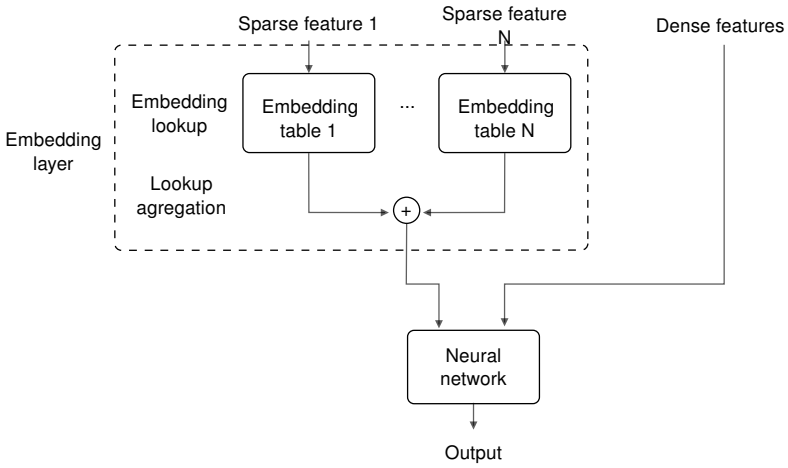


Figure 2.4: Deep Learning Recommendation System architecture example. First, in the embedding layer, the sparse features are converted into numerical vectors (embedding lookup) and combined together (lookup aggregation). Second, the neutral network processes the embedding layer outcome and the dense features.

Given that each DLRS implementation its adapted to a specific use case, the structure shown in Figure 2.4 can vary. Some common variations are:

- **Activation:** The most common activation functions used in the neural network are ReLU [9, 47, 53], PReLU [4, 48, 49], Sigmoid [48].
- **Layers:** The number of layers can increase depending on the use case. For example, an RNN layer between the embedding layer and the neural network can be used to process the preferences of users over time [48].
- **Neural network embedding:** In the embedding layer a neural network computes the embedding vectors during inference, instead of accessing values stored in embedding tables [9, 47, 53].

2.2.3.2 Requirements

The requirements can vary depending on the use case and structure of the DLRS. The most common ones are:

- **Memory footprint:** The size of the model can vary form tens and hundreds of gigabytes [55] up to terabytes [6, 50].
- **Bandwidth needs:** Multiple entries of several embedding tables located in main memory have to be accessed simultaneously to translate sparse features to the numerical vectors, leading to a high memory bandwidth need [50, 56, 57].
- **Compute intensive:** Neural networks matrix–matrix multiplications and vector–vector adds are compute driven and can be parallelized.[6, 47]
- **Real time requirements:** The models typically interact with users in real-time, so the latency of processing inferences is required to deliver immediate results [4, 9, 47, 47, 53, 58].
- **Data types:** Parameters in the neural network and embedding vectors are stored in INT32, FP32, FP16, or BF16 data types [6, 9, 47, 58, 59]. Lower INT4, INT8 can be used to reduce the model's memory footprint of the embedding layer [58, 59]. Some models report that parameters in the embedding vector occupy 1-4 bytes, without specifying the data types [60]. These data types can be combined, storing the most used parameters in FP32 and the less accessed in lower precisions [6, 61].

2.2.3.3 Hardware platforms

The hardware platform can vary depending on the type of features processed and the structure of the model:

- **CPUs:** The model spends most of the execution time in the embedding layer where the number of operations per byte is low [(4)]. The use of CPU is more cost effective than GPUs response latency is not critical [47, 56].
- **GPUs:** The model spends most of the execution time in the neural network layer. Most of the execution time is spent in computations [50, 56, 58].

[(4)] The number of operations done with each acceded embedding vector is low.

- **Proprietary accelerators** reduce power consumption while executing models with high need of memory footprint, bandwidth, and computational power, One example is Meta Training and Inference Accelerator (MTIA) [50]. These accelerators are also focused on increasing flexibility, so they can be used in future models with different requirements [50, 59].

DLRMs main requirements and hardware platforms are summarized in Table 2.12

Data type	Memory footprint	Production hardware platforms	Features
1-4 byte INT4 INT8 INT32 BF16 FP16 FP32	<10 TB	<10 GPUs + HBMx 12 MTIA + LPDDR5 <10 CPU	Real time requirements Neural network compute intensive Embedding light compute and high memory capacity and bandwidth needs

Table 2.6: Deep Learning Recommendation Systems' inference requirements and hardware platforms.

2.2.4 Training

DLRS training uses input datasets with correct outputs to train the model. The process iteratively adjusts the weights and biases through a series of forward and backward passes in the embedding and neural network layers. In the forward pass, the model performs the output prediction, i.e. the DLRS inference detailed in Section 2.2.3. In the backward pass the model quantifies the prediction error and updates the weights and biases to decrease it [58].

2.2.4.1 Steps

DLRS training comprises two steps: forward pass and backward pass.

1. **Forward pass:** Process of passing the input through the different layers generating the output, described in Section 2.2.3.
2. **Backpropagation:** After the forward pass, backpropagation adjust the parameters by reducing the difference between the predicted and actual output values. This involves performing dense vector–vector adds and computing partial derivatives over a vector. The updated parameters include, the weights and biases in the neural network layer, and the numerical values of the embedding vectors [6, 52, 58, 61].

To prevent overfitting, some models add a **dropout** step which sets to zero parameters in the neural network or the embedding layer. This involves dense random binary vector generation and dense vector–vector multiply [4, 49, 62].

Step	Operations	Sparsity
Forward pass	All inference layers	All inference operations
Backpropagation	Vector-vector add	Dense vector
	Partial derivatives over a vector	Dense vector

Table 2.7: Deep Learning Recommendation Systems training comprise two steps: a forward pass and and a backpropagation steps.

2.2.4.2 Requirements

The requirements can vary depending on the use case and structure of the DLRS. The most common ones are:

- **Memory footprint:** The size of the model can vary from tens and hundreds of gigabytes [55] up to terabytes [6, 50]. This size is multiplied by a factor of 4 during training because of the intermediate values that software optimizers use to improve training speed and results [4, 63]. Also, the size of the datasets are in the range of several petabytes, requiring distributed network storage [6].
- **Bandwidth needs:** Multiple training inputs are process together to accelerate training [6, 57]. To process this inputs multiple parameters of the embedding layer and the neural network has to be accessed from main memory [8, 59, 60].
- **Compute intensive:** Neural networks matrix–matrix multiplications and vector–vector adds are compute bound and can be parallelized [6, 47].
- **Data types:** Parameters in the neural network and embedding vectors are stored in INT32, FP32, FP16, or BF16 data types [6, 9, 47, 58, 59]. Lower INT4, INT8 can be used to reduce the model's memory footprint of the embedding layer [58, 59]. Some models report that parameters in the embedding vector occupy 1-4 bytes, without specifying the data types [60]. These data types can be combined, storing the most used parameters in FP32 and the less accessed in lower precisions [6, 61].

2.2.4.3 Hardware platforms

The hardware platforms can vary depending on the type of features processed and the structure of the model:

- **CPUs:** Traditional training is distributed across multiple CPUs. This is done by partitioning the embedding tables to increase the memory capacity and aggregated bandwidth. Also the model can be replicated, each replica will be trained independently with different inputs, later all replicas parameters synchronize together to get the final model [6].
- **GPUs:** Aggregate compute power and memory bandwidth of multiple (tens of) GPUs is used when the model's execution time is dominated by the neural network.
- **Proprietary accelerators** reduce power consumption while executing models with high need of memory footprint, bandwidth, and computational power, One example is Meta Training and Inference Accelerator (MTIA) [50]. These accelerators are also focused on increasing flexibility, so they can be used in future models with different requirements [50, 59].

Data type	**Memory footprint**	**Production hardware platforms**	**Features**
1-4 byte INT4 INT8 INT32 BF16 FP16 FP32	<10 TB	10s GPUs + HBMx	Neural network compute intensive Embedding light compute and high memory capacity and bandwidth needs

Table 2.8: Deep Learning Recommendation Systems training requirements and hardware platforms.

2.3 Recurrent Neural Networks

2.3.1 Motivation

Recurrent Neural Networks (RNN) track relationships and learn from sequential data, sequences of elements whose meaning is determined by the elements' values and their order. Examples are words in the text, pixels in images or frames in videos. RNN are mainly used in the industry for language processing (NLP) and speech recognition [64].

- **Google:** Natural language processing (NLP), Translation, Speech recognition, Autonomous driving [10, 11, 12, 13]
- **Baidu:** Speech recognition [5]
- **Amazon:** Speech recognition [14]

2.3.2 Overview

RNN neurons have a *recursive connection* that allows them to compute relations between elements in the input sequence. Figure 2.5a shows the structure of a single RNN neuron. The sequence is processed in multiple time steps (t), at each time step one element of the sequence (x_t) is processed to compute two values: one *activation value* (a_t) and one *output value* (y_t). The output value is part of the final output, and the activation value is used in the computations of the next time step ($t+1$). This way, on each time step t, each input x_t is processed using the activation value generated in the previous time step a_{t-1}. This produces one activation value a_t and one output value y_t. Figure 2.5b shows the representation of this neuron through time, processing each element in the input sequence with the activation value of the previous time step and generating one output and the new activation. Figure 2.5c shows the compact representation of this RNN neuron [64, 65].

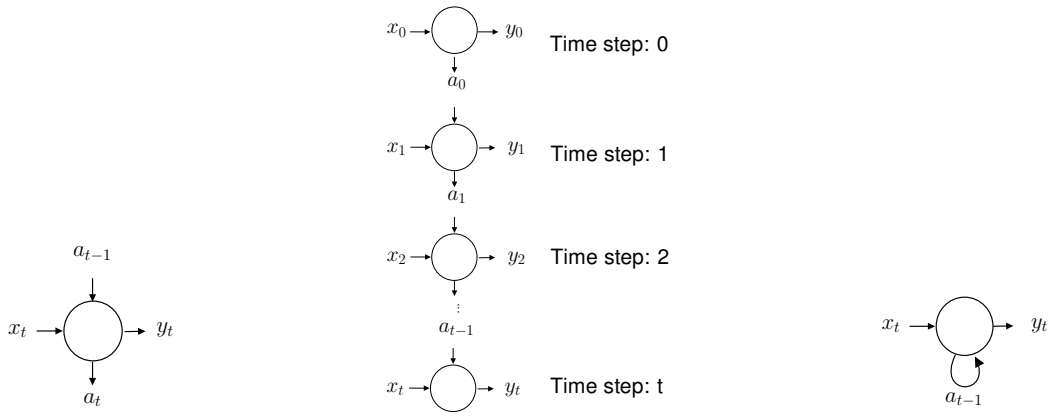


Figure 2.5: (a) RNN neuron structure, at each time step t one element of the sequence x_t is processed using the activation value generated in the previous time step a_{t-1} to get one activation value a_t and one output value y_t. (b) Representation of an RNN neuron through time, processing one input with the activation value of the previous time step and generating one output and the new activation. (c) Compact representation of an RNN neuron.

Figure 2.6 shows the internal structure of an RNN neuron. This neurons use the same concepts of weights, bias and activation function explain in Section 2.1.2. To produce the activation value a_t and output value y_t on each time step the neuron executes two phases [64, 65]:

- **Activation computation:** The input element x_t and the activation value from the previous time step a_{t-1} are multiplied by two different weights W_x and W_a. The result of both multiplications is added with a bias b_1 and then process by an activation function δ generating a new activation value a_t. This output activation a_t is used as the input activation in the next time step.
- **Output computation:** The output of the activation computation phase is again multiplied by a weight W_y, added with a bias b_2 and process by another activation function δ to get the neuron's output y_t.

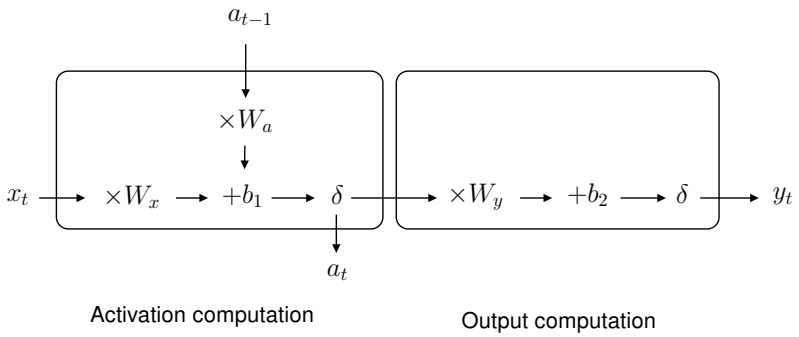


Figure 2.6: RNN neuron internal structure divided in two phases: the activation computation and the output computation.

2.3.3 Inference

2.3.3.1 Layers

RNN comprise two layers: an embedding and a recurrent layer. Their main features are presented in Table 2.9 and the structure is shown in Figure 2.7.

- **Embedding:** Each element in a sequence is substituted by an *embedding vector*, a numerical vector that captures its meaning. The vector values are computed during training and stored in memory for access during inference [10, 66, 67, 68, 69, 70, 70]
- **Recurrent layers:** Multiple RNN neurons process sequentially the output of the embedding layer. Each neuron performs the activation computation and the output computation phases. This phases capture patterns and relations within the elements in the sequence to generate the output [64, 65]. This implies:
  - **Weights and bias operations:** Applying dense vector–matrix multiply for the weights operations and dense vector–vector add for the bias operations.
  - **Activation:** The result of the weights and bias operation goes through an activation function like sigmoid or ReLU [65, 71].

Layer	Sublayers	Operations	Sparsity	Percentage of running time
Embedding		Load from memory	—	—
Recurrent Neurons	Weights and bias operations	Vector-matrix multiply	Dense vector. Dense matrix	—
		Vector-vector add	Dense vectors	—
	Activation function	ReLU, Sigmoid or Tanh	Dense vector	—

Table 2.9: Recurrent Neural Networks comprises two layer: an embedding layer and layer composed of recurrent neurons.

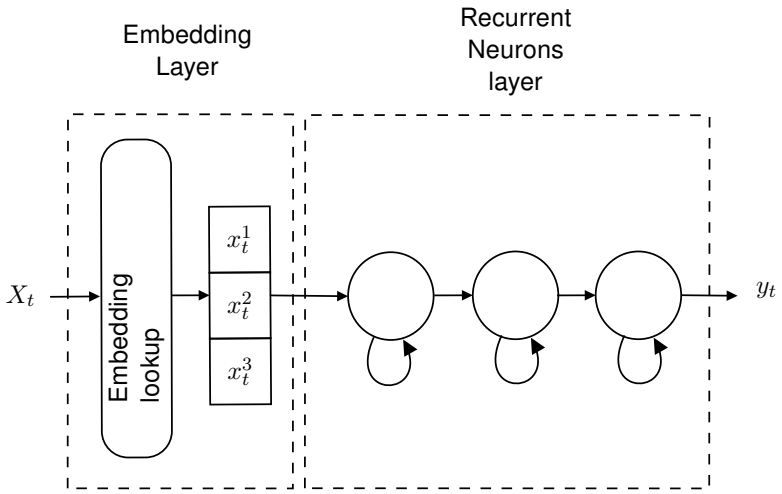


Figure 2.7: Recurrent Neural Networks architecture comprising an embedding layer and a layer composed of recurrent neurons.

The structure of the network and the operations inside the neuron shown in Figure 2.7 and Figure 2.6 can vary depending on the use case. Some common variations are:

- **Activation:** The most common activations are ReLU, sigmoid and tanh [67, 71, 72]. Some models use other linear activations to improve models scalability [10, 11].
- **Layers:** The number and order of the layers may change depending on the use case, e.g. adding transformer's attention layers [10] or convolution layers [73] to improve the model's scalability.
- **Neuron operation:** more vector–vector element-wise multiplications and vector–vector additions can be added to improve the neuron capability of making relations between elements in the sequence [64, 65].

RNNs can also be used to improve other models' capability to compute relations between elements in the input sequence, e.g., improve the understanding of users' interactions with products in deep learning recommendation systems [48].

2.3.3.2 Requirements

- **Sequentiality:** The elements of the input sequence have to be processed sequentially, limiting the scalability [11, 67, 74].

- **Real-time requirements:** The models typically interact with users in real-time, so the latency of processing inferences is required to deliver immediate results [10, 72, 75].
- **Memory footprint:** The size of the model can vary between tens and hundreds of megabytes [76, 77, 78].
- **Data types:** Small models use FP32 data type to store parameters and perform operations. To reduce inference latency, lower precision INT8, INT16, and BF16 data types are used to perform operations in larger models [10, 72].

2.3.3.3 Hardware platforms

Models are generally executed in one TPU or GPU to meet the explained requirements. [10, 11, 67, 72] RNN's main requirements and hardware platforms are summarized in Table 2.10

Data type	Memory footprint	Production hardware platforms	Features
INT8 FP32 FP16 BF16	10s MB to 100s MB	1 GPU + HBMx 1 TPU + HBMx	Sequential execution dependencies Real time requirements

Table 2.10: Recurrent Neural Networks' inference requirements and hardware platforms.

2.3.4 Training

RNN training uses input datasets with correct outputs to train the model. The process iteratively adjusts the weights and biases through a series of forward and backward passes in the embedding and recurrent layers. In the forward pass, the model performs the output prediction, i.e. the RNN inference detailed in Section 2.3.3. In the backward pass, the model quantifies the prediction error and updates the weights and biases to decrease it [79].

2.3.4.1 Steps

RNN training comprises two steps: forward pass and backward pass through time.

1. **Forward pass:** Process of passing the elements in the input sequence through the different layers generating the output, described in Section 2.3.3.
2. **Backpropagation through time:** After the forward pass, for each element processed, backpropagation through time adjust the parameters by reducing the difference between the predicted and the actual output values. This involves performing dense vector–vector adds and computing partial derivatives over a vector. The updated parameters include, the weights and biases in the recurrent layer, and the numerical values of the embedding vectors [10, 67, 68, 69, 79].

To prevent overfitting, some models add a dropout step which sets the output of some neurons to zero. This involves dense random binary vector generation and dense vector–vector multiply [62].

Step	Operations	Sparsity
Forward pass	All inference layers	—
Backpropagation through time	Vector-vector add	Dense vectors
	Partial derivatives over a vector	Dense vector

Table 2.11: Recurrent Neural Networks' training comprise two steps: a forward pass and and a backpropagation through time steps.

2.3.4.2 Requirements

The requirements can vary depending on the use case and structure of the RNN. The most common ones are:

- **Scalability:** Elements in the sequences must be processed in order during training. The models can be parallelized in a pipeline manner, distributing layers among devices. Multiple instances of the same model can be trained in parallel with different input data; After training, all the instances synchronize to get the final model [71, 72]. Some models use linear activations to improve the parallelism of computations, processing elements in the sequence independently [10, 11].
- **Data types:** To avoid loss of accuracy, FP32 data types are used during training [72]. In large models, BF16 are used to reduce data transfer overhead of parameters and auxiliary variables used during training [10].

2.3.4.3 Hardware platforms

RNN models are generally trained on one or multiple GPUs or TPUs depending on the explained requirements.

RNNs' main requirements and hardware platforms are summarized in Table 2.12.

Data type	Memory footprint	Production hardware platforms	Features
FP32 BF16	—	1 TPU + HBMx 10s GPU + HBMx	Can exploit model and data parallelism

Table 2.12: Recurrent Neural Networks' training requirements and hardware platforms.

2.4 Transformers

2.4.1 Motivation

Transformers track relationships and learn from sequential data, sequences of elements whose meaning is determined by the elements' values and their order. Examples are words in the text, pixels in images or frames in videos. Transformers are widely used by the industry for natural language processing (NLP), image, audio and video processing [80, 81, 82, 83, 84, 85, 86, 87]:

- **Google:** NLP [80, 88, 89, 90, 91], video analysis [80], image analysis [80], audio processing [80], image generation [80] and protein structure prediction [92]
- **OpenAI:** NLP [83, 84, 85, 86], image generation [87] and image analysis [86]
- **Facebook:** NLP [82, 93, 94, 95]
- **Microsoft:** NLP [63] and image generation [87]

2.4.2 Overview

Transformer models were initially used to translate and summarize text. Currently they have many applications, including text generation, image and video processing. Although the model implementations are adjusted to each application domain they all share a common base which is discussed in this document.

2.4.3 Inference

The sequence elements are called tokens, e.g. words in texts, pixels in images, or frames in videos. In general, multiple tokens are processed in parallel during inference [83, 84, 85, 88, 89, 93].

2.4.3.1 Layers

Transformers inference comprise seven layers: embedding, positional encoding, multi-head attention, concatenation, linear, neural network and normalization. Their main features are presented in Table 2.13, and the structure is shown in Figure 2.8.

1. **Embedding:** Each token in a sequence is substituted by an *embedding vector*, a numerical vector that captures its meaning. This process results in matrix where each row is an embedding vector. The vectors values are computed during training and stored in memory for access during inference [63, 81, 82, 83, 84, 85, 88, 89, 91, 93, 94, 95].
2. **Positional encoding:** Adds to each embedding vector a positional vector (dense vector–vector addition) with information about the position of the tokens in the sequence. The values that compose each positional vector are generally computed during training and stored in memory for access during inference [82, 83, 84, 85, 89, 93, 94, 95].
3. **Multi-head attention:** Computes values that represent the relations between the tokens in the sequence [88]. Depending on the implementation, this layer can perform *dense attention* that computes the relation between all pairs of tokens or *sparse attention* which only computes a subset of tokens pairs. This layer is executed various times in parallel, as shown in Figure 2.8a. Each parallel execution comprises four operations: linear, query-key multiply, softmax and probability-value multiply, see Figure 2.8b.

i **Linear:** The input matrix, composed of the embedding vectors, is replicated three times into three different matrices called *query*, *key*, and *value*. Each matrix goes through a different linear transformation involving a dense matrix–matrix multiply and optionally a dense matrix–matrix addition.

ii **Query–key multiply:** With dense attention, this layer performs a dense matrix–matrix multiply of query and key. With sparse attention, different sparse patterns are introduce in key making some values of the matrix zero, so the operation would be a dense matrix–sparse matrix multiply.

iii **Softmax:** All the elements in the resulting matrix are normalized into values between 0 and 1 with the softmax function [83, 84, 85, 89]. For each row, we raise Euler's constant e to each element (e^{vector}). Then, we compute the sum of all the elements in the row (dense vector to scalar reduction) and divide each element by this sum (dense vector-scalar division). This will give us a probability matrix of values between 0 and 1.

iv **Probability–value multiply:** The probability matrix is then multiplied by the projected value matrix. If the model uses dense attention, both the probability matrix and the projected value matrix are dense, so the operation would be a dense matrix–matrix multiply. If the model uses sparse attention, both the probability matrix and the projected value matrix are sparse, so the operation would be a sparse matrix-sparse matrix multiply.

4. **Concatenation:** The output matrices of the multi-head attention layers are concatenated by rows (vector–vector concat).

5. **Linear:** The concatenated matrix goes through a linear transformation involving a dense matrix–matrix multiply and optionally a dense matrix–matrix addition.

6. **Neural network** processes the output of the multi-head attention layer to capture patterns and relationships within the data (see Section 2.1.2). The number of hidden layers is typically small between two [83, 84, 88] and three [82, 94, 95]. Each layer applies:

   - **Weights and bias operations:** Applying dense matrix–matrix multiply for the weights operations and dense vector–vector add for the bias operations.
   - **Activation:** The result of the weights and bias operation goes through an activation function like ReLU[88].

7. **Normalization:** The output of the neural network is then normalized by rows [83, 84, 85, 88, 91, 96]. For each element in a row, we subtract the mean and divide by the standard deviation. This implies mainly dense vector–vector multiply and dense vector–vector add.

Layer	Sublayers	Operations	Sparsity	Percentage of running time
Embedding		Load from memory	—	—
Positional encoding		Vector-vector add	Dense vectors	—
Multi-head Attention	Linear	Matrix-matrix multiply	Dense matrices	5 % – 35 %
		Matrix-matrix add	Dense matrix. Dense or sparse matrix	
	Query-key multiply	Matrix-matrix multiply	Dense matrix. Sparse or dense matrix	5 % – 25 %
	Softmax	e^{vector}[5]	Dense vector	5 % – 40 %
		Vector to scalar reduction	Dense vector	
		Vector-scalar divide	Dense vector	
	Probability-value multiply	Matrix-matrix multiply	Sparse or dense matrix Sparse or dense matrix	5 % – 25 %
Concatenation		Vector-vector concat	Dense vector	—
Linear		Matrix-matrix multiply	Dense matrices	5 % – 35 %
		Matrix-matrix add	Dense matrix. Dense or sparse matrix	
Neural network	Weights and bias operations	Matrix-matrix multiply	Dense matrices	10 % – 55 %
		Vector-vector add	Dense vectors	
	Activation function	ReLU	—	
Normalization		Vector-vector add	Dense vectors	<1 %
		Vector-vector multiply	Dense vectors	

Table 2.13: Transformers' inference comprise seven layers: embedding, positional encoding, multi-head attention, concatenation, linear, neural network and normalization.

The organization of these layers is shown in Figure 2.8a. The input sequence goes through the embedding layer to get the embedding vectors. Then, a positional encoding is added to this embedding vectors. After this, the output of the positional encoding is processed by multiple sequential blocks comprising multiple multi-head attention layers, and the concatenation, linear and neural network layers. The multi-head attention applies the operations shown in Figure 2.8b various times in parallel. This way the value, key and query matrices are linearly projected multiple times and for each projection we compute the query–key multiply, softmax and probability–value

[5] $e \approx 2.71828$ (Euler's constant)

multiply operations as explained. The outputs of the parallel layers are concatenated to get one matrix then processed by the linear an neural network layers. The result of the sequential layers goes through a normalization layer to get the final output.

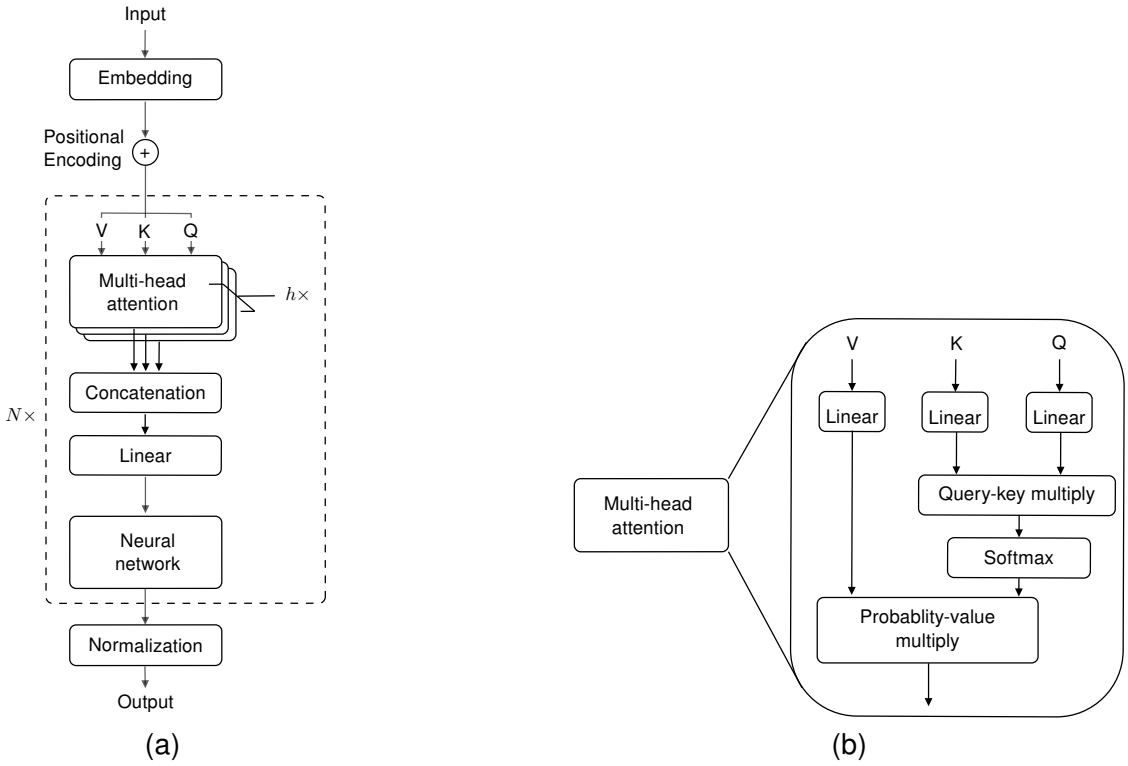


Figure 2.8: (a) Transformer structure with N sequential blocks and h parallel multi-head attention layers. The input sequence is processed by the embedding layer to get the embedding vectors. Then, a positional encoding is added to this embedding vectors. The output of the positional encoding is processed by multiple sequential blocks comprising multiple multi-head attention layers, and the concatenation, linear and neural network layers. The result of the sequential layers goes through a normalization layer to get the final output. (b) Multi-head attention layer comprising the linear, query–key multiply, softmax and probability–value multiply layers.

Given the wide range of transformers models and use cases, the structure shown in Figure 2.8 can vary. Some common variations are:

- **Multi-query attention:** to reduce the memory bandwidth, key and value linear projections are shared between all parallel multi-head attention layers [81, 91, 97].
- **Position of the normalization layer:** Depending on the model, the position of the normalization layer can vary.
  - Before the multi-head attention and neural network layers [81, 83, 85].
  - After the linear projection and neural network layers [84, 89].
  - Before the multi-head attention and neural network layers, and after the linear and neural network layers [98].
- **Activation:** The most common activation functions are: ReLU [88], SwiGLU [82, 91, 94, 95], GeLU [84, 89, 93, 99], and GeGLU [81]
- **Layers:** The number of layers may increase depending on the use case. For example, an additional softmax layer may be added to process the output for text generation [88].

- **Mixture of experts:** To reduce the number of parameters used during inference of the largest models, the neural network is divided into multiple sub-networks called *experts*. A new neural network called *router* decides which expert is going to be used to process the output of the multi-head attention [100, 101].

2.4.3.2 Requirements

The requirements can vary depending on the use case and structure of the transformer. The most common ones are:

- **Memory footprint:** The number of parameters composing the embedding and positional encoding layers, the linear transformations, and the neural network layer can reach the hundreds of billions. This way, the size of the model can vary from tens and hundreds of gigabytes [81, 82, 85, 94, 94], up to terabytes [63, 86, 90, 91, 101].

- **Bandwidth needs:** The number of operations per byte is low in comparison with the bandwidth requirements to access all the parameters of the model. The largest models require aggregate memory bandwidth across multiple devices [5, 85, 102].

- **Good scalability:** The main operations across layers are matrix–matrix multiply, which can be parallelized on one or more devices.

- **Real time requirements:** The models typically interact with users in real-time, so the latency of processing inferences is required to deliver immediate results [80, 85, 86, 90, 91, 101].

- **Data types:** Different data types can be used to store the weights and perform computations. Typically, older models store weights in FP32 [63, 83, 84, 89, 93], while larger and newer models use FP16 [98, 103] or BF16 [81, 82, 95]. Operations can be adjusted also to use FP32 [63, 81, 83, 84, 89, 93], FP16 or BF16 [82, 95]. In some models, when there is memory capacity limitations, the weights and operations can be done in lower 8 or 4 bits data types [81].

2.4.3.3 Hardware platforms

Transformers have good scalability, so they are generally executed on one or multiple GPUs or TPUs depending on the explained requirements. [5, 104]

Transformers' main requirements and hardware platforms are summarized in Table 2.16

Data type	Memory footprint	Production hardware platforms	Features
FP32 FP16 BF16	100 GB to TBs	10s GPUs + HBMx 10s TPUs + HBMx	High memory BW requirements Real time requirements Good scalability

Table 2.14: Transformers' inference requirements and hardware platforms.

2.4.4 Training

Transformers training uses input datasets with correct outputs to train the model. The process iteratively adjusts the weights and biases through a series of forward and backward passes in the embedding, positional encoding, multi-head attention, and neural network layers. In the forward pass, the model performs the output prediction, i.e. the transformers inference detailed in Section 2.4.3. In the backward pass, or backpropagation, the model quantifies the prediction error and updates the weights and biases to decrease it. Training is divided into two phases:

1. **Pre-training:** Trains the model a general understanding of how tokens are organized in a sequence, e.g., common word orders and grammatical structures in texts. During this phase, the model is given sequences with missing tokens that the model tries to predict using the surrounding tokens [80, 83, 84, 85, 86, 89, 101].

2. **Fine-tuning:** Adjusts the parameters of the pre-trained model for specific tasks or domains, e.g., English to Spanish translation. The model is trained using labeled data of the specific task to learn, e.g., pairs of English and Spanish sentences with the same meaning [80, 86, 90, 91, 101].

2.4.4.1 Steps

Transformers training comprises three steps in both pre-training and fine-tuning phases: dropout, forward pass, and backward pass. Their main functions are presented in Table 2.15.

1. **Dropout:** To prevent overfitting, some values in the network are set to zero during training. This involves dense random binary vector generation and dense vector–vector multiply. Depending on the implementation, dropout can be applied to:

   - Output of the multi-head attention layer and the neural network [88]
   - Multi-head attention operations, dropping a percentage of the elements during the matrix–matrix multiply [84]
   - Fine-tuning phase only [91]
   - Linear transformations and neural network layers only [98]

2. **Forward pass:** Process of passing the input through the different layers generating the output, described in Section 2.4.3.

3. **Backpropagation:** After the forward pass, the difference between the output values obtained and the correct values is used to adjust the parameters. This involves performing dense vector–vector adds and computing partial derivatives over a vector. The updated parameters include, the weights and biases in the neural network layer, the linear transformations in the multi-head attention layer, and the numerical values of the embedding and positional vectors [82, 83, 84, 85, 89, 93, 94, 95].

Step	Operations	Sparsity
Dropout	Random binary vector generation	Dense vector
	Vector-vector add	Dense vectors
Forward pass	All inference layers	—
Backpropagation	Vector-vector add	Dense vector
	Partial derivatives over a vector	Dense vector

Table 2.15: Transformers' training comprise three steps: dropout, forward pass and backward pass.

2.4.4.2 Requirements

The requirements can vary depending on the use case and structure of the transformer. The most common ones are:

- **Memory footprint:** The size of the model can reach up to terabytes [63, 86, 90, 91, 101], as explained in Section 2.4.3.2. This size is multiplied by a factor of 4 during training because of the intermediate values that software optimizers use to improve training speed and results [82, 84, 85, 89, 91, 94, 95]. The size of the datasets has also increased up to trillions of tokens [82, 94, 95].
- **Memory bandwidth:** The number of operations per byte is low in comparison with the bandwidth requirements to access all the model's parameters and intermediate values of the optimizer [5, 85, 102]. The largest models require aggregate memory bandwidth across multiple devices (CPUs, TPUs, etc.).
- **Data types:** Multiple data types are used simultaneously. Generally, values are stored in FP16 or BF16, also keeping a copy in FP32. Operations can be performed in FP32, BF16 or FP16 [63, 81, 82, 94, 95, 98].
- **Good scalability:** The main operations across layers are matrix–matrix multiply, which can be parallelized on one or more devices [105, 106].

2.4.4.3 Hardware platforms

Training is generally done in multiple GPUs or TPUs to exploit the aggregated memory bandwidth and computational power [80, 85, 88, 91, 98, 101].

Transformers' main requirements and hardware platforms are summarized in Table 2.16

Data type	Memory footprint	Production hardware platforms	Features
FP32 FP16 BF16	10s TB	1000s GPUs + HBMx 1000s TPUs + HBMx	High memory BW requirements Good scalability

Table 2.16: Transformers' training requirements and hardware platforms.

2.5 Summary

This chapter **analysed four major types of Artificial Intelligence (AI) models** commonly used in the industry. The models studied were Deep Neural Networks (DNNs), Deep Learning Recommendation Systems (DLRS), Recurrent Neural Networks (RNNs), and Transformer models, particularly large language models (LLMs) architecture. Each model type was evaluated based on its industrial adoption, internal structure, and hardware requirements. All relevant data and comparisons are available on the FAiNDER platform [1], which provides an interactive overview of the analysed models.

Deep Neural Networks (DNNs) are the foundational building blocks of many modern AI applications. They consist of multiple layers of artificial neurons connected by weights and biases, and are used in tasks such as speech recognition, computer vision, and healthcare. Inference in DNNs demands low latency, high memory bandwidth, and compute-intensive operations like matrix multiplications and activations (e.g., ReLU). Training is even more demanding, requiring iterative updates to weights and biases over large datasets. This process is typically performed using higher precision formats like FP32 or mixed precision (e.g., FP16), and often involves multi-GPU systems due to large memory needs.

Deep Learning Recommendation Systems (DLRS) are heavily used by companies like Netflix, Amazon, and Meta to provide personalised content. These systems handle both dense and sparse input data using embedding layers and neural network layers. Inference requirements include large memory footprints and high bandwidth access to embedding tables, as well as compute-intensive neural network layers. The dominant hardware depends on the layer that takes most execution time: CPUs are used when memory access dominates, while GPUs or specialised accelerators are chosen for compute-heavy workloads. Training DLRS models involves handling massive datasets and complex memory patterns, often requiring distributed systems or hybrid hardware.

Recurrent Neural Networks (RNNs) are designed to process sequential data by maintaining information across time steps. They have been widely used in speech and language tasks and are characterised by their recursive structure. Inference is performed step-by-step, which limits scalability but allows for accurate sequence tracking. Memory and compute requirements are moderate, and smaller models may use FP32, while larger models benefit from lower precision to improve latency. Training involves a process called backpropagation through time, which updates the model parameters after each sequence step. Although inherently sequential, RNNs can benefit from certain parallelisation techniques.

Transformer models, particularly large language models (LLMs), have become the dominant architecture in natural language processing and other AI domains. These models process tokens in parallel using multi-head attention and feed-forward layers, and are highly scalable. Inference involves high memory and bandwidth requirements due to the large number of parameters and operations with low operational intensity. Execution typically occurs across multiple GPUs or TPUs. Training is performed in two stages—pre-training and fine-tuning—and involves extremely large datasets and high precision arithmetic. The training process is memory- and bandwidth-intensive and relies on mixed precision formats (e.g., FP32, FP16, BF16) to improve efficiency.

This analysis shows the diverse structural and performance characteristics of the four main AI model types. Among them, **transformer-based large language models present the most complex** and demanding performance challenges. Their widespread use and scalability justify their selection for the experiments in the following chapters. The FAiNDER platform remains a useful tool for navigating and comparing these models across industry applications.

Chapter 3

Llama 3 8B profiling

In this chapter, we focus on the profiling and performance analysis of Llama3-8B during inference. We begin by introducing the architecture of the chosen model and describing the experimental setup used for evaluation. Then, we present a methodology for measuring compute and memory usage across different workloads and levels of granularity. This helps us to better understand the behavior of AI models and enables more accurate comparisons across hardware platforms. Finally, we discuss the performance results obtained and explain how these insights can be used to optimize AI model execution and improve system efficiency.

3.1 Objectives

The experiments aim to:

1. Determine the resource requirements of different model components, including stages and layers.

2. Identify which components (kernels, stages, or layers) are most suitable for offloading to increase hardware utilization.

3.2 Model selection

Based on the data presented in Section 2, we analyze the characteristics of transformer models and justify their selection for our study, with a specific focus on large language models (LLMs). The key reasons for choosing transformer models, particularly LLaMA 3, are outlined below:

1. **Structural Composition**: Transformer models share layers and usage among the other studied models. For example, they share embedding layers with the recommendation systems and feed-forward networks with deep neural networks (DNNs). In relation to recurrent neural networks (RNNs), this models have been largely replaced by transformers in language processing tasks such as text recognition, translation, and summarization. This makes transformer's results also representative for other models like DNN, DLRS, and RNNs.

2. **Widespread Industry Adoption**: Transformers are among the most widely used models in industry, with applications spanning natural language processing [63, 80, 82, 83, 84, 85, 86, 88, 89, 90, 91, 93, 94, 95], video analysis [80], audio processing [80], image generation [87], image analysis [86], and protein structure prediction [92].

3. **Representation of State-of-the-Art**: Transformers embody the latest advancements in AI model design, incorporating recent techniques. This makes them an ideal subject for studying current trends and developments in AI.

4. **Need of structured information**: Transformer models vary significantly in their use cases and parameter counts, making them highly adaptable to different tasks. However, this flexibility results in diverse hardware requirements influenced by model size, layer structure, and parameters such as input prompt, sequence length, and batch size. This high variability provides an opportunity to demonstrate how our methodology can help researchers gain insights, even across a wide range of scenarios.

Given these factors, our analysis focuses on transformer models, specifically LLMs. Among the available models, we select LLaMA 3 for our experiments due to the following reasons:

- **Regular Updates**: LLaMA 3 is frequently updated by Meta AI, ensuring it remains relevant and incorporates the latest advancements.
- **Wide Research Adoption**: It is one of the most utilized models in academic and industrial research, supporting a broad range of applications.
- **Open-Source Availability**: The model's code, architecture, and pre-trained weights are publicly accessible on GitHub [107], facilitating transparency and reproducibility.
- **Variety of Model Sizes**: LLaMA 3 offers versions ranging from 7B to 70B and 405B parameters, providing flexibility for different experimental scenarios.

For our experiments, we specifically choose LLaMA 3 8B with a batch size of 1 during inference. We focus on inference because it is not only relevant on its own, but also forms an integral part of the training process, as inference is performed repeatedly during training. This configuration serves as a baseline for our study, allowing us to compare results with future experiments that may include the training phase or vary other parameters, such as batch size or model size.

3.3 Experimental environment

This section details the hardware and software configurations used, ensuring a robust setup for accurate and reproducible experiments. The experiments are conducted on the MareNostrum 5 supercomputer at the Barcelona Supercomputing Center (BSC) [108].

3.3.1 Hardware environment

The experiments utilize accelerated compute nodes on MareNostrum 5, equipped with the following specifications:

- **Processor**: 2x Intel Xeon Platinum 8460Y+, each with:
  - 40 cores, 80 threads
  - Maximum clock rate: 3.4 GHz
  - Main memory: 512 GB DDR5 (16x 32 GB DIMMs, 4.8 GHz)
  - Cache:
    - L1d: 3.8 MiB
    - L1i: 2.5 MiB

    - L2: 160 MiB
    - L3: 210 MiB

- **GPU**: 4x NVIDIA H100, each with:
  - Maximum clock rate: 1.98 GHz
  - 132 Streaming Multiprocessors (SM), each containing:
    - 128 FP32 cores
    - 4 Tensor Cores
    - 1 Special Function Unit (SFU) for trigonometric functions
    - Maximum threads: 2048
    - L1 cache: 48 KB
  - Main memory: 64 GB HBM2, 1.59 GHz clock rate, 1024-byte bus width
  - L2 cache: 40 MB
  - CUDA capability: 9.0
- **Network**: 4x ConnectX-7 NDR200 InfiniBand, providing 800 Gb/s bandwidth per node
- **Storage**: 480 GB NVMe SSD for fast data access

3.3.2 Software environment

The software environment is tailored to ensure compatibility, flexibility, and precise profiling of the LLaMA 3 8B model. The key components include:

- **Model**: LLaMA 3 8B, sourced directly from the official Meta AI GitHub repository [107], enabling modifications and measurements of specific model components as needed. This model is based on the transformer architecture explained in Section 2.4.3.1. Figure 3.1 illustrates the Llama3 architecture: part (a) shows the full implementation, while part (b) details the implementation of the multi-query attention layer. While Llama3 retains the core structure of the transformer, it introduces some differences that affect the performance results if we compare it to other LLMs. This differences include:

  1. **Multi-Query Attention Mechanism:** Unlike standard multi-head attention, where each head has distinct query, key, and value projections, multi-query attention projects queries independently for each head while sharing the keys and values across all heads. This design reduces memory usage and computational overhead by decreasing the number of parameters and operations needed for keys and values, making it particularly efficient for large-scale models. [109]

     Formally, in multi-query attention:
     - For each head h, the query vector q_h is computed independently.
     - The key k and value v vectors are shared across all heads.

     The attention score for head h between a query at position m and a key at position n is calculated as:

     $$\text{score}_h(m, n) = (q_h^{(m)})^T k^{(n)}$$

     where $q_h^{(m)}$ is the query vector for head h at position m, and $k^{(n)}$ is the shared key vector at position n. The output is a weighted sum of the shared value vectors $v^{(n)}$, determined by these attention scores.

2. **Rotary Positional Embedding (RoPE)**: In transformer models, self-attention needs the positional embedding layer to encode token order. Some of this methods, like sinusoidal or learned embeddings, add positional information to token embeddings before the transformer layers (See Section 2.4.3.1). In contrast, RoPE embeds positional information directly into the self-attention process [110].

   RoPE applies a rotation to the query and key vectors based on their positions. For each position m, a rotation matrix R_m is defined and applied to the query vector q and key vector k. The attention score between a query at position m and a key at position n is computed as:

   $$(R_m q)^T (R_n k) = q^T R_m^T R_n k$$

   The rotation matrices are constructed such that $R_m^T R_n$ depends on the relative position $m - n$, effectively encoding relative positional information into the attention scores.

   For a d-dimensional embedding, RoPE splits the dimensions into $d/2$ pairs. For each pair $j = 1, 2, \ldots, d/2$, a 2D rotation matrix is applied with an angle proportional to the position m and a frequency θ_j. The rotation for pair j at position m is:

   $$R(m, j) = \begin{bmatrix} \cos(m\theta_j) & -\sin(m\theta_j) \\ \sin(m\theta_j) & \cos(m\theta_j) \end{bmatrix}$$

   The full rotation matrix R_m is a block-diagonal matrix composed of these 2D rotations. This transformation ensures that the dot product $(R_m q)^T (R_n k)$ captures both content and relative positional information.

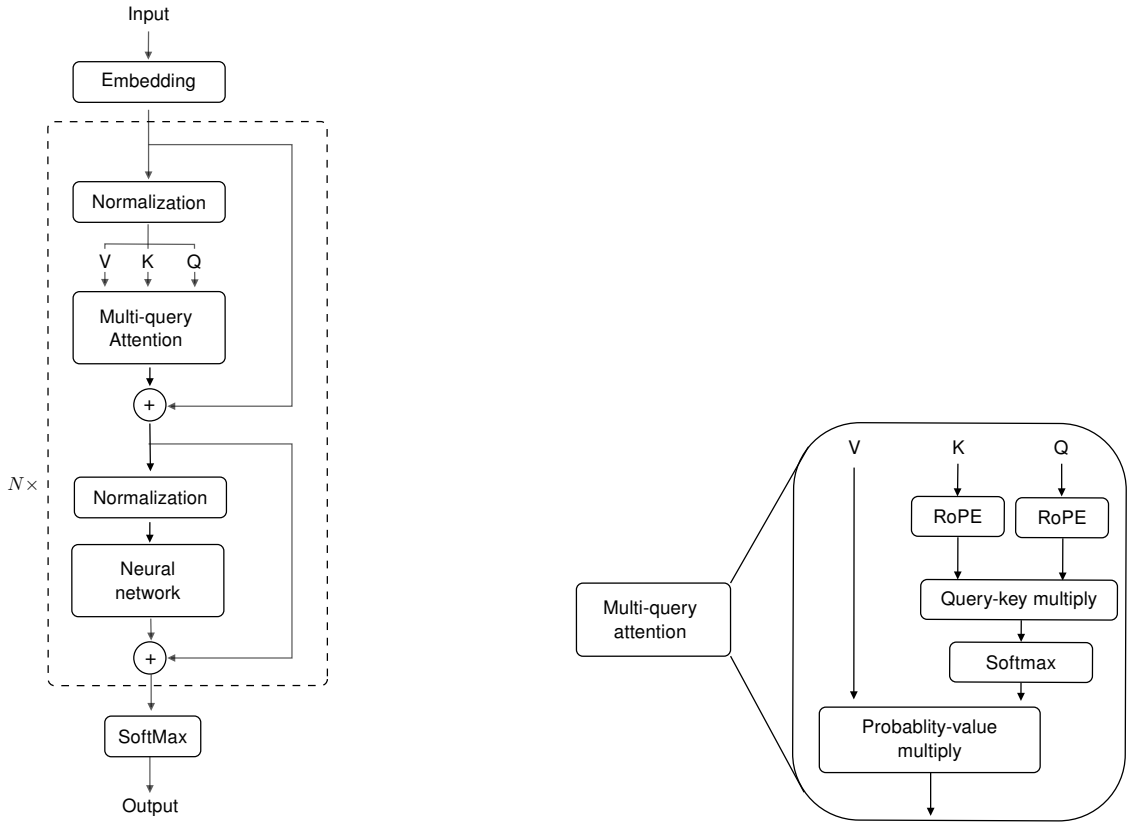
   RoPE's integration within the attention layer modifies how queries and keys interact during attention computation. Rather than adding positional data to input embeddings, RoPE applies position-dependent rotations to queries and keys, directly influencing attention scores. This method improves the model's ability to capture relative positional dependencies and enhances generalization, particularly for longer sequences.

- **Framework**: PyTorch with CUDA, used for model inference and GPU acceleration.
- **Dependencies**: Python 3.10, with standard machine learning libraries included by default in the LLaMA 3 repository.
- **Profiling Tools**:
  - NVIDIA Nsight Systems: For system-level profiling, capturing detailed performance metrics such as kernel execution times and memory usage.
  - NVIDIA Nsight Compute: For fine-grained analysis of GPU hardware counters, minimizing software-induced overhead and noise.

The experiments are executed on dedicated nodes to ensure isolation and reproducibility. In all cases, the model is the only process running during execution.

3.3.3 Methodology

To ensure reproducible and extensible results, the experiments are organized into three groups based on the granularity of measurements: full model, stages, and layers. This structure allows for detailed analysis of the LLaMA 3 8B model's performance and hardware characteristics.



(a) Full Llama3 architecture (b) Multi-query attention layer

Figure 3.1: (a) Complete Llama3 transformer architecture. (b) Detailed view of the multi-query attention mechanism used in Llama3.

3.3.3.1 Measurement Granularity

The experiments are divided into three levels of granularity:

- **Full Model**: Measurements capture the overall execution of the model without distinguishing between stages or layers.
- **Stages**: Measurements focus on the two primary stages of model execution:
  - *Prefill Stage*: Processing the input prompt provided by the user.
  - *Decode Stage*: Generating the output sequence token by token.
- **Layers**: Measurements target individual layers, including Embedding, Attention, Feed Forward Network, and Softmax.

This structure supports increased granularity if needed. For example:

- **Stages**: Measure the cost of loading model weights or analyze data movement between CPU (host) and GPU (device) memory.
- **Layers**: Subdivide layers, such as the Attention layer, into linear projections, Q-K multiplication, P-V multiplication, and Softmax sublayers, as described in Section 2.4.3.1.

3.3.3.2 Experiment Groups

For each granularity level (full model, stages, and layers), three types of experiments are conducted:

- **Time Measurements**: Measure the execution time of each granularity level.
- **Memory Measurements**: Analyze four key GPU Memory Operations:
  - *Memcpy DtoD*: Device-to-device memory copy within GPU memory.
  - *Memcpy DtoH*: Device-to-host memory transfer (GPU to CPU).
  - *Memcpy HtoD*: Host-to-device memory transfer (CPU to GPU).
  - *Memset*: Setting GPU memory to a specific value.

  These operations are evaluated in three ways:

  1. Amount of memory (MB) copied or set.
  2. Percentage of total memory operation time attributed to each operation.
  3. Percentage of total execution time (including kernel execution) attributed to each operation.
- **Kernel Measurements**: measure the performance of individual kernels within the model. This includes:
  - *Kernel Selection*: Identify the most significant kernels based on their execution time, focusing on those with the longest runtime.
    - *Execution Time*: Kernels with the longest runtime.
  - *Kernel Analysis*: For selected kernels:
    - Plot on bandwidth-latency curves of the target server.
    - Plot on roofline models of the target server.

3.3.3.3 Deterministic parameters and scenarios

Three parameters are controlled in all experiments:

- **Input Prompt**: Number of tokens fed to the model.
- **Output Sequence**: Number of tokens generated by the model.
- **Maximum Sequence Length**: Total tokens processed, including input prompt and output sequence.

Three experimental scenarios are defined:

- Fix the input prompt size and vary the number of output tokens.
- Fix the output sequence size and vary the input prompt size.
- Fix the maximum sequence length and vary the proportion of input prompt and output tokens.

Since the output of the model is non-deterministic by default, it is necessary to control the number of tokens processed and generated to ensure consistent measurements. To achieve this, we made modifications to the `generation.py` file:

```
1 if all(eos_reached):
2     None
```

Code 3.1: Modification to `generation.py` in the LLaMA 3 model to guarantee a deterministic number of generated tokens

With this change, the model continues generating tokens until it reaches the maximum allowed number, regardless of early stop conditions. This guarantees that the number of generated tokens remains fixed across different executions.

3.4 Results

In this section, we presents the results of the experiments conducted following the methodology explained in Section 3.3.3. As explained, in all cases we evaluate the performance Llama3-8B during inference without using batching. We always process one input sequence at a time.

3.4.1 Full model

In this group of tests we make the measurements over the model without making any distinction of stages and layers. This way we get a general view of how the model behaves as a whole under different conditions. In specific, we wrap for profiling all the `chat_competition` function of the Llama3 model as shown in Code 3.2. This `chat_competition` function contains all the steps needed to generate an output sequence for a giving input prompt.

```
torch.cuda.nvtx.range_push("inference")
results = generator.chat_completion(
        dialogs,
        max_gen_len=max_gen_len,
        temperature=temperature,
        top_p=top_p
    )
torch.cuda.nvtx.range_pop()
```

Code 3.2: `chat_completition` funtion wrapped with nvtx tags for profiling all stages and layers during inference.

3.4.1.1 Time measurements

In this group of experiments we measure how the execution time gets affected while varying the input prompt, output sequence length and maximum sequence length as explained in Section 3.3.3. The profiler characteristics are:

- **Profiler used:** Nvidia Nsight Systems.
- **Traces measured:** nvtx.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

When we increase the number of output tokens generated, the execution time increases linearly as shown in Table 3.1 and Figure 3.2.

Output Tokens	Inference Time (s)
504	7.9
1016	17.1
2040	33.6
4088	71.9
8184	161.6

Table 3.1: Full model inference time for different numbers of output tokens generated.

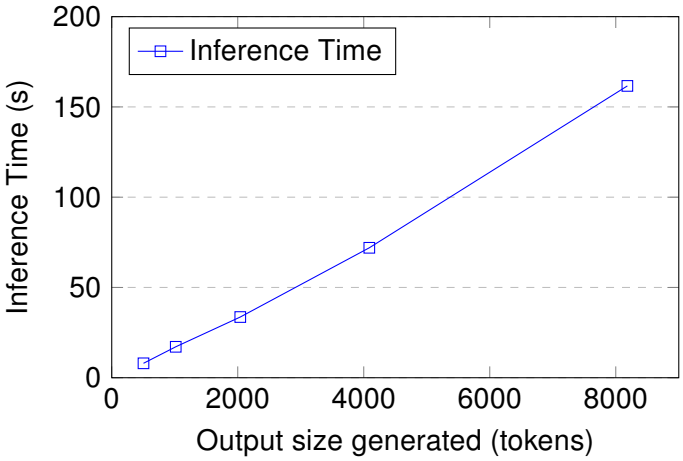


Figure 3.2: Full model inference time for different numbers of output tokens generated

According to the literature, the compute requirements and inference time increases with the square of the sequence length [89, 104, 111, 112], but on the results we observe a linear increment. This linear behavior is related to the use of a KV cache, which reduces the amount of re-computations that the model has to perform [104]. This can be demonstrated empirically byAccording to the literature, the compute requirements and inference time increases with the square of the sequence length deactivating the KV cache and observing the resulting execution times. For a execution time T proportional to the square of the sequence length L we have:

$$T(L) = k \times L^2, \tag{3.1}$$

where k is a constant.

For a base execution time of 12.7 s when generating 504 tokens, in the ideal case without a constant k, the execution times would be:

- For sequence length $2L$:

$$T(2L) = k \times (2L)^2 = 4kL^2 = 4T(L) \approx 4 \times 12.7 \text{ s} \approx 50.8 \text{ s}.$$

- For sequence length $4L$:

$$T(4L) = k \times (4L)^2 = 16kL^2 = 16T(L) \approx 16 \times 12.7 \text{ s} \approx 203.5 \text{ s}.$$

- For sequence length $8L$:

$$T(8L) = k \times (8L)^2 = 64kL^2 = 64T(L) \approx 64 \times 12.7 \text{ s} \approx 814.0 \text{ s}.$$

- For sequence length $16L$:

$$T(16L) = k \times (16L)^2 = 256kL^2 = 256T(L) \approx 256 \times 12.7 \text{ s} \approx 3256.0 \text{ s}.$$

In Table 3.2 we can see the comparison of the ideal square increment and the measured results without KV cache. We can see how the execution increases around this ideal quadratic function, being even bigger when we generate 8184 tokens.

Output Tokens	Measured Inference Time (s)	Ideal quadratic Inference Time (s)
504	12.7	—
1016	41.5	50.8
2040	175.2	203.5
4088	981.2	814.0
8184	6636.2	3256.0

Table 3.2: Full model inference time for different numbers of output tokens generated without KV cache.

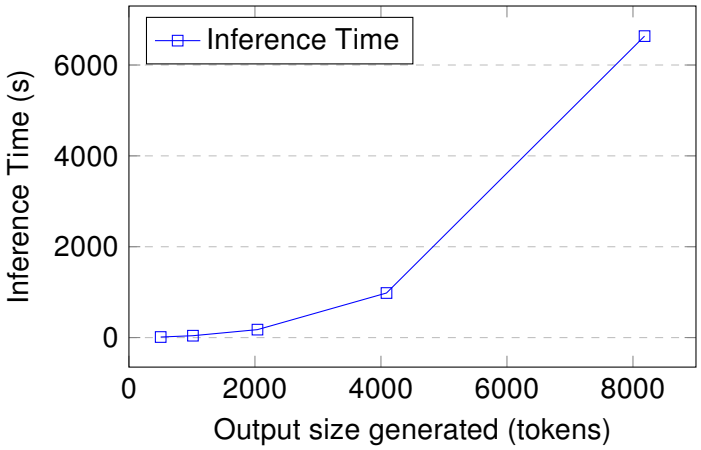


Figure 3.3: Full model inference time for different numbers of output tokens generated without KV cache

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

When we increase the number of input tokens we can see how the execution time increases, as shown in Table 3.3 and Figure 3.4. We also can see how, the effect that the input prompt has over the execution time is lower than the one seen while increasing the number of output tokens. In specific, generating 4088 output tokens takes 72 s while processing 4484 input tokens takes 1.03 s. This behavior was expected since the input prompt is processed only once during execution, while the output is generated iteratively token by token as explained in Section 2.4.3.

Input Tokens	Inference Time (s)
16	0.44
25	0.43
55	0.43
109	0.45
224	0.43
524	0.49
991	0.49
2205	0.57
3769	0.88
4484	1.03
6319	1.85

Table 3.3: Full model inference time while increasing the size of the input prompt.

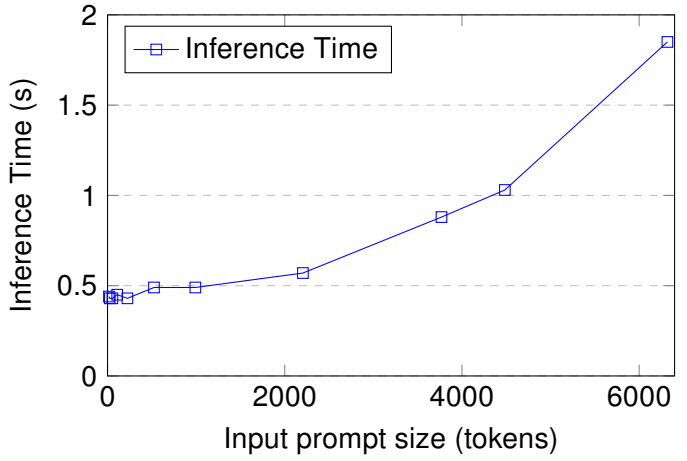


Figure 3.4: Full model inference time evolucion while increasing the size of the input prompt.

Varying the relation of total input and output tokens

- Varying the input prompt size: [16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319] tokens.
- Varying the output prompt size: [8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873] tokens.
- Fixed sequence length: [8192] tokens.

In Table 3.3, we observe that execution time decreases as the proportion of input prompt tokens increases in the total sequence length. This aligns with previous results, reinforcing the notion that the impact of increasing the number of generated output tokens is bigger than the one of increasing the number of processed input tokens.

Relation in/out (%)	Inference Time (s)
0.2 : 99.8	161.5
0.3 : 99.7	162.9
0.7 : 99.3	160.4
1.3 : 98.7	159.6
2.7 : 97.3	158.9
6.4 : 93.6	153.8
12.1 : 87.9	146.1
26.9 : 73.1	123.7
46.0 : 54.0	95.0
54.7 : 45.3	80.7
77.1 : 22.9	43.5

Table 3.4: Full model inference time while varying the ratio of input prompt and output tokens.

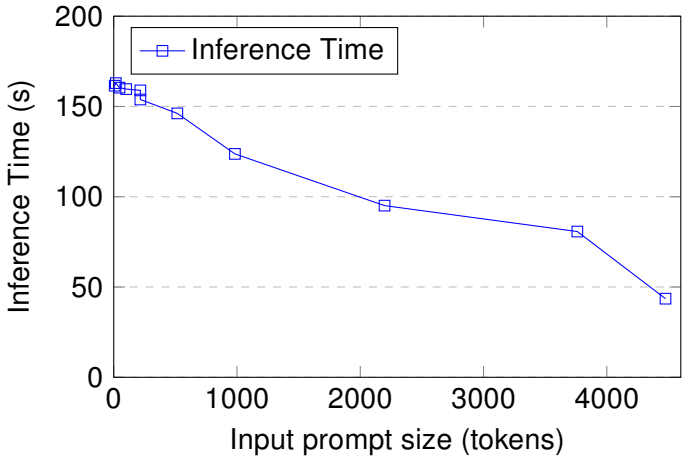


Figure 3.5: Full model inference time while varying the ratio of input prompt and output tokens.

Takeouts:

1. The models execution time increases linearly while increasing the output prompt due to the KV cache.
2. Increasing the output prompt size has a bigger impact on the execution time than increasing the input prompt size.

3.4.1.2 Memory operations

In this group of expermients we measure the effect that varying the model parameters have over the memory operations. In specific we will distinguish between four operations: Memcpy DtoD, Memcpy DtoH, Memcpy HtoD and Memset as described in Section 3.3.3. The profiler characteristics are:

- Profiler: Nvidia Nsight Systems.
- Traces measured: cuda,nvtx,osrt,cudnn,cublas.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

In Figure 3.6 we can see how the amount of memory copied and set increments as we increase the number of output tokens generated. This behavior can be seen clearer in Table 3.5 in specific we can see how:

- The amount of memory copied form host to device is constant for all executions, around 16 GB. This amount of memory matches the size of the weights of the model.
- The amount of memory copied and set inside the GPU increases linearly while increasing the output tokens generated. This behavior, can be explained again by the use of the KV cache. The KV cache reduces the amount of re-computations that the model has to perform by storing the key and value tensors in the GPU memory. This way, the amount of memory that this KV cache occupies increases linearly with the size of the sequence length [104].
- The amount of memory copied from device to host increases linearly while increasing the output tokens generated. This matches the amount of tokens that are generated as an output, given that this tokes are moved from the GPU memory to the host memory once inference is done.

In Figure 3.7 we can see how the execution time increases linearly with the number of output tokens generated. This behavior has the same explanation as the one seen in the memory operations size. Finally in Figure 3.8 we study how this memory operations affect in the overall inference time. We see that copying weights from host to device takes the most time. But as we generate more output tokens, memory operations take up a smaller share of the total inference time, reaching a minimum of 2% when generating 8184 tokens.

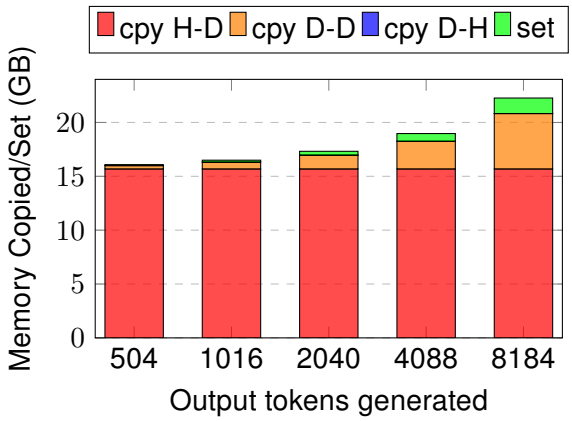


Figure 3.6: Full model memory operations size while increasing the number of the output tokens.

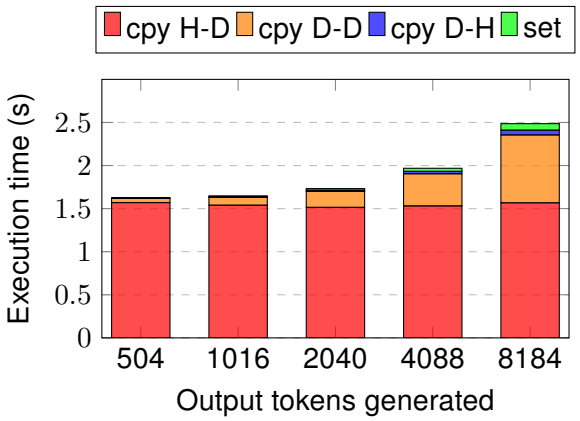


Figure 3.7: Full model memory operations execution time while increasing the number of the output tokens.

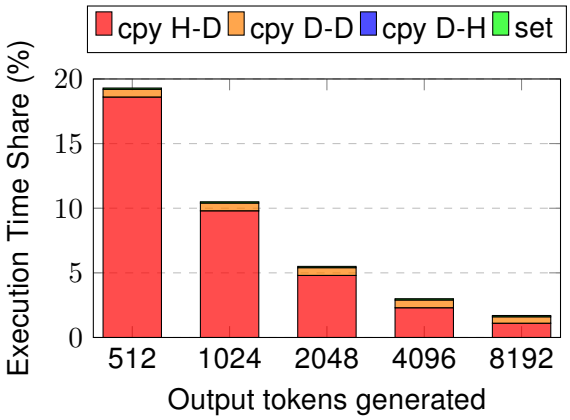


Figure 3.8: Full model percentage of execution time spent on memory operations while increasing the number of the output tokens.

Output Tokens	Memcpy H-D (MB)	Memcpy D-D (MB)	Memset (MB)	Memcpy D-H (MB)
504	16060.5	321.4	89.4	0.009
1016	16060.5	651.2	181.6	0.018
2040	16060.5	1310.8	366.2	0.037
4088	16060.5	2629.9	735.3	0.074
8184	16060.5	5268.2	1473.6	0.147

Table 3.5: Memory operations size in MegaBytes while varying the number of output tokens generated.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

As before, for the memory copy from host to device (Memcpy H-D), remains constant for all executions, around 16 GB, adding some extra memory related to the size of the input prompt that the model processes. For the rest of operations we can see how:

- The amount of memory copied inside the GPU again increases linearly while increasing the input prompt. This is because each new token adds a fixed amount of information, so the memory use grows linearly.
- The memset operation shows a near-constant value, likely being used to initialize a fixed-size portion of memory or a buffer whose size doesn't depend significantly on the input prompt.
- The amount of memory moved from device to host again increases linearly with the input prompt. In this case this increment is lower than the one seen while increasing the number of output tokens generated. This is because now the result of the inference query is fixed to 8 tokens, so the amount of data moved with the result is fixed.

This behavior can be seen in Figure 3.9 and Table 3.6. We also see how, the operations with the biggest memory footprint are the Memcpy D-D, reaching up to 800 MB with the biggest input prompt. On the other hand, we also see how this memory operations account for a small percentage of the total execution time, taking in most of the cases less than 2 ms. Representing less than 0.1 % of the total execution time.

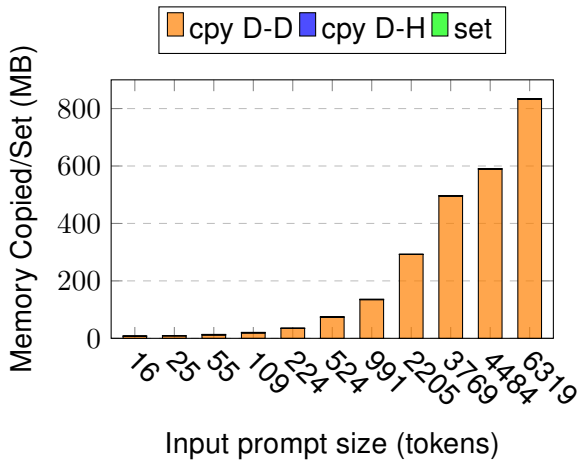


Figure 3.9: Full model memory operations size while increasing the number of the input tokens.

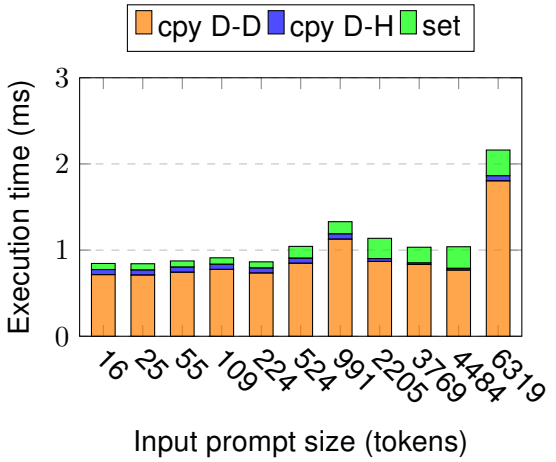


Figure 3.10: Full model memory operations execution time while increasing the number of the input tokens.

Output Tokens	Memcpy D-D (MB)	Memset (MB)	Memcpy D-H (MB)
16	7.1	1.4	0.000
25	8.2	1.4	0.000
55	12.2	1.4	0.001
109	19.3	1.4	0.001
224	34.3	1.4	0.002
524	73.7	1.4	0.004
991	134.9	1.4	0.008
2205	292.0	0.7	0.018
3769	496.0	0.3	0.030
4484	589.7	0.3	0.036
6319	833.3	1.4	0.051

Table 3.6: Full model memory operations size while varying the number of input tokens generated.

Varying the relation of total input and output tokens

- Varying the input prompt size: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Varying the output prompt size: 8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873 tokens.
- Fixed sequence length: 8192 tokens.

As before, the amount of memory copied from host to device remains constant for all executions, around 16 GB. In the previous results, increasing either the input prompt or the output tokens increases the amount of memory copied within the device. However, the impact of increasing output tokens is significantly greater, using around 4 to 5 times more memory for the same number of tokens processed. For example, processing 500 tokens as input requires 71 MB, while processing 500 tokens as output takes 300 MB. As a result, when we increase the proportion of input tokens (while reducing the number of output tokens), the amount of memory copied within the device decreases. This happens because the reduction in memory from fewer output tokens outweighs the increase from more input tokens. A similar pattern occurs with Memset operations inside the device. It remains constant when varying the input prompt but grows as output tokens increase. Thus, increasing the share of output tokens increases the memory allocated. For memory transferred from device to host, increasing output tokens increases the data returned to the host after inference. So, when the percentage of output tokens decreases, the amount of memory transferred from device to host also drops, since the effect of increasing the input prompt is much smaller. In Figures 3.12 and 3.11, and Table 3.7 we can see this results, with a general reduction of the memory copied and set while increasing the share of tokens processed in the input.

Also matching previous results, this memory operations represent a small amount of the execution time taking together less than 1 s. Representing less than 0.7 % of the total execution time. If we include the memory operations related to copying the weights, the total amount of time that the memory operations represent from the total execution time is still low, less than 3.5 %.

Relation in/out (%)	Memcpy D-D (MB)	Memset (MB)	Memcpy D-H (MB)
0.2 : 99.8	5268.2	1473.6	0.14
0.3 : 99.7	5263.6	1472.0	0.14
0.7 : 99.3	5248.2	1466.6	0.14
1.3 : 98.7	5220.5	1456.9	0.14
2.7 : 97.3	5161.5	1436.1	0.14
6.4 : 93.6	5007.6	1382.1	0.14
12.1 : 87.9	4768.0	1297.9	0.13
26.9 : 73.1	4143.1	1078.4	0.12
46.0 : 54.0	3339.7	796.1	0.11
54.7 : 45.3	2972.9	667.2	0.10
77.1 : 22.9	2034.6	337.5	0.08

Table 3.7: Full model memory operations size while varying the ratio of input prompt and output tokens.

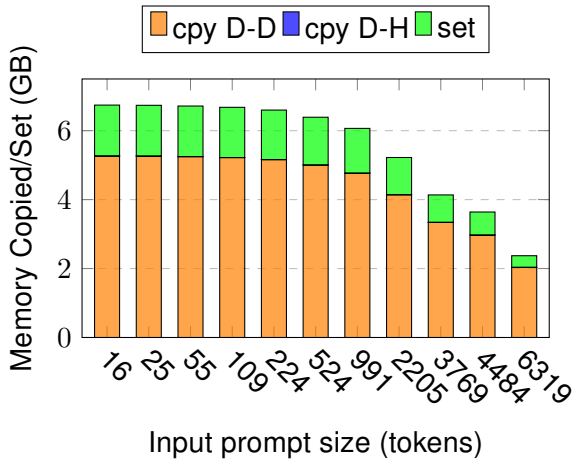


Figure 3.11: Full model memory operations size while varying the ratio of input prompt and output tokens.

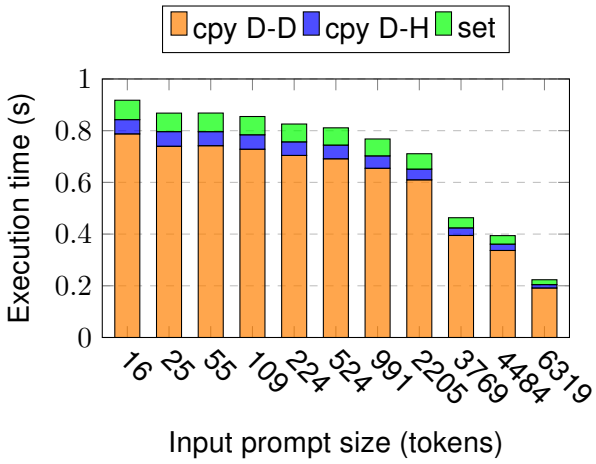


Figure 3.12: Full model memory operations execution time while varying the ratio of input prompt and output tokens.

Takeouts:

1. The amount of memory moved from host to device (Memcpy H-D) is constant. Around 16GB, matching the size of the weights of the model.
2. While increasing only the number of output tokens:
   - 2.1 The amount of memory copied and set inside the GPU increases linearly due to the KV cache.
   - 2.2 The execution time of the memory operations decreases.
3. While increasing only the number of input tokens:
   - 3.1 The amount of memory copied inside the GPU increases linearly because each new token adds a fixed amount of information.
   - 3.2 The amount of memory set inside the GPU is low (less than 2 MB) and remains constant.
   - 3.3 The amount of memory moved from device to host is low (less than 0.06 MB) and increases linearly.
4. While varying the ratio of input and output tokens:
   - 4.1 With a minimum input prompt , the cost of the memory operations not related to copying the weights is very low less than 0.1 %.
   - 4.2 Increasing the number of output tokens has a larger effect on memory operations size and execution time than increasing the number of input tokens.
   - 4.3 When the sequence length is large (whether it consists mostly of input tokens or output tokens), the time spent on memory operations is low (less than 3.5 %), with most of the time dedicated to kernels execution.

3.4.1.3 Kernel executions

In this group of tests we study the characteristics of the four most relevant kernels during the execution of the model. This kernels are selected in relation to their execution time as explained 3.3.3. The profilers characteristics are:

- Profiler: Nvidia Nsight Systems.
  - Traces measured: cuda,nvtx,osrt,cudnn,cublas
- Profiler: Nvidia Nsight Compute.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

The four most time-consuming kernels are matrix-matrix and matrix-vector multiplications. As shown in Figure 3.13, these kernels consistently make up to 60–80% of the total runtime across all executions.

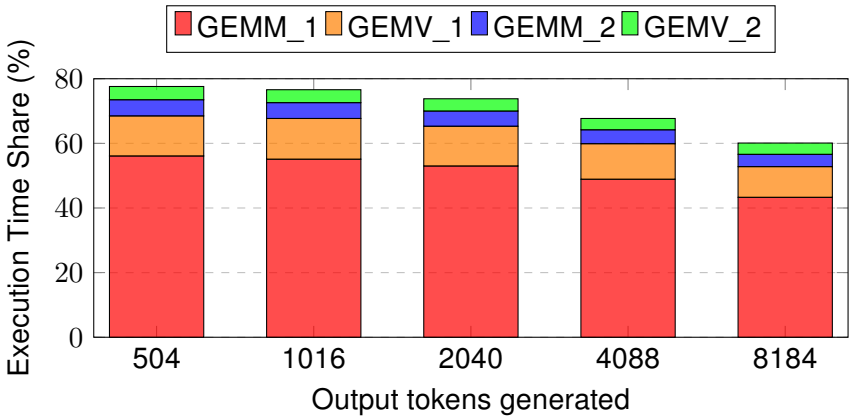


Figure 3.13: Full model percentage of total execution time for the four most time-consuming kernels across different output sequence lengths.

The full characteristics of this kernels are explained in Chapter 7.2. In specific, GEMM_1, GEMM_2, GEMV_1 and GEMV_2 are explained in Tables 7.1 and 7.2. We can see how these kernels are mainly executed in tensor cores, perform operations in BF16 and store the results in FP32. This is a common practice, given that the models weights are stored in BF16. Across all executions the bandwidth requirements of this kernels keep constant, around 1300 GB/s for GEMM_1, 1030 GB/s for GEMV_1, 1400 GB/s for GEMM_2 and 700 GB/s for GEMV_2.

We study this behaviour over the Nvidia H100 GPU's bandwidth-latency curves. The bandwidth-latency curves are a way to describe the memory system performance. They illustrate the inherent dependency between the used memory bandwidth and the access latency. Each curve in the family corresponds to a specific ratio between the read and write memory traffic, plotted with different shades of blue. On these curves, the x-axis corresponds to memory bandwidth, while the y-axis represents memory access latency. This way we can see how the memory access latency is affected by the memory bandwidth. Generally, memory access latency is initially roughly constant and then increases with higher memory pressure (bandwidth) due to resource contention [113].

In in Figure 3.14 we can see the bandwidth-latency curves, and position of the four most time-consuming kernels while increasing the number of output tokens generated. This makes GEMM_2 the kernel with the biggest bandwidth requirements and the third most time in the execution of the model.

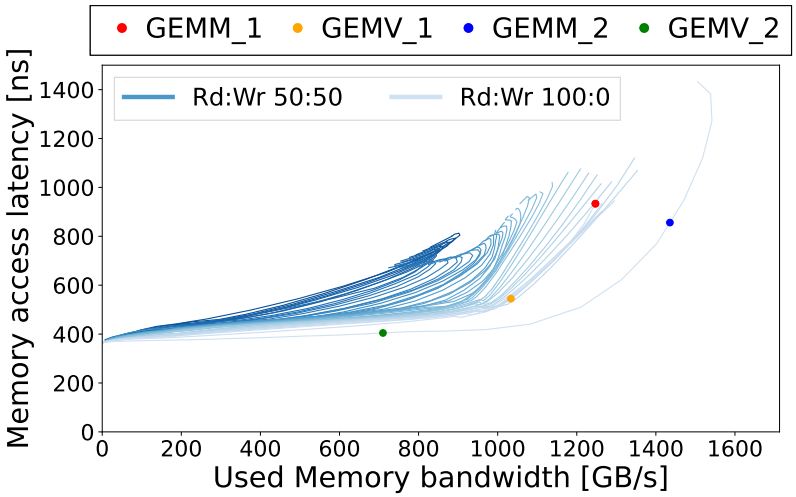


Figure 3.14: Bandwidth latency curves for the most time consuming kernels while increasing the number of output tokens in the full model.

We also study the arithmetic intensity of the kernels, which is defined as the number of floating-point operations performed per byte of memory transferred. Additionally, we measure the performance of each kernel in terms of floating-point operations per second (FLOP/s). By combining these two metrics, we build a roofline model that helps us understand the balance between memory usage and compute requirements. This provides a clearer view of the performance characteristics and limitations of the kernels. To measure and plot these values, we use specific counters from the Nvidia Nsight Compute documentation [114]. Since each kernel can use different compute units, their performance varies depending on which unit they use. The roofs (maximum performance limits) are defined as follows:

For Tensor Cores:

$$Roof_{TensorCore} = MaxTensorOpPerCycle \cdot 512\,(Flops/TensorOp) \cdot AvgCyclePerSec \tag{3.2}$$

Where:

- $MaxTensorOpPerCycle$: the maximum number of tensor operations the SM can sustain per cycle.
- $AvgCyclePerSec$: the actual GPU frequency during execution.

We multiply the maximum tensor operations per cycle by 512 (the floating-point operations per tensor operation) and then by the GPU's actual frequency during the kernels exeution. This gives us the theoretical peak performance of the Tensor Cores.

For FP32 cores:

$$Roof_{FP32} = MaxFP32OpPerCycle \cdot AvgCyclePerSec \tag{3.3}$$

Where:

- $MaxFP32OpPerCycle$: the maximum number of FP32 operations the SM can sustain per cycle.

We use the same approach for other data types like FP64 and FP16.

Finally we compute the roof for DRAM as:

$$Roof_{DRAM} = MaxDRAMThroughputPerCycle \cdot AvgDRAMCyclePerSec \tag{3.4}$$

Where:

- $MaxDRAMThroughputPerCycle$: the maximum bytes per cycle that can be read/written from DRAM.
- $AvgDRAMCyclePerSec$: the actual DRAM frequency during execution.

Once we have the DRAM and Cores roofs, we can plot each kernels' performance.

For Tensor Cores:

$$Performance_{Kernel}(GFlops/s) = TensorOP \cdot 512 \tag{3.5}$$

Where:

- $TensorOP$: the number of tensor operations performed by the kernel.

For FP32 cores, we calculate performance as:

$$Performance_{Kernel}(GFlops/s) = FP32AddOpsPerSec + FP32MultOpsPerSec + FP32FmaOpsPerSec \cdot 2 \tag{3.6}$$

Where:

- $FP32AddOpsPerSec$: FP32 addition operations per second.
- $FP32MultOpsPerSec$: FP32 multiplication operations per second.
- $FP32FmaOpsPerSec$: FP32 multiply-add operations per second (counted as two operations).

To find the arithmetic intensity, we divide the kernel's performance by the DRAM throughput:

$$AI_{Kernel} = \frac{Performance_{Kernel}}{DRAMThroughput} \tag{3.7}$$

Where:

- $DRAMThroughput$: the amount of DRAM memory transferred per second.

All the counters used to compute the roofs and kernels data are listed in Tables 3.9 and 3.8.

Metric	Counter
$TensorOP$	`smsp__inst_executed_pipe_tensor.sum.per_second` (Tensor operations performed)
$FP16AddOpsPerSec$	`smsp__sass_thread_inst_executed_op_hadd_pred_on.sum.per_second` (FP16 addition operations per second)
$FP16MultOpsPerSec$	`smsp__sass_thread_inst_executed_op_hmul_pred_on.sum.per_second` (FP16 multiplication operations per second)
$FP16FmaOpsPerSec$	`smsp__sass_thread_inst_executed_op_hfma_pred_on.sum.per_second` (FP16 multiply-add operations per second)
$FP32AddOpsPerSec$	`smsp__sass_thread_inst_executed_op_fadd_pred_on.sum.per_second` (FP32 addition operations per second)
$FP32MultOpsPerSec$	`smsp__sass_thread_inst_executed_op_fmul_pred_on.sum.per_second` (FP32 multiplication operations per second)
$FP32FmaOpsPerSec$	`smsp__sass_thread_inst_executed_op_ffma_pred_on.sum.per_second` (FP32 multiply-add operations per second)
$FP64AddOpsPerSec$	`smsp__sass_thread_inst_executed_op_dadd_pred_on.sum.per_second` (FP64 addition operations per second)
$FP64MultOpsPerSec$	`smsp__sass_thread_inst_executed_op_dmul_pred_on.sum.per_second` (FP64 multiplication operations per second)
$FP64FmaOpsPerSec$	`smsp__sass_thread_inst_executed_op_dfma_pred_on.sum.per_second` (FP64 multiply-add operations per second)

Table 3.8: Metrics and counters of variable values used for kernels performance analysis in roofline model.

Metric	Counter
$MaxTensorOpPerCycle$	`sm__inst_executed_pipe_tensor.sum.peak_sustained` (Maximum tensor operations per cycle)
$AvgCyclePerSec$	`sm__cycles_elapsed.avg.per_second` (Actual GPU frequency)
$MaxFP16OpPerCycle$	`sm__sass_thread_inst_executed_op_dfma_pred_on.sum.per_second` (Actual GPU frequency)
$MaxFP32OpPerCycle$	`sm__sass_thread_inst_executed_op_ffma_pred_on.sum.per_second` (Actual GPU frequency)
$MaxFP64OpPerCycle$	`sm__sass_thread_inst_executed_op_hfma_pred_on.sum.peak_sustained` (Maximum FP32 operations per cycle)

Table 3.9: Metrics and counters of constant values used for kernels performance analysis in roofline model.

In Figure 3.15 we can see the roofline model for the four most time-consuming kernels during the execution of the full model. For each kernel, we represent up to three different points, one for each type of operation the kernel performs. A single kernel may execute Tensor Core, FP32, and FP16 operations during its execution. Each type of operation is shown with a different marker in the plot: circles for Tensor Core operations, triangles for FP32 operations, and crosses for FP16 operations. This is illustrated in Figure 3.15, where we can observe that, in all cases, the kernels significantly underutilize the compute capabilities of the GPU. This underutilization is consistent across all operation types and data precisions.

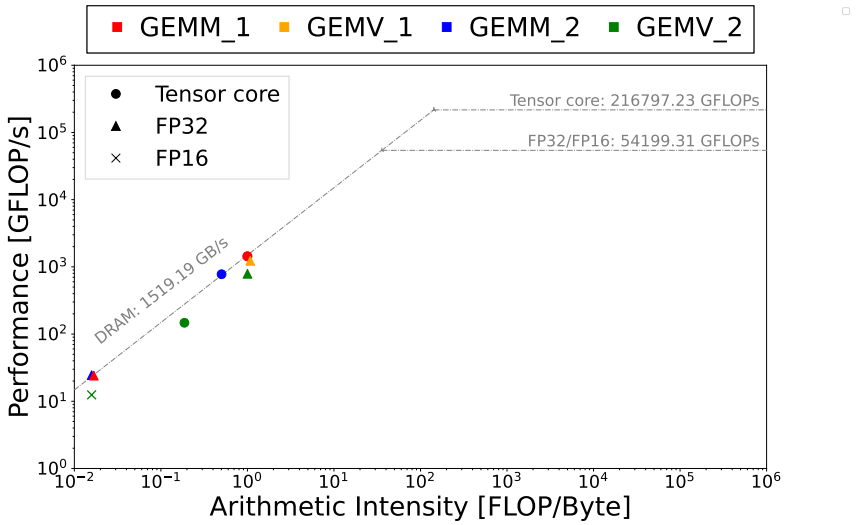


Figure 3.15: Performance and arithmetic intensity of main kernels during full model execution while varying the number of output tokens.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

When the input prompt is “small” (less than 1000 tokens), we can see how again GEMM_1, GEMV_1, GEMM_2 and GEMV_2 explained before, are the most time-consuming kernels. Figure 3.16 shows that these four kernels account for 40–80% of the total execution time.

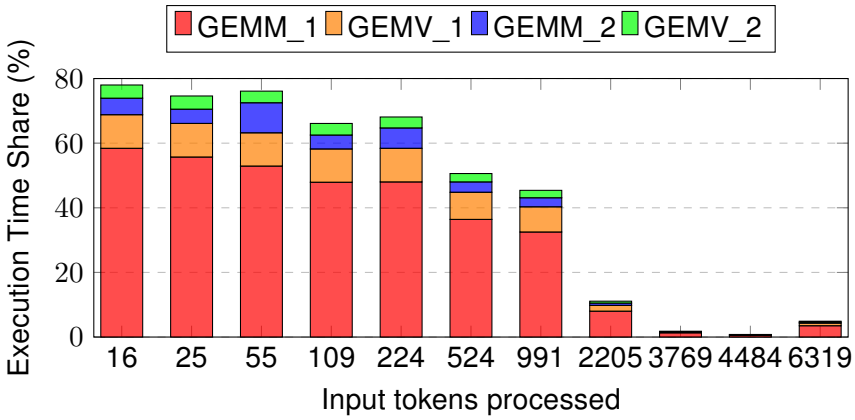


Figure 3.16: Full model percentage of total execution time for the four most time-consuming kernels across different input prompt lengths.

When the input prompt is “big” (more than 1000 tokens), we can see how other kernels take over the execution time. These are:

- **SoftMax:** A kernel that performs the SoftMax operation explained in Section 2.4.3.1.
- **Elementwise_1, Elementwise_2 and Elementwise_3:** standard cuda kernels that perform element-wise operations.

This way we can see how operations like SoftMax and non tensor operations gain importance in the execution of the model when the input prompt increases. Figure 3.17 shows that this kernels account for 40–50% of the total execution time, when we increase the input prompt over 1000 tokens.

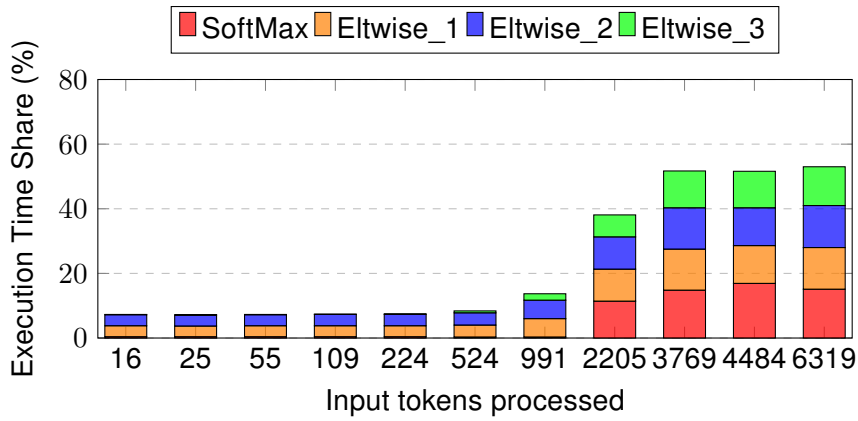


Figure 3.17: Full model percentage of total execution time for the four most time-consuming kernels across different input prompt lengths.

In terms of bandwidth and arithmetic intensity, kernels GEMM_1, GEMV_1, GEMM_2, and GEMV_2 show consistent behavior, as previously described when varying the output. However, the SoftMax and elementwise kernels exhibit different behavior. These kernels show greater variability because they are executed multiple times with different grid and block configurations. As a result, the same kernel can appear at different points on the latency-bandwidth curve as shown in Figure 3.18.

In our tests, the highest bandwidth usage occurs when the kernel is launched with a very large grid size but this grid configurations represents a small percentage of the kernels launch. For example, 96% of the SoftMax kernel calls use a grid shape of (32, 1, 1), while only 0.2% of the calls use a grid shape of (202208, 1, 1), which significantly increases the bandwidth requirement, reaching up to a terabyte per second. The same behavior is observed in the three elementwise kernels, especially in Elementwise 1 and Elementwise 2. These kernels perform a `direct_copy_kernel_cuda` operation which copies data between tensors on a CUDA device. This explains the high bandwidth usage. Since changing the grid shape directly affects the amount of data being copied, increasing the grid size also increments the amount of memory copied. This results in a wide range of behaviors across the bandwidth-latency curves.

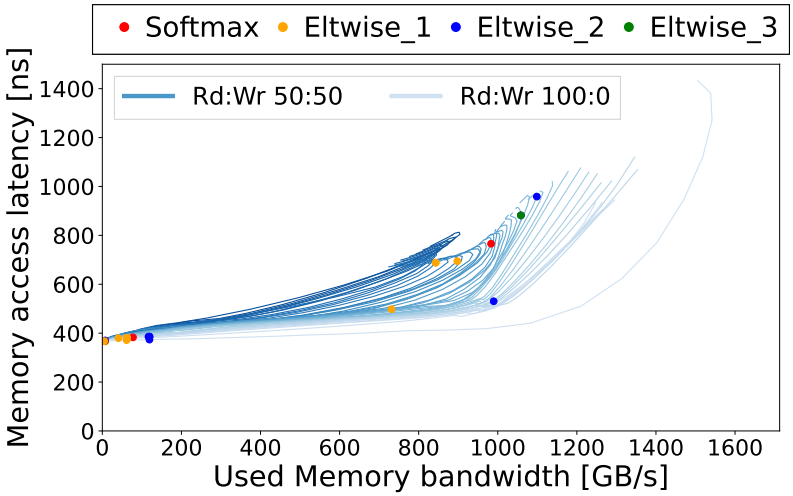


Figure 3.18: Full model bandwidth-latency curves for the most time consuming kernels when the number of input tokens is over 1000.

In Figure 3.19, we present the roofline model for the SoftMax and elementwise kernels. We observe that Elementwise 1 and Elementwise 2 do not appear in the plot. This is because these kernels are used to copy tensors and do not perform any arithmetic operations over the data.

For the SoftMax kernel, we again see multiple points corresponding to the different grid and block configurations used during its execution. The points that reach the roof—indicating saturation of bandwidth—are those with the largest grid sizes. Despite the variation in bandwidth, all these executions show a similar arithmetic intensity, appearing aligned along the horizontal axis when executed on the same type of core. Elementwise 3 consistently saturates the available bandwidth. Once again, none of the kernels come close to saturating the compute capabilities of the GPU.

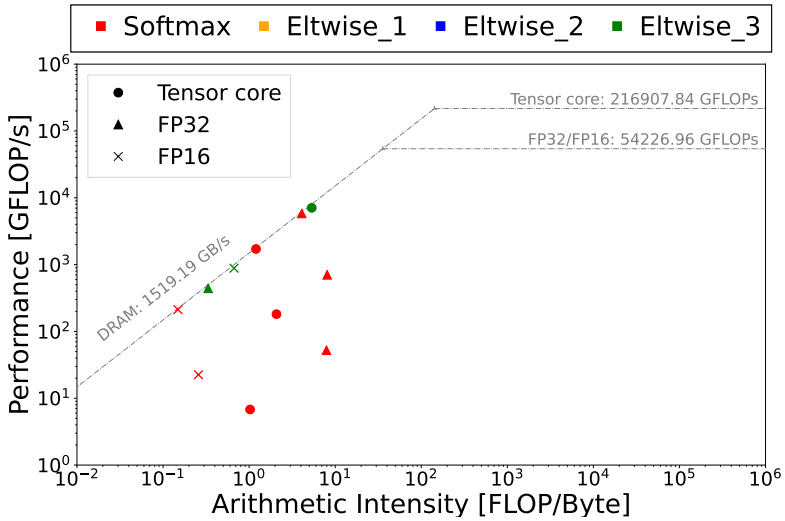


Figure 3.19: Full model Performance and arithmetic intensity of main kernels when the number of input tokens is over 1000.

Varying the relation of total input and output tokens

- Varying the input prompt size: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Varying the output prompt size: 8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873 tokens.
- Fixed sequence length: 8192 tokens.

In figure 3.20 we observe that the results remain consistent while varying the ratio between input and output tokens. The kernel that takes the largest portion of the execution time is GEMM_1, the same kernel discussed earlier when varying the number of output tokens. It accounts for approximately 40% of the total execution time. GEMV_1, also analyzed previously, is the kernel with the third highest execution time share. In addition to these two, we identify two new kernels: Elementwise 4 and GEMV_3.

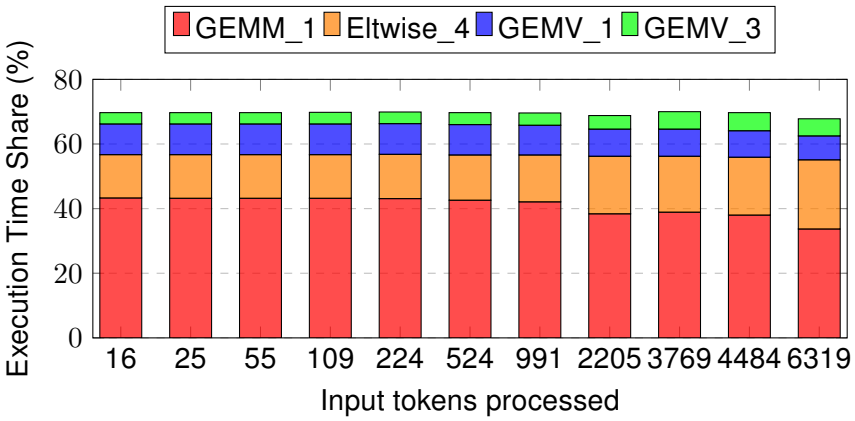


Figure 3.20: Full model percentage of total execution time for the four most time-consuming kernels while varying the ratio of input prompt and output tokens.

In terms of bandwidth requirements, we can see in Figure 3.21 that GEMM_1 and GEMV_1 show the same behavior as observed when varying the number of output tokens. On the other hand, the GEMV_3 kernel has a significant impact, saturating the bandwidth and reaching up to 1.2 terabytes per second. In contrast, the Elementwise_4 kernel has a lower impact on bandwidth usage.

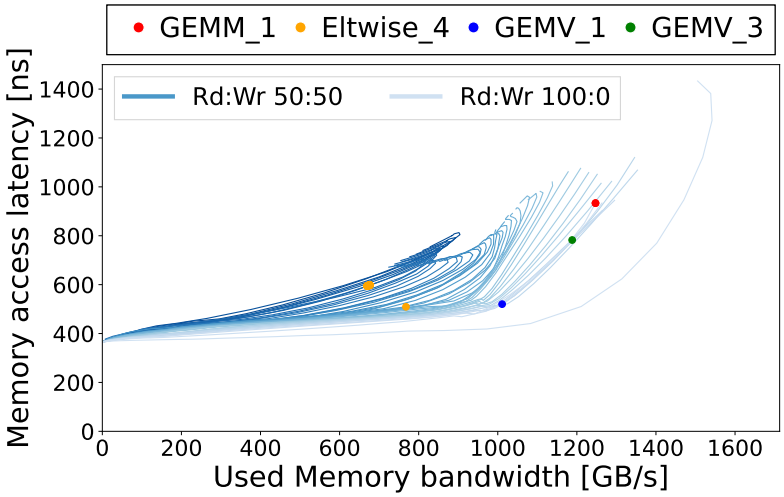


Figure 3.21: Bandwidth latency curves for the most time consuming while varying the ratio of input prompt and output tokens.

In the roofline model shown in Figure 3.22, we can see that GEMM_1 and GEMV_1 show the behavior previously described. On the other hand, again Elementwise_4 and GEMV_3 do not reach the compute roof. In all cases the kernels are near the DRAM roof.

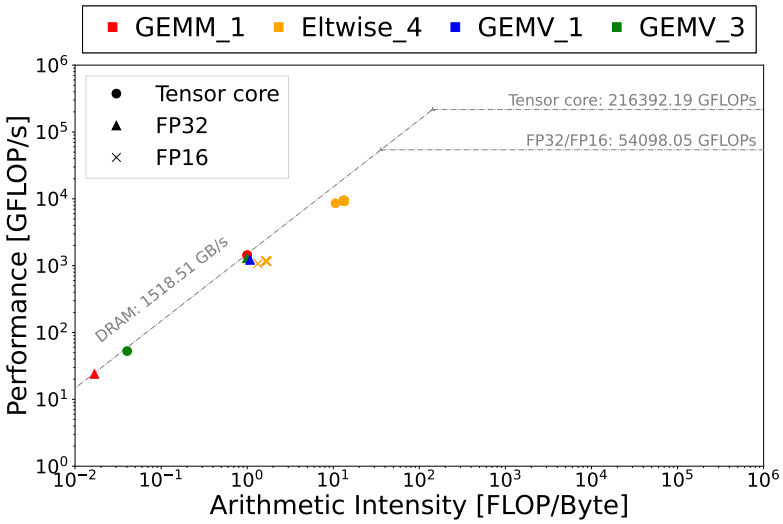


Figure 3.22: Performance and arithmetic intensity of main kernels when the number of input tokens is over 1000 in the full model.

Takeouts:

1. When varying the number of output tokens:
   - 1.1 The most time-consuming kernels are GEMMs and GEMVs operations.
   - 1.2 All of them have a high bandwidth requirement, ranging form 700GB/s to 1,4TB/s.
   - 1.3 Non of the kernels saturate the compute capabilities of the GPU.
2. When varying the number of input tokens:
   - 2.1 When the input is small the main kernels are the same GEMMs and GEMVs over-served while varying the output tokens.
   - 2.2 When the input is big, the most time-consuming kernels are SoftMax and elementwise operations.
   - 2.3 This SoftMax and elementwise operations have a wide range of bandwidth requirement, depending on the grid and block sizes used during the invocation.
   - 2.4 Non of the kernels saturate the compute capabilities of the GPU.
3. When varying the ratio of input and output tokens:
   - 3.1 The most time-consuming kernels are GEMMs, GEMVs and elementwise operations.
   - 3.2 The results are constant for all the rations of input and output tokens tested.
   - 3.3 All of them have a high bandwidth requirement, ranging form 700GB/s to 1,4TB/s.
   - 3.4 Non of the kernels saturate the compute capabilities of the GPU.

3.4.2 Stages

In this group of tests we make the measurements dividing the model in two stages, prefill and decode:

- **Prefill stage**: The model processes the input prompt to generate the first output token.
- **Decode stage**: The model processes the all the tokens processed so far to generate the next one.

We divide in two the generation loop of the model in the `generation.py` file. In the first division we profile the first iteration of the loop which performs the **prefill stage**. In the second division, we profile the rest of the loop iterations together matching the **decode stage**.

3.4.2.1 Time measurements

In this group of experiments, we measure how the execution time is affected by varying the input prompt length, the output sequence length, and the ratio between them, while keeping the total sequence length fixed, as described in Section 3.3.3. The profiler characteristics are:

- **Profiler used:** Nvidia Nsight Systems.
- **Traces measured:** nvtx.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

In Figure 3.23 and Table 3.10 we can see how the execution time of the prefill stage is constant. This is because the prefill stage only processes the input prompt to generate the first token. Given that we fix the input prompt, the execution do not vary, taking in all cases around 0.3 s.

Output Tokens	Prefill Execution Time (s)
504	0.32
1016	0.32
2040	0.30
4088	0.35
8184	0.32

Table 3.10: Prefill inference time for different numbers of output tokens generated.

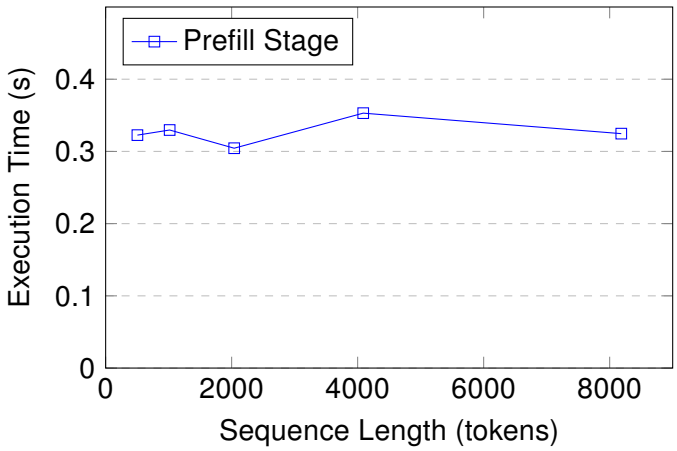


Figure 3.23: Prefill inference time for different numbers of output tokens generated.

On the other hand, in Figure 3.24 and Table 3.11 we can see how the execution time of the decode stage increases linearly with the number of output tokens generated. This behavior matches the one observed in the experiments of the full model, explained in Section 3.4.1.1.

Output Tokens	Decode Execution Time (s)
504	7.7
1016	16.1
2040	33.5
4088	71.4
8184	160.3

Table 3.11: Decode inference time for different numbers of output tokens generated.

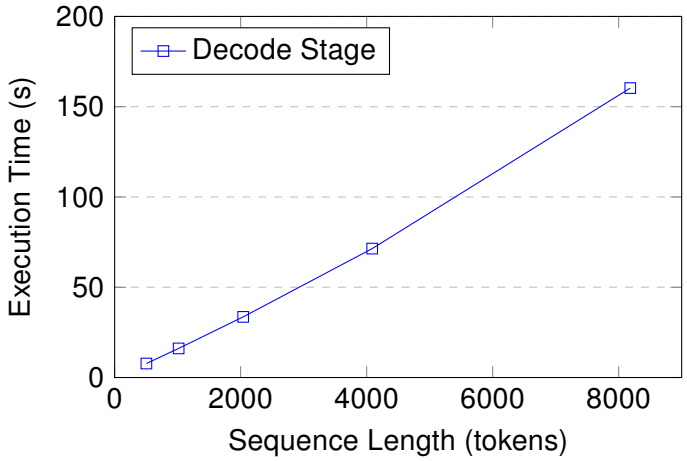


Figure 3.24: Decode inference time for different numbers of output tokens generated.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

In Figure 3.25 and Table 3.12, we observe the same behavior described in Section 3.4.1.1 when varying the input prompt size. As expected, the execution time of the prefill stage increases with the input prompt length. On the other hand, the decode stage shows no variation, since it processes only the output tokens, which in this case are fixed to 8. We also confirm that although increasing the number of input tokens affects the execution time of the prefill stage, this impact is significantly smaller than the effect of increasing output tokens in the decode stage (as discussed in Section 3.4.1.1). For example, expanding the input prompt to **4,088 tokens** results in a prefill stage time of 0.97 s, while generating this amount of tokens as an output output causes the decode stage to take 71.40 s. This confirms once again that increasing the input prompt length has a lower performance cost than increasing the number of generated output tokens.

Input Tokens	Inference Time (s)
16	0.31
25	0.30
55	0.31
109	0.31
224	0.29
524	0.35
991	0.33
2205	0.50
3769	0.80
4484	0.97
6319	1.65

Table 3.12: Prefill inference time for different input prompt sizes.

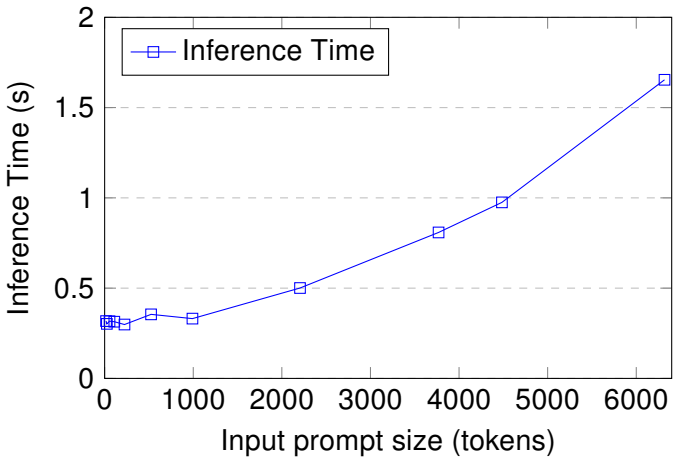


Figure 3.25: Prefill inference time for different input prompt sizes.

Varying the relation of total input and output tokens

- Varying the input prompt size: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Varying the output prompt size: 8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873 tokens.
- Fixed sequence length: 8192 tokens.

As we increase the input prompt size, the percentage of tokens from the total that are processed in the prefill stage increases. Given that the total sequence length is fixed, the amount of tokens processed in the decode stage decreases, as explained in Section 3.4.2. This way, while increasing the input prompt the execution time of the prefill stage increases as shown in Table 3.13 and Figure 3.26. On the other hand, the execution time of the decode stage decreases as shown in Table 3.14 and Figure 3.27. We also can see how increasing the number of tokens processed by the decode stage, has a bigger impact on the overall execution time. This matches the behavior shown in Section 3.4.1.1, where the execution time increased when the percentage of output tokens generated increased.

This also confirms onces again the bigger impact that the decode stage has on the overall execution time of the model, compared to the prefill stage.

Relation in/out (%)	Prefill Time (s)
0.2 : 99.8	0.33
0.3 : 99.7	0.28
0.7 : 99.3	0.29
1.3 : 98.7	0.30
2.7 : 97.3	0.27
6.4 : 93.6	0.31
12.1 : 87.9	0.30
26.9 : 73.1	0.50
46.0 : 54.0	0.81
54.7 : 45.3	0.96
77.1 : 22.9	1.63

Table 3.13: Prefill execution time for different ratios of input prompt and output tokens.

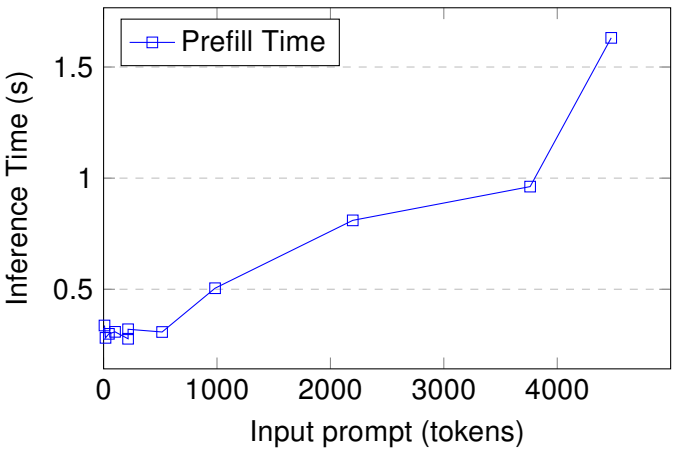


Figure 3.26: Prefill time evolucion while increasing the number of the input prompt.

Relation in/out (%)	Decode Time (s)
0.1 : 99.9	161.7
0.2 : 99.8	162.5
0.5 : 99.5	159.3
1.2 : 98.8	158.7
2.6 : 97.4	158.5
6.2 : 93.8	153.0
11.9 : 88.1	146.0
26.9 : 73.1	123.6
45.9 : 54.1	93.7
54.9 : 45.1	79.7
76.8 : 23.2	41.8

Table 3.14: Decode inference time for different ratios of input prompt and output tokens

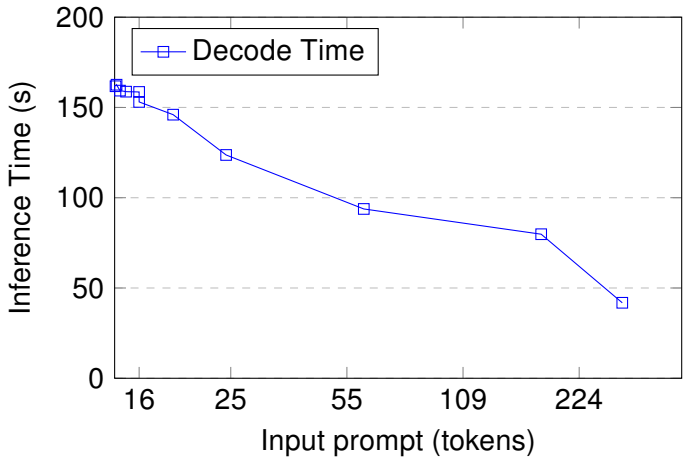


Figure 3.27: Decode inference time for different ratios of input prompt and output tokens

Takeouts:

1. The execution time of the prefill stage increases with the number of input tokens processed.
2. The execution time of the decode stage increases with the number of output tokens generated.
3. The decode stage execution time has a bigger impact on the overall execution time than the prefill stage.

3.4.2.2 Memory operations

In this group of experiments we measure the effect that varying the model parameters have over the memory operations. In specific we will distinguish between four operations: Memcpy DtoD, Memcpy DtoH, Memcpy HtoD and Memset as described in Section 3.3.3. The profiler characteristics are:

- Profiler: Nvidia Nsight Systems.
- Traces measured: cuda,nvtx,osrt,cudnn,cublas.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

In both stages, the memory operations registered are *Memcpy DtoD*, *Memcpy HtoD* and *Memset*. We see how there is no data movements form the host to the device. This is because the only *Memcpy HtoD* operations happen during the parameter load, before the prefill and decode stages. This behavior aligns with what we saw in Section 3.4.1.2 where the amount of memory copied form host to the device matches the size of the parameters of the model, 16 GB.

The prefill stage processes the input prompt to generate the first token. Given that the input prompt is fixed in all cases, the amount of memory copied and set during this stage is constant for all memory operations. In specific for the *Memcpy DtoD* operation, the amount of data copied is around 2.61 MB, for the *Memcpy HtoD* operation is around 0.01 MB and for the *Memset* operation is less than 1.0 kB.

On the other hand, we can see how in the decode stage the amount of memory copied and set increases linearly with the number of output tokens generated, reaching up to 6 GB as shown in Figure 3.28 and Table 3.15. The same behavior is observed for the execution time of the memory operations, as shown in Figure 3.29.

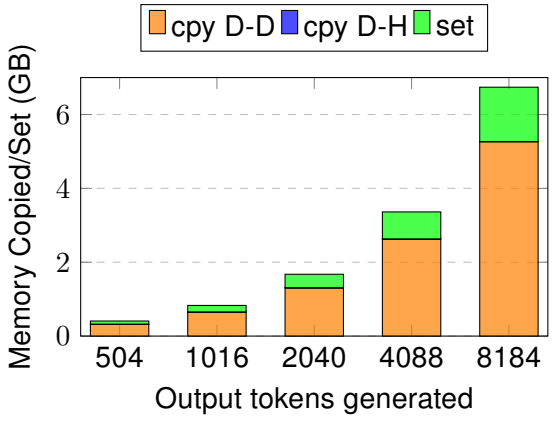


Figure 3.28: Decode stage memory operations size while increasing the number of the output tokens.

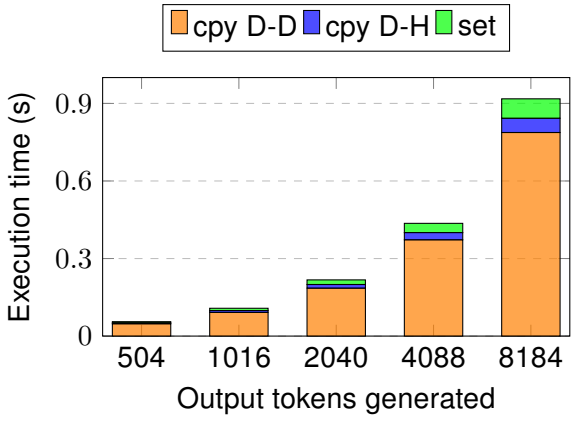


Figure 3.29: Decode stage memory operations execution time while increasing the number of the output tokens.

Output Tokens	Memcpy D-D (GB)	Memcpy D-H (GB)	Memset (GB)
504	0.3	0.000005	0.08
1016	0.6	0.000010	0.18
2040	1.3	0.000020	0.36
4088	2.6	0.000041	0.73
8184	5.2	0.000082	1.47

Table 3.15: Decode stage memory operations size in GigaBytes while varying the number of output tokens generated.

Finally, we can also see how the amount of time spent in this memory operations is again low, under 1% as shown in Figure 3.30.

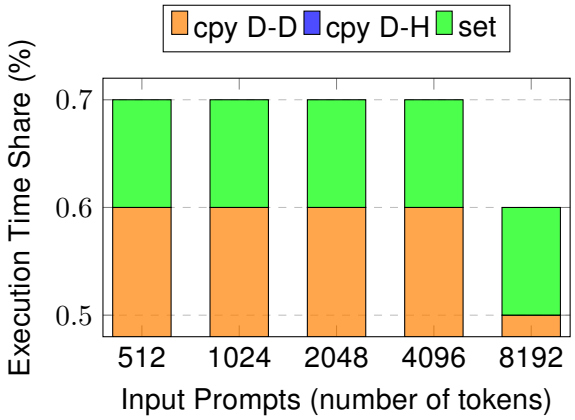


Figure 3.30: Percentage of execution time spent on memory operations during decode.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

We see how *Memcpy DtoD*, *Memcpy DtoH*, and *Memset* operations scale with the input prompt size. Among these, *Memcpy DtoD* exhibits the most significant increase, reaching up to 800 MB (see Figure 3.31). These findings align with the results presented in Section 3.4.1.2, demonstrating that memory demands primarily grow during the prefill stage when the input prompt size increases. Moreover, the execution time of these operations remains negligible, always below 2 ms during the whole execution. In all cases, this constitutes less than 0.1% of the total stage execution time.

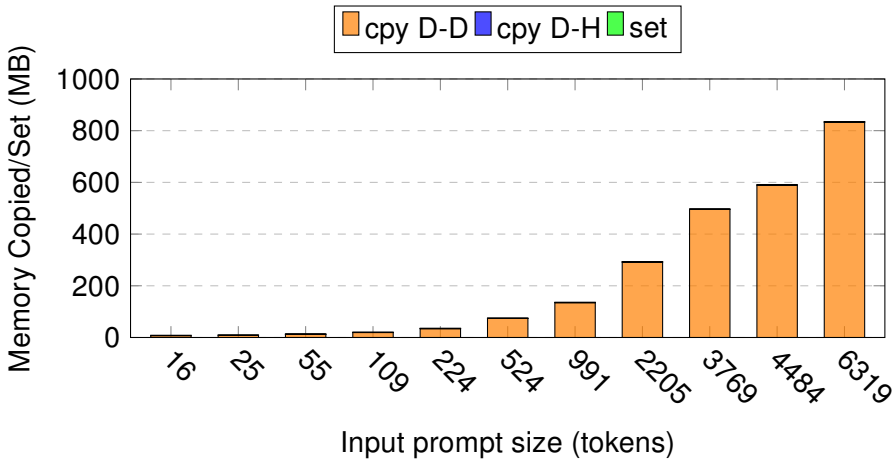


Figure 3.31: Prefill stage memory operations size while increase the input prompt size.

Varying the relation of total input and output tokens

- Varying the input prompt size: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Varying the output prompt size: 8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873 tokens.
- Fixed sequence length: 8192 tokens.

We further validate the results from previous experiments, demonstrating that increasing the input prompt size primarily affects the memory footprint of the prefill stage, while generating more output tokens predominantly increases the memory usage during the decode stage. This relationship is clearly illustrated in Figure 3.32 and Figure 3.33. The execution time patterns remain consistent with those observed when independently varying input prompts and output tokens in earlier tests.

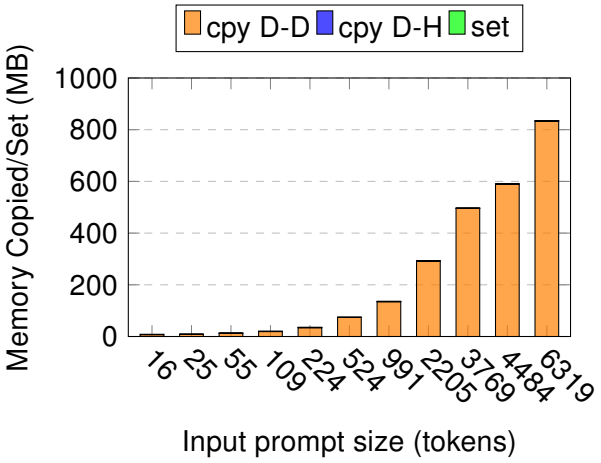


Figure 3.32: Prefill stage memory operations size while varying the rate of input and output tokens.

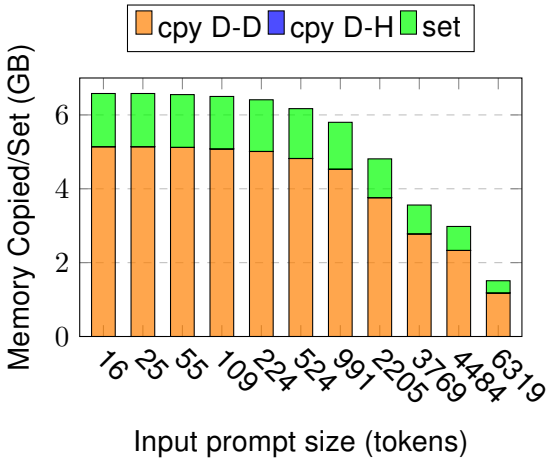


Figure 3.33: Decode stage memory operations size while varying the rate of input and output tokens.

Takeouts:

1. **Input prompt size variation** mainly affects memory operations in the prefill stage.
2. **Output size variation** mainly affects memory operations in the decode stage.
3. The decode stage consistently exhibits a **larger memory footprint** compared to the prefill stage.
4. The prefill stage is dominated by *Memcpy D-D* operations, while the decode stage involves both *Memcpy D-D* and *Memset* operations.
5. In all cases, memory operations account for **less than 1%** of total execution time in both stages.

3.4.2.3 Kernel executions

In this group of tests we study the characteristics of the four most relevant kernels during the execution of the model. This kernels are selected in relation to their execution time as explained in Section 3.3.3. The profilers characteristics are:

- Profiler: Nvidia Nsight Systems.
  - Traces measured: cuda,nvtx,osrt,cudnn,cublas
- Profiler: Nvidia Nsight Compute.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

In the prefill stage, since this phase processes the input prompt—which is fixed—the set of execution kernels remains the same across runs. As a result, the percentage of total execution time for each kernel is also consistent. The main kernels is GEMM_1 (65%) previously described in Section 3.4.1 along with a Reduce (5%), GEMM_2 (4%), and GEMM_4 (4%) kernels.

In the decode stage, the main kernels are GEMM_1, GEMV_1, GEMM_2, and GEMV_2, which together account for 60% to 80% of the total execution time. These kernels exhibit the same bandwidth demands and arithmetic intensity as observed in the full model execution (see Section 3.4.1.1).

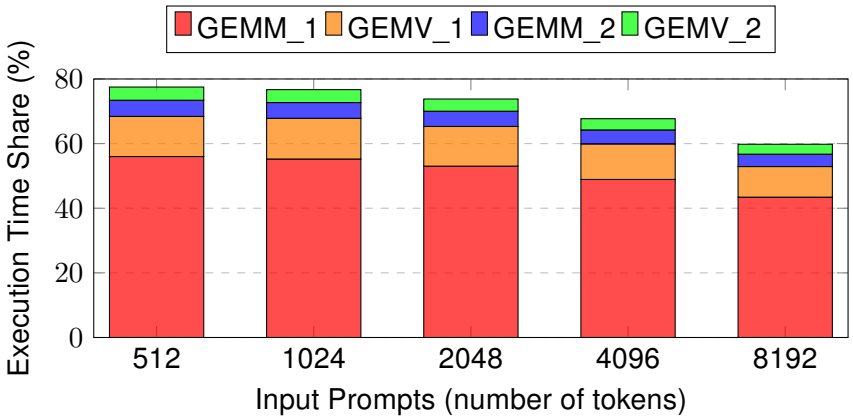


Figure 3.34: Decode stage percentage of total execution time for the four most time-consuming kernels across different output sequence lenghts.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

During the prefill stage, we observe two different behaviors depending on the size of the input prompt. As shown in Figure 3.35, when the input prompt is *small*, the main kernels executed are the previously discussed GEMM_1 and GEMM_2, described in Section 3.4.1, along with two new kernels: a Reduce kernel and a new GEMM_4. However, both this new kernels have a negligible execution time.

We also observe that the execution times are not very consistent. This is explained by two factors:

- For small input sizes, the total prefill time remains around 0.3 seconds. As a result, even small variations in kernel execution can significantly affect the overall runtime.
- Some kernels are selected dynamically at runtime by the PyTorch library. For instance, GEMM_1 disappears when the input prompt is 109 tokens. In this case, the library selects a different GEMM kernel with a tile configuration of $128 \times 256 \times 64$, which makes the execution time of GEMM_1 zero.

In Figure 3.36, when the input prompt is *big*, we observe the same behavior described in Section 3.4.1 while varying the input size. The main kernels executed are SoftMax and several Elementwise operations.

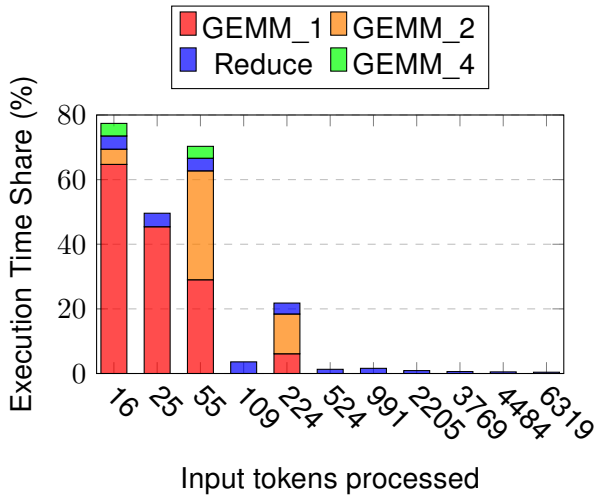


Figure 3.35: Percentage of total execution time for the four most time-consuming kernels during the prefill stage, when varying the number of input tokens with input sizes smaller than 1000 tokens.

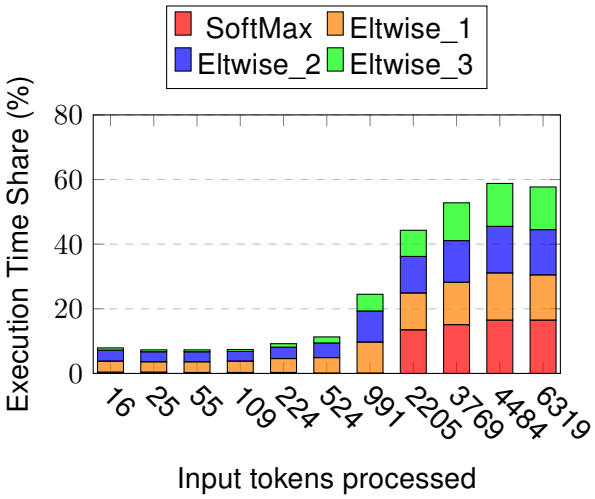


Figure 3.36: Percentage of total execution time for the four most time-consuming kernels during the prefill stage, when varying the number of input tokens with input sizes bigger than 1000 tokens.

On the other hand, in Figure 3.37 we can see how in the decode stage, the main kernels are again GEMM_1, GEMV_1, GEMM_1 and GEMV_2. Explained in Section 3.4.1 while varying the output size.

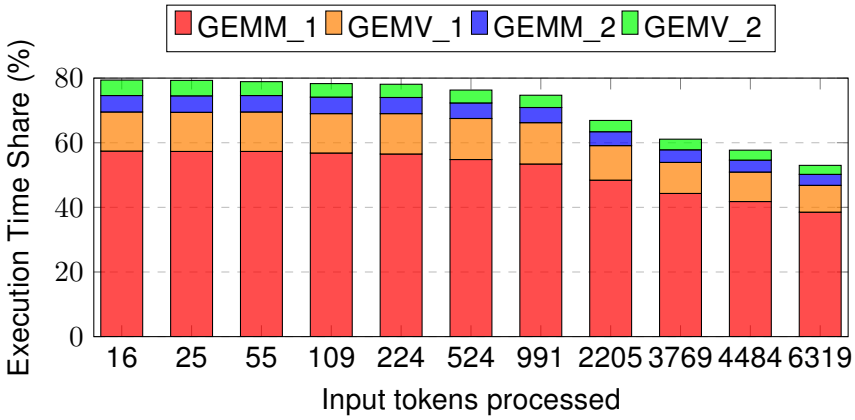


Figure 3.37: Percentage of total execution time for the four most time-consuming kernels during the decode stage, when varying the number of input tokens.

All this kernels have the same bandwidth needs and arithmetic intensity as the ones observed in the full model execution (see Section 3.4.1.1).

Varying the relation of total input and output tokens

- Varying the input prompt size: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Varying the output prompt size: 8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873 tokens.
- Fixed sequence length: 8192 tokens.

During the prefill stage, we again observe two different behaviors depending on whether the input prompt constitutes or not the largest portion of the sequence. In Figure 3.38, we can see that when the input prompt is *small* (less than 1000 tokens), the main kernels executed are GEMM_1, Reduce, GEMM_2, and GEMM_4, as described in previous sections. However, when the input prompt is *large* (more than 1000 tokens), the dominant kernels are SoftMax and various elementwise operations. We also observe inconsistent execution times when the input prompt is small. This is due to the same reasons explained in the previous section: the total prefill time is short, so small changes in kernel execution can have a significant impact, and PyTorch may dynamically select different kernels during runtime.

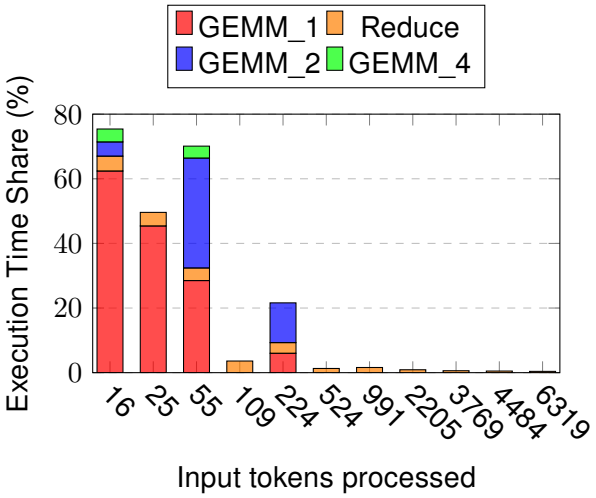


Figure 3.38: Percentage of total execution time for the four most time-consuming kernels during the prefill stage, when varying the ratio of input and output tokens with input sizes smaller than 1000 tokens.

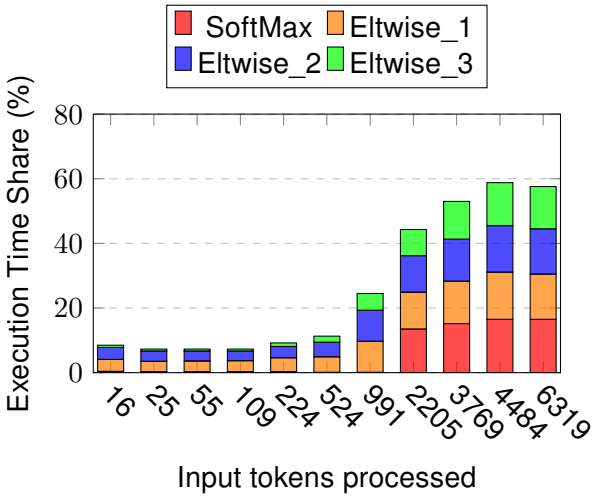


Figure 3.39: Percentage of total execution time for the four most time-consuming kernels during the prefill stage, when varying the ratio of input and output tokens with input sizes bigger than 1000 tokens.

In Figure 3.40 we can see how the main kernels are again GEMM_1, GEMV_1, GEMM_2, and GEMV_2. The bandwidth and arithmetic intensity of this kernels are the same as the ones observed in the full model execution (see Section 3.4.1.1).

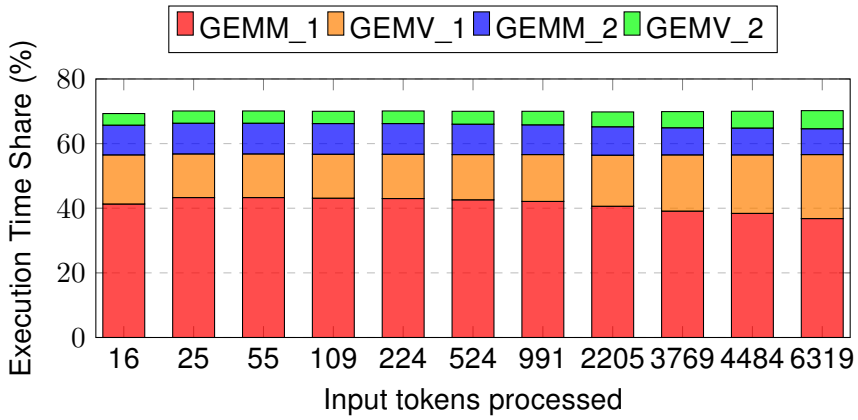


Figure 3.40: Percentage of total execution time for the four most time-consuming kernels during the decode stage, when varying the ratio of input and output tokens.

Takeouts:

1. The main kernels in the decode stage are always the same: GEMM_1, GEMV_1, GEMM_2 and GEMV_2, for all the different scenarios tested. This kernels have the same bandwidth and arithmetic intensity as the ones observed in the full model execution, in general stressing the memory bandwidth without fully utilizing the compute capabilities of the device.
2. The prefill stage, have different kernels depending on the size of the input prompt.
   - When the input prompt is small, the main kernels are GEMM_1, Reduce, GEMM_2 and GEMM_4.
   - When the input prompt is big, the main kernels are SoftMax and several elementwise operations.

   Again this kernels have the same bandwidth and arithmetic intensity as the ones observed in the full model execution.

3.4.3 Layers

3.4.3.1 Time measurements

In this group of experiments we measure how the execution time gets affected while varying the input prompt, output sequence length and maximum sequence length as explained in Section 3.3.3. The profiler characteristics are:

- **Profiler used:** Nvidia Nsight Systems.
- **Traces measured:** cuda,nvtx,osrt,cudnn,cublas.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

In Figure 3.41, we observe that the execution time increases linearly across all layers as the number of generated output tokens increases. Additionally, the Attention layer consistently accounts for the largest portion of execution time. All layers maintain a similar proportion of the total execution time regardless of the number of output tokens. Specifically, the Attention layer accounts for approximately 60%, the Feed-Forward Network (FNN) for 30%, the rotational embeddings (ROT_EMB) for 10%, and the SoftMax layer for around 4%.

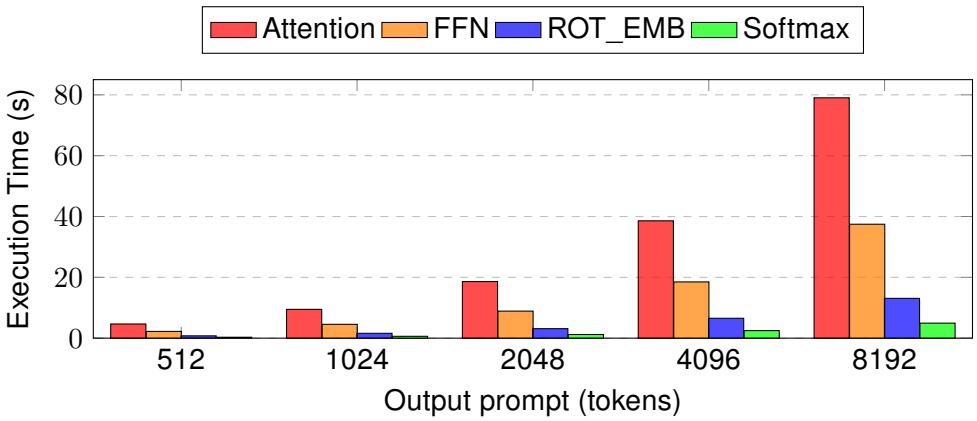


Figure 3.41: Main layers execution time while increasing the number of output tokens.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

Again, in Figure 3.42, we observe that the Attention layer is the one consuming the most execution time, accounting for approximately 60% of the total. The total execution time of the layers remains relatively constant across most input prompt sizes. However, it increases significantly when the input prompt becomes *large* (greater than 3000 tokens), rising from under 0.4 seconds to almost 1 second.

Nevertheless, it is important to note that these execution times are relatively small compared to the time required to generate the output tokens. As a result, even small variations in kernel execution can lead to noticeable fluctuations in the measurements.

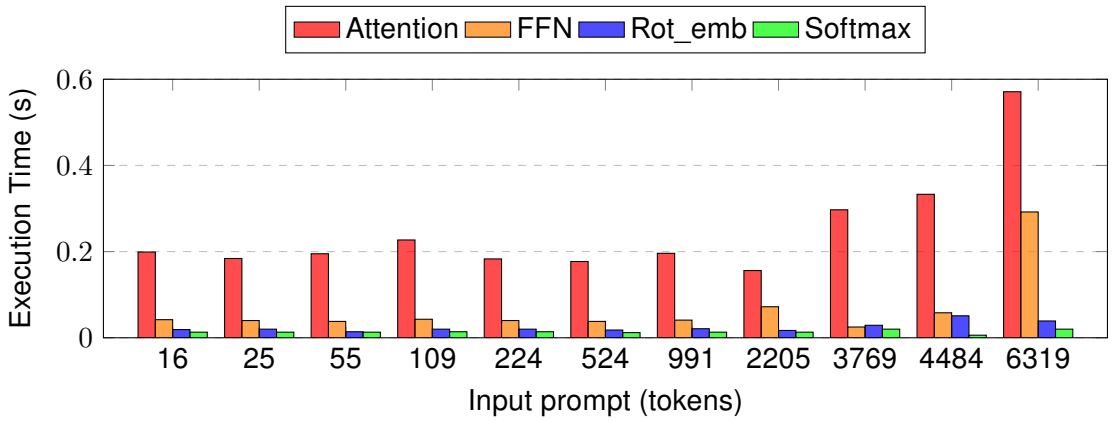


Figure 3.42: Main layers execution time while increasing the number of input tokens.

Varying the relation of total input and output tokens

- Varying the input prompt size: [16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319] tokens.
- Varying the output prompt size: [8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873] tokens.
- Fixed sequence length: [8192] tokens.

As explained in Sections 3.4.1 and 3.4.2, processing tokens as input takes less time than processing them as output. Therefore, when the ratio of input tokens is higher, the total execution time is reduced. We observe the same behavior at the layer level: the total execution time of the layers continues to decrease as the input prompt size increases. However, if we analyze the percentage of total execution time that each layer represents, we can see that it remains constant. Specifically, the distribution is the same as observed when varying the number of output tokens: the Attention layer accounts for 60%, the Feed-Forward Network (FNN) for 30%, the ROT_EMB layer for 10%, and the SoftMax layer for 4%.

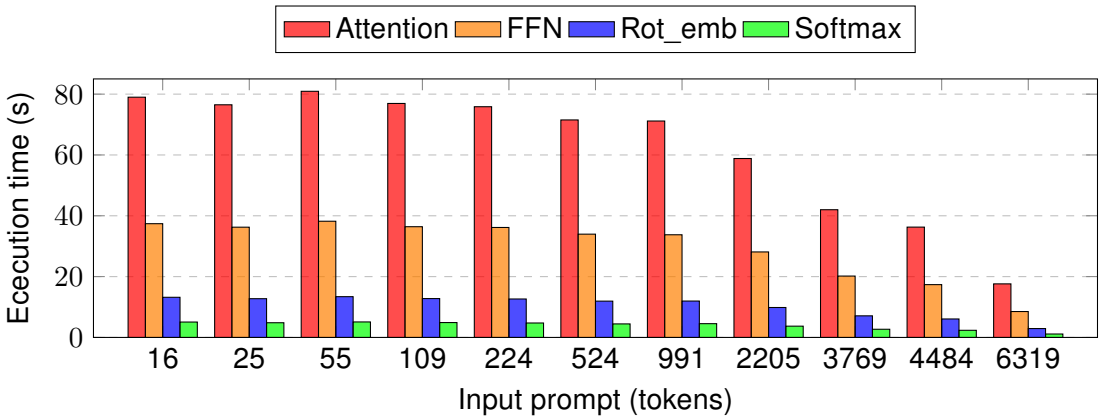


Figure 3.43: Main layers execution time while increasing the ratio of input and output tokens.

Takeouts:

1. Across all experimental scenarios, we observe consistent patterns in layer execution times. The Attention layer consistently requires the most computation time, followed by the Feed-Forward Network (FFN), the Rotary Embedding (Rot_emb) and the Softmax operations layer. This hierarchy remains stable regardless of varying the input prompt and output generated.

3.4.3.2 Memory operations

In this group of experiments we measure how the GPU memory operations are affected while varying the input prompt, output sequence length and maximum sequence length. In specific we will distinguish between four operations: Memcpy DtoD, Memcpy DtoH, Memcpy HtoD, Memset as described in Section 3.3.3. The execution is done allocating a full node to guarantee isolated executions. The profiler characteristics are:

- Profiler: Nvidia Nsight Systems.
- Traces measured: cuda,nvtx,osrt,cudnn,cublas.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

For all the output token configurations tested, we observed that only one type of memory operation occurs: *Memcpy DtoD*, which copies data between locations within the GPU memory. The only layer active during these memory operations is the Attention layer. As shown in Figure 3.44, the amount of memory copied increases linearly, doubling as the number of generated output tokens doubles. This behavior is due to the use of KV caches, as explained in Section 3.4.1. The KV cache, used by the Attention layer, helps avoid recomputation by storing the attention values of previous elements in the sequence. However, this optimization comes with the cost of a linear

increase in memory usage, as shown in Figure 3.44. We verified this empirically by running the same experiment with the KV cache disabled. In that case, no Memcpy operations were observed.

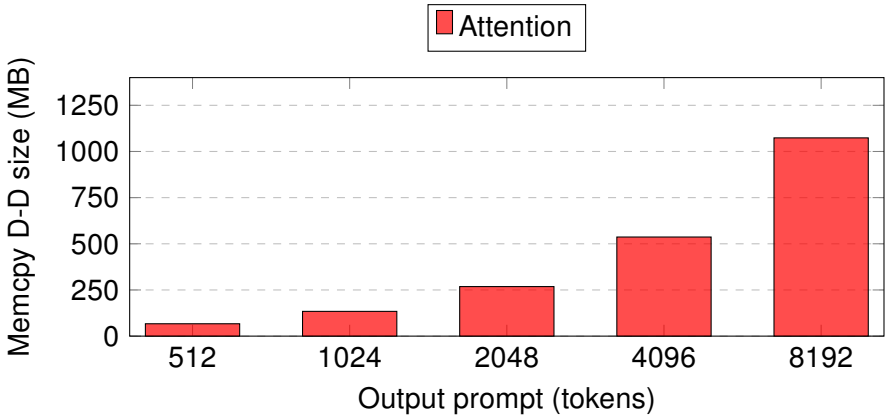


Figure 3.44: Attention layer Memcpy D-D operation xsize while increasing the output tokens generated.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

When we increase the input prompt size, we observe a similar behavior to that seen when increasing the number of output tokens. The only memory operation involved is *Memcpy DtoD*, and the only layer active during this operation is the Attention layer.

In general, the amount of memory copied increases as the input prompt size increases. However, an exception occurs at a prompt size of 6319 tokens, where the copied memory drops to as little as 26.8 MB. It is important to note that repeating the experiment multiple times yields varying results. In some runs, the copied memory reaches up to 60 MB. This variability makes it difficult to establish a consistent relationship between input prompt size and the amount of memory copied.

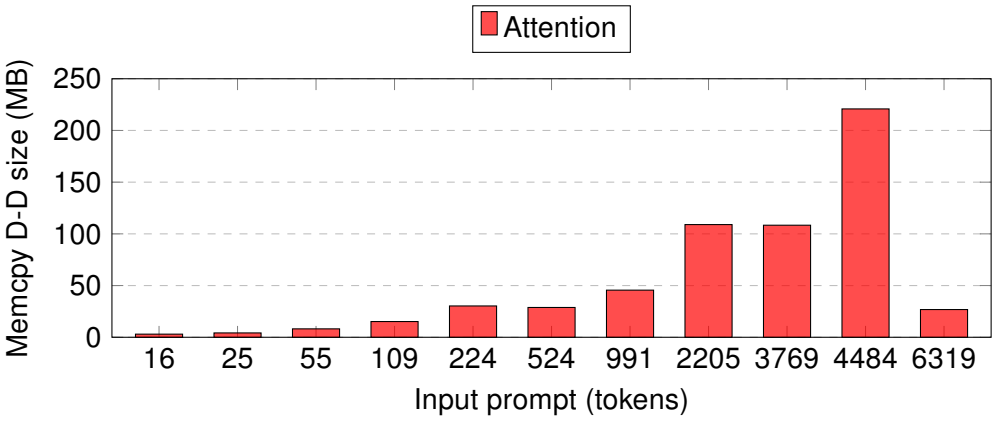


Figure 3.45: Attention layer Memcpy D-D operation size while increasing the input prompt size.

Varying the relation of total input and output tokens

- Varying the input prompt size: [16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319] tokens.
- Varying the output prompt size: [8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873] tokens.
- Fixed sequence length: [8192] tokens.

As in the previous experiments, the only layer that performs memory operations is the Attention layer. In Figure 3.46, we observe that the amount of memory copied decreases as the ratio of input tokens increases. This behavior is consistent with previous results, where increasing the number of generated output tokens led to a larger memory footprint compared to increasing the input prompt size. It also reflects the observation that memory requirements when varying the input prompt are not consistent across repeated experiments.

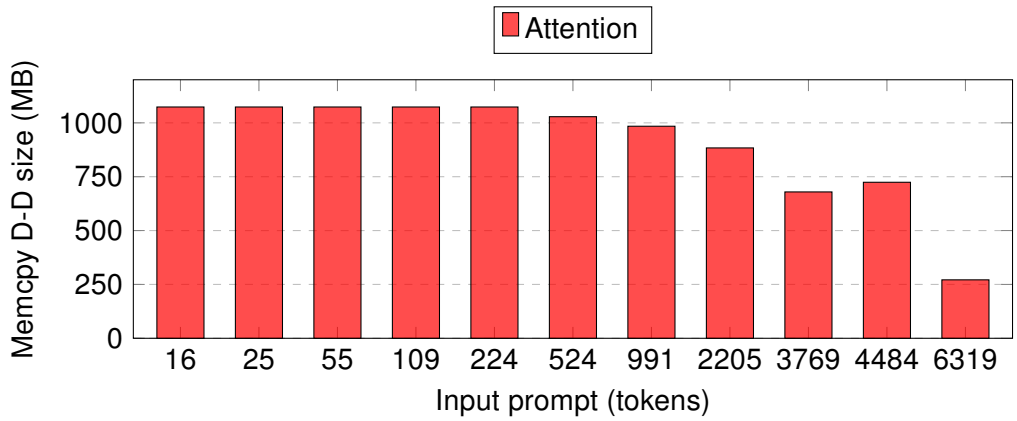


Figure 3.46: Attention layer Memcpy D-D size while varying the ratio of input and output tokens.

Takeouts:

1. The only layer that performs memory operations is the Attention layer, mainly because of the use of the KV cache.
2. *Memcpy DtoD* is the only memory operation observed in all cases.

3.4.3.3 Kernel executions

In this group of tests we study the characteristics of the four most relevant kernels during the execution of the model. This kernels are selected in relation to their execution time as explained 3.3.3. The profilers characteristics are:

- Profiler: Nvidia Nsight Systems.
  - Traces measured: cuda,nvtx,osrt,cudnn,cublas
- Profiler: Nvidia Nsight Compute.

Varying the number of output tokens

- Fixed input prompt: 8 tokens.
- Output tokens generated: 504, 1016, 2040, 4088, 8184 tokens.
- Total sequence length: 512, 1024, 2048, 4096, 8192 tokens.

Attention layer:

In Figure 3.47, we can see that the share of total execution time taken by each kernel remains constant across all cases. The Attention layer is consistently executed using two GEMM kernels and two elementwise kernels. Specifically, these kernels are: GEMM_1, which accounts for 38% of the execution time; GEMM_4, with 13%; Elementwise_1, with 7%; and Elementwise_2, also with 7% (all of them described in Section 7.2).

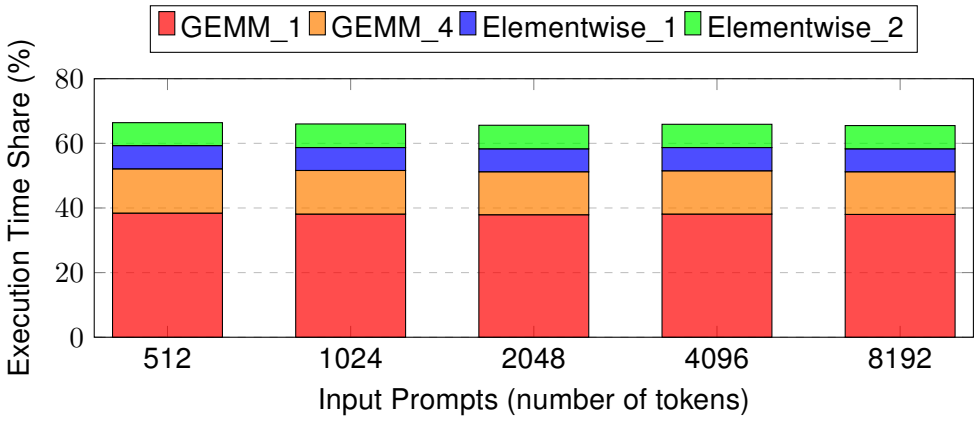


Figure 3.47: Attention layer percentage of total execution time for the four most time-consuming kernels across different output sequence lengths.

In Figure 3.48, we observe once again, as explained in Section 3.4.1, that the elementwise kernels do not perform arithmetic operations. Instead, they are used to copy data between different locations in GPU memory. On the other hand, both GEMM kernels execute operations using FP32 and Tensor Cores and place significant pressure on memory bandwidth. However, they still underutilize the compute capabilities of the GPU.

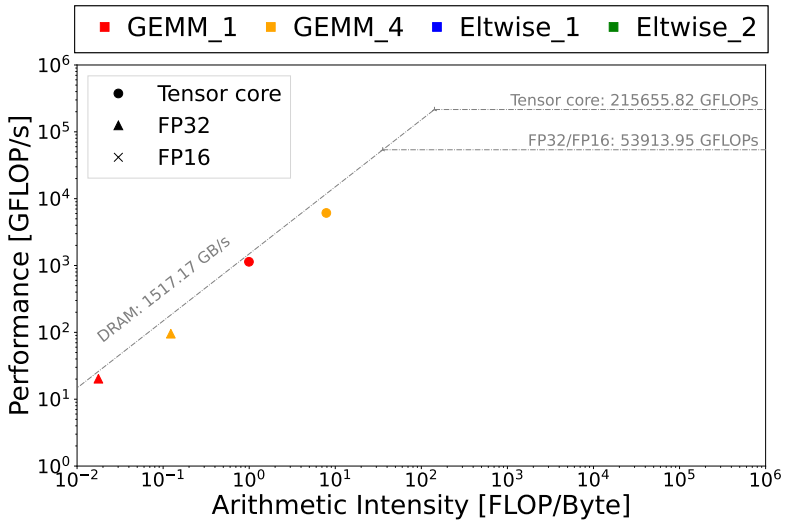


Figure 3.48: Performance and arithmetic intensity of main kernels during the Attention layer execution while varying the number of output tokens.

Feed Forward Network (FNN) layer:

In the Feed-Forward Network (FNN), we again observe that the set of kernels remains consistent across all cases. GEMM_1 kernel is the most time-consuming, accounting for over 80% of the total execution time.

In addition to this, we observe a Reduce kernel, which represents approximately 3% of the execution time, as well as the previously identified Elementwise_2 kernel. A new kernel, Elementwise_5, also appears. However, both elementwise kernels contribute very little to the total execution time, around 1%

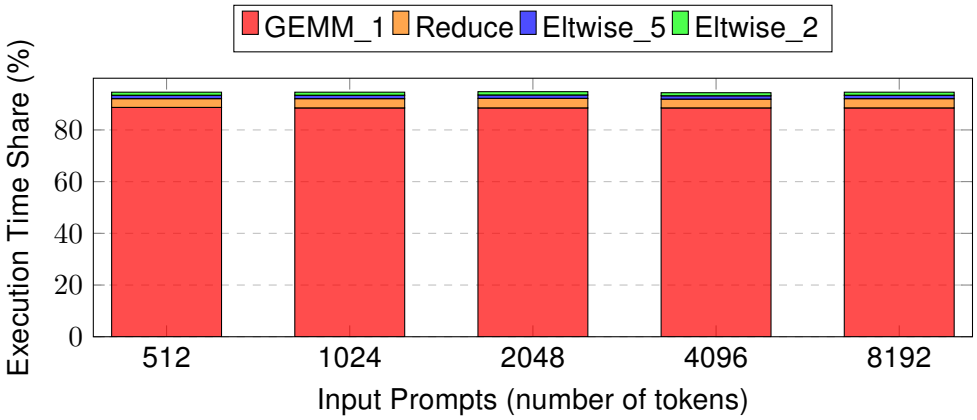


Figure 3.49: Feed Forward Network layer percentage of total execution time for the four most time-consuming kernels across different output sequence lengths

In Figure 3.50, we observe the following: GEMM_1 uses both FP32 and Tensor Cores and saturates the GPU memory bandwidth. The Elementwise_2 kernel, as before, does not perform any arithmetic operations. The Reduce and Elementwise_5 kernels show lower bandwidth usage. However, none of the kernels fully utilize the compute capabilities of the GPU.

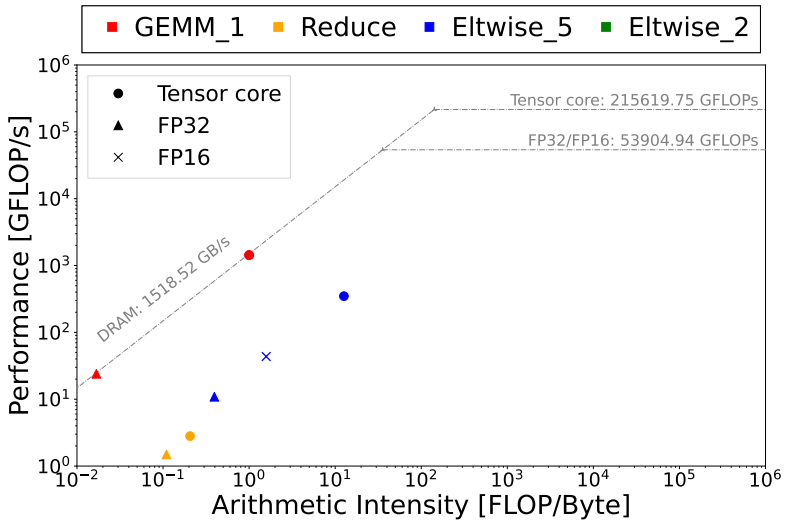


Figure 3.50: Performance and arithmetic intensity of main kernels during the Feed Forward Network layer execution while varying the number of output tokens.

Rotationary Embedding (Rot_emb) layer:

Once again, we observe that the set of kernels remains the same in all cases. In this example, the rot_emb layer only uses three element-wise kernels. Elementwise_1 accounts for 35% of the execution time, Elementwise_2 for 33%, and Elementwise_6 for 32%. In Figure 3.51, we can see again that Elementwise_1 and Elementwise_3 do not perform arithmetic operations, so they are not shown in the roofline plot. We also observe that Elementwise_6 does not use a high memory bandwidth and does not have a high arithmetic intensity. We also notice the great variability in how this kernel runs. It performs operations using the tensor cores and works with both FP32 and FP16 data types. In addition, it is executed with different block and grid sizes, which results in varying performance and arithmetic intensities.

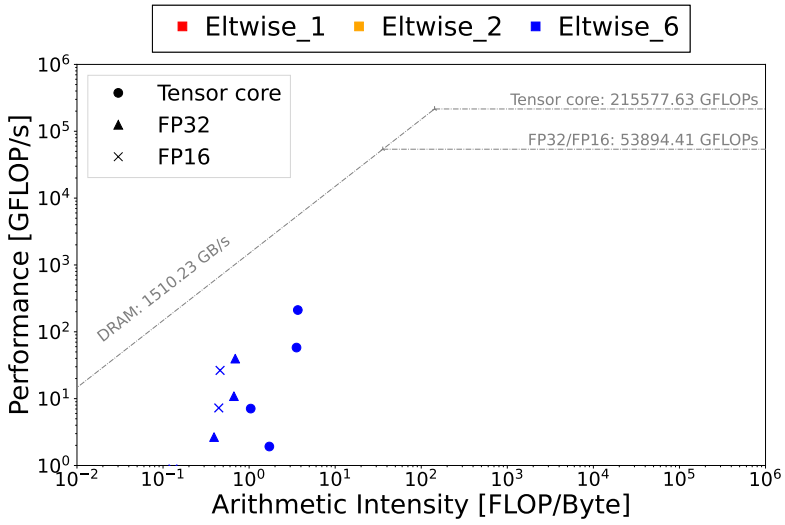


Figure 3.51: Performance and arithmetic intensity of main kernels during the Rotationary Embedding layer execution while varying the number of output tokens.

SoftMax layer:

The layer is composed of three kernels: Elementwise_2, which takes 39% of the execution time; Elementwise_1, which takes 34%; and a softmax_2 operation, which takes the remaining 27%. In Figure 3.52, we can see how that the Elementwise_1 and Elementwise_2 kernels do not perform arithmetic operations, so they are not shown in the roofline plot. The softmax_2 kernel uses only FP32 cores and never reaches the maximum bandwidth or compute capacity of the device.

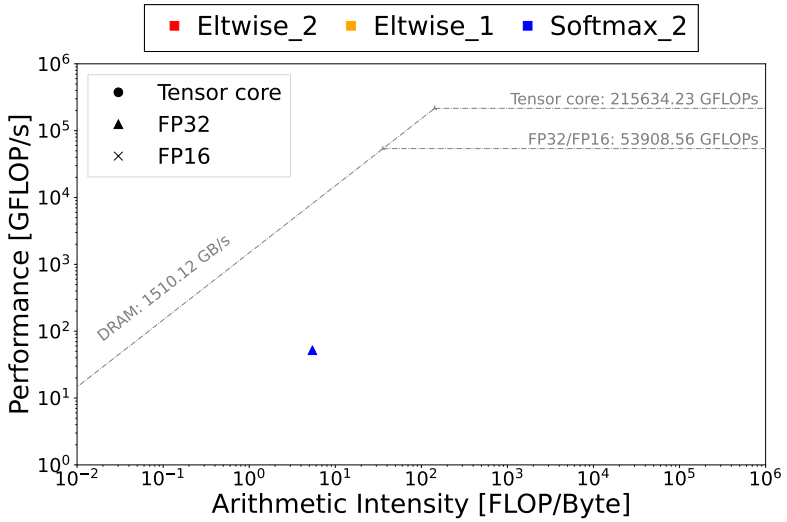


Figure 3.52: Performance and arithmetic intensity of main kernels during the SoftMax layer execution while varying the number of output tokens.

If we analyze the behavior of the layers based on their main kernels, we can compute the weighted average of the arithmetic intensity and performance to represent the whole layer with a single point for a specific data type and core used. In this way, we combine the values of each kernel and compute the average, taking into account the execution time of each kernel. That is, for a given layer, we calculate the weighted sum of the arithmetic intensity (in GFLOP/s per byte) and the performance (in GFLOP/s), using the percentage of execution time that each kernel represents:

$$\text{Arithmetic Intensity}_{\text{layer}} = \sum_{i=1}^{n} (\text{AI}_i \cdot w_i) \tag{3.8}$$

$$\text{Performance}_{\text{layer}} = \sum_{i=1}^{n} (\text{Perf}_i \cdot w_i) \tag{3.9}$$

where AI_i is the arithmetic intensity of kernel i in GFLOP/s per byte, Perf_i is the performance of kernel i in GFLOP/s, and w_i is the weight of kernel i based on its share of the total execution time.

Using this method, we obtain Figure 3.53, where we can see that the Attention and Feed Forward Network layers have the highest bandwidth requirements and the highest number of floating-point operations per second. This is expected, as these layers are mostly occupied by GEMM operations.

On the other hand, the SoftMax and rot_emb layers have the lowest bandwidth usage and the lowest performance.

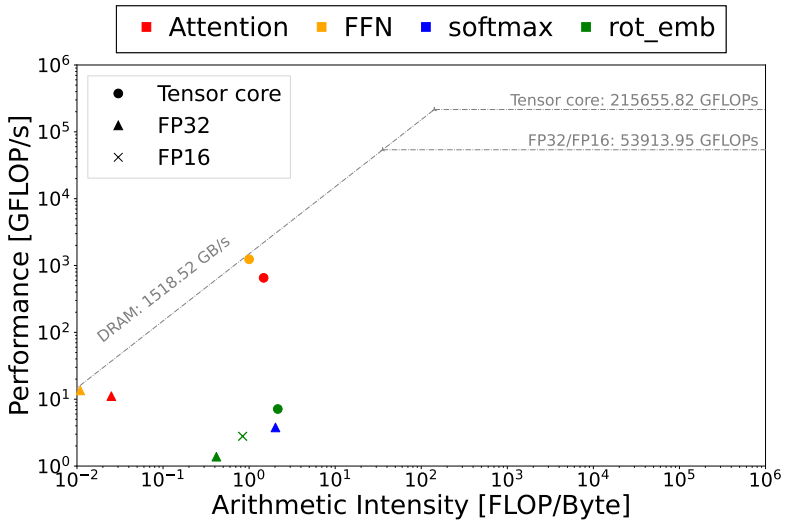


Figure 3.53: Performance and arithmetic intensity of the layers while varying the number of output tokens.

Varying the number of input prompt tokens

- Input prompt tokens: 16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319 tokens.
- Fixed number of output tokens generated: 8 tokens.
- Total sequence length: 24, 33, 63, 117, 232, 532, 999, 2213, 3777, 4492, 6327 tokens.

Attention layer:

We observe the same behavior described in Section 3.4.2.3 when varying the number of input tokens. As in the prefill stage, the main operations are GEMM kernels when the input prompt is small (See Figure 3.54). However, as the input prompt increases, SoftMax and elementwise operations begin to dominate the execution time of the layer (See Figure 3.55). It is important to note that although we study SoftMax as a separate layer, it is also a component of the Attention layer. This explains the presence of SoftMax kernels in this analysis.

As previously discussed in Section 3.4.2.3, some kernels are selected dynamically at runtime by the PyTorch library. For example, GEMM_1 disappears when the input prompt size is 109 tokens, as the library selects a different GEMM kernel with a tile configuration of 128×64×64. In general, for small input prompt sizes, GEMM_1 and GEMM_4 are replaced by alternative kernels with different tile distributions. Similarly, for larger input prompt sizes, the same dynamic substitution occurs with GEMM_5.

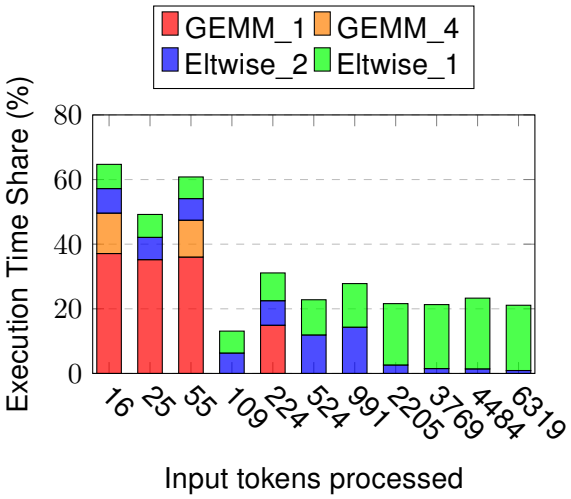


Figure 3.54: Percentage of total execution time for the four most time-consuming kernels during the attention layer, when varying the input prompt with sizes smaller than 1000 tokens.

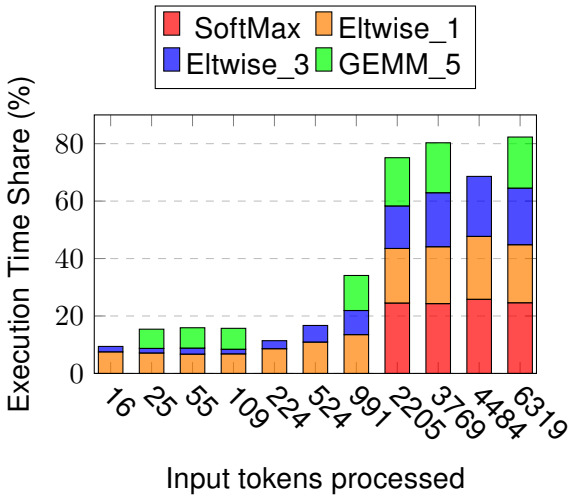


Figure 3.55: Percentage of total execution time for the four most time-consuming kernels during the attention layer, when varying input prompt with sizes bigger than 1000 tokens.

Regarding bandwidth and arithmetic intensity of the kernels, we observe that for small input prompts (less than 1000 tokens), the results match those shown in Figure 3.48, as the same kernels are executed. However, for larger input prompts, the kernel characteristics change significantly, as demonstrated in Figure 3.56. The SoftMax kernel exhibits multiple data points corresponding to different grid and block configurations. Bandwidth saturation occurs primarily in configurations with the largest grid sizes, while all executions maintain similar arithmetic intensity (appearing horizontally aligned when run on the same core type). Among these, Elementwise 3 consistently achieves full bandwidth utilization. Additionally, we observe GEMM_5 exhibiting the highest arithmetic intensity recorded in our experiments thus far. Notably, none of the kernels approach the GPU's theoretical compute limits.

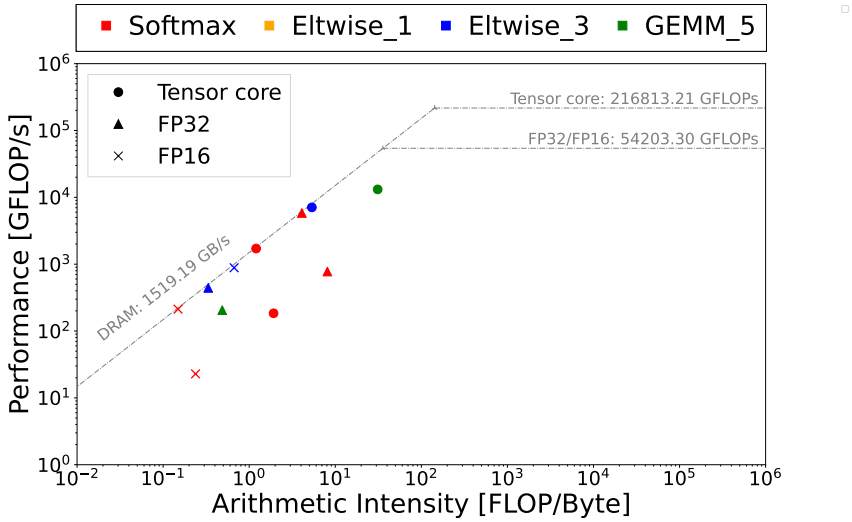


Figure 3.56: Performance and arithmetic intensity of the Attention layer with an input prompt size of 6319 tokens.

Feed Forward Network (FNN) layer:

As within the attention layer, we observe two distinct computational behaviors depending on input prompt size:

1. For **small input prompts**, GEMM_1 remains the most time-consuming kernel, consistent with our earlier findings.
2. For **large input prompts**, SoftMax and element-wise operations dominate the execution time profile.

The arithmetic intensity and kernel performance align with the detailed analysis presented in previous sections.

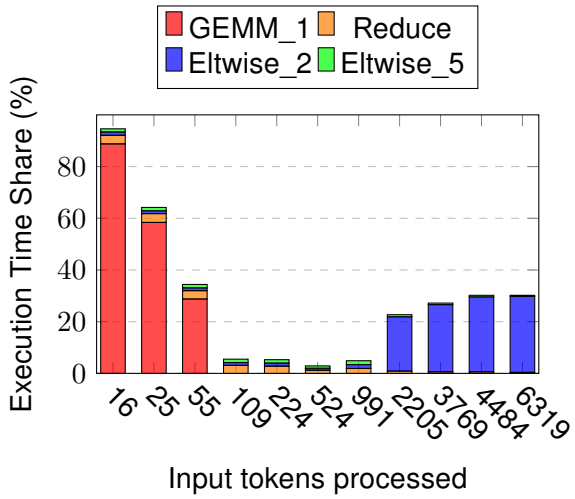


Figure 3.57: Percentage of total execution time for the four most time-consuming kernels during the Feed Forwad Network layer, when the input prompt is small, while varying the number of input prompt.

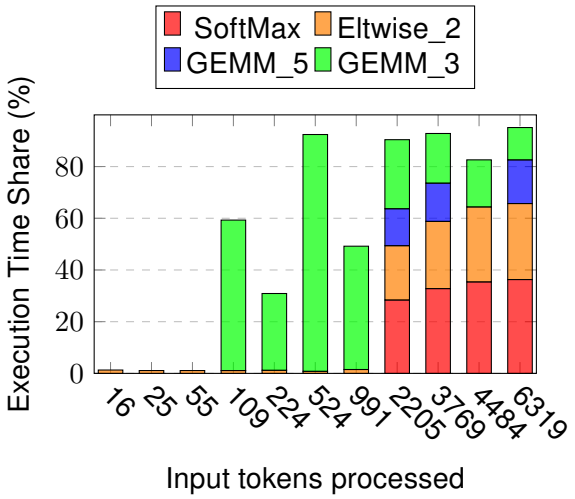


Figure 3.58: Percentage of total execution time for the four most time-consuming kernels during the Feed Forwad Network layer, when the input prompt is small, while varying the number of input prompt.

Rotationary Embedding (Rot_emb) layer:

For small input prompt sizes, we observe the same results as when varying the output generated. In these cases, elementwise kernels dominate, with Elementwise 1 and Elementwise 2 again not perfoming arithmetic operations.

However, as input prompt size increases a GEMM operations appear

These findings regarding arithmetic intensity and kernel performance remain fully consistent with the detailed analysis presented in earlier sections.

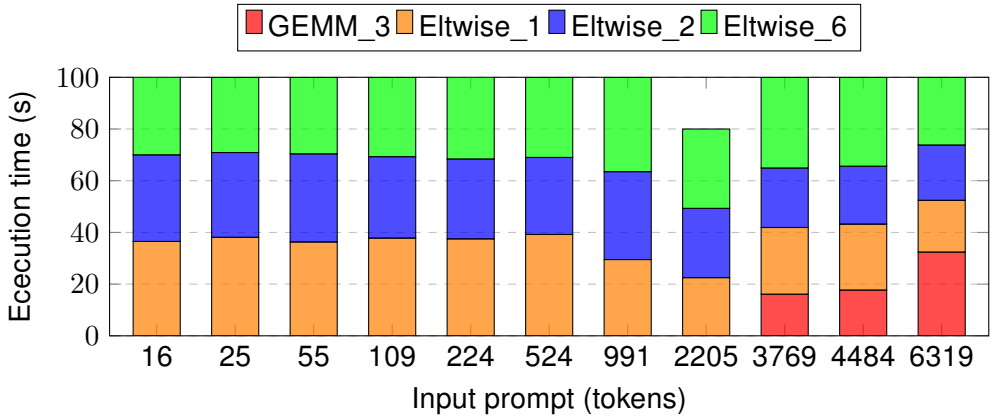


Figure 3.59: Performance and arithmetic intensity of main kernels during the Rotationary Embedding layer execution while varying the number of input tokens.

SoftMax layer:

When the input prompt is small, we observe similar results to those seen when varying the number of output tokens. However, as we increase the input prompt size, we notice the appearance of the gemm_5 operation, which becomes more significant (See Figures 3.67).

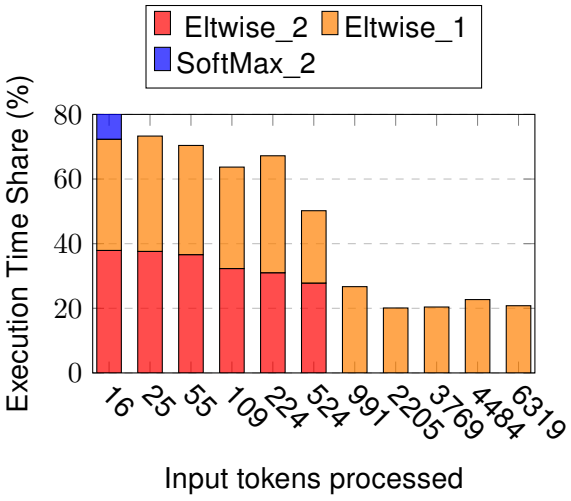


Figure 3.60: Percentage of total execution time for the four most time-consuming kernels during the SoftMax, when the input prompt is small, while varying the size of input prompt.

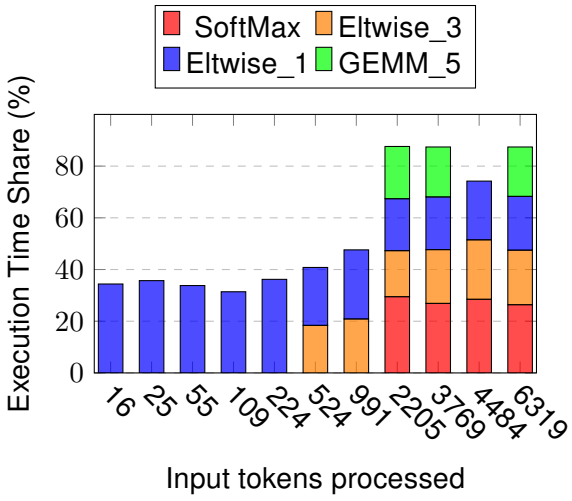


Figure 3.61: Percentage of total execution time for the four most time-consuming kernels during the SoftMax, when the input prompt is big, while varying the size of input prompt.

It is interesting to observe that some operations are dynamically selected at runtime by the PyTorch library. For example, the softmax operation may be executed using different kernels depending on the input size. These kernels vary in terms of the cores they use and their execution patterns.

If we compare the arithmetic intensity and performance of this two softmax kernels (see Figure 3.62), we can see a clear difference. When the input prompt is small, the selected softmax kernel (Softmax_2) uses only the F32 cores and does not fully utilize the compute or memory bandwidth of the device. In contrast, when the input prompt is larger, PyTorch selects a different softmax kernel (Softmax) that runs on both F32 cores and Tensor Cores. In some cases, this kernel reaches full utilization of the available memory bandwidth and compute resources, especially when executed with large grid sizes, as discussed in previous sections.

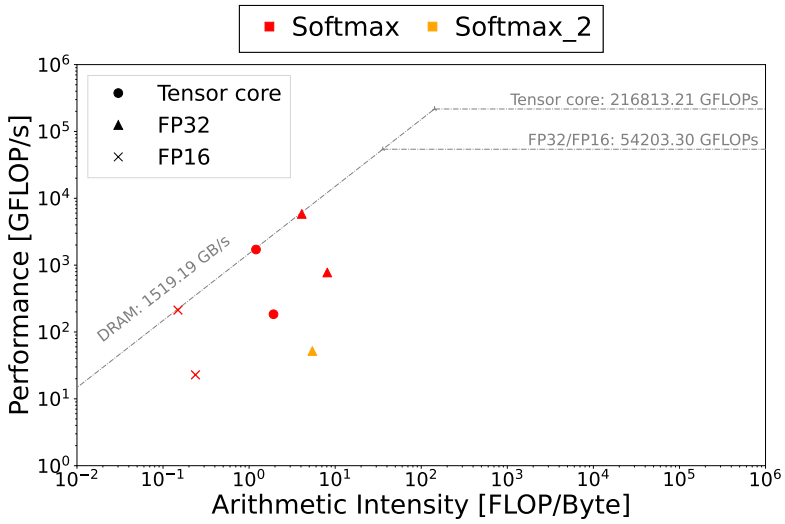


Figure 3.62: Performance and arithmetic intensity of the Softmax kernels used in the Softmax layer while varying the input prompt size.

Varying the relation of total input and output tokens

- Varying the input prompt size: [16, 25, 55, 109, 224, 524, 991, 2205, 3769, 4484, 6319] tokens.
- Varying the output prompt size: [8176, 8167, 8137, 8083, 7968, 7668, 7201, 5987, 4423, 3708, 1873] tokens.
- Fixed sequence length: [8192] tokens.

Attention layer:

In Figures 3.63 and 3.64, we observe the same behavior as in the case of varying the number of input tokens.

When the input prompt is small, the main operations are GEMM kernels. However, as the input prompt increases, SoftMax and elementwise operations begin to dominate the execution time of the layer.

The arithmetic intensity and kernel performance are consistent with the detailed analysis presented in the previous sections.

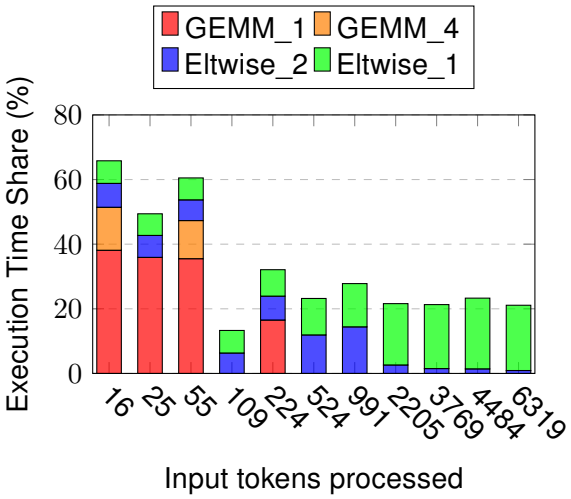


Figure 3.63: Percentage of total execution time for the four most time-consuming kernels during the Attention layer, when most of the sequences is is composed by the input prompt.

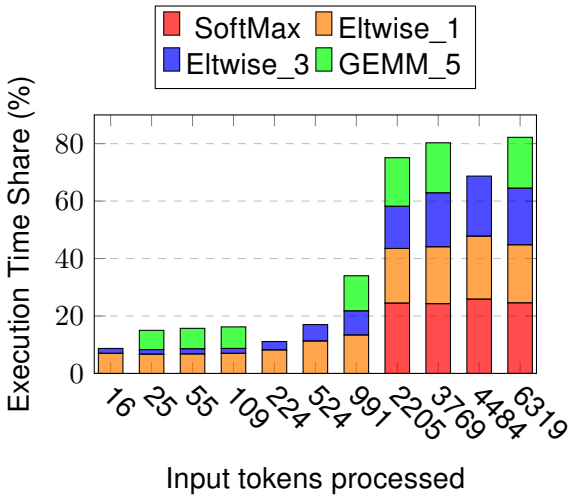


Figure 3.64: Percentage of total execution time for the four most time-consuming kernels during the Attention layer, when most of the sequences is is composed by the input prompt.

Feed Forward Network (FNN) layer:

As observed in the attention layer, we see the same behavior when varying the input prompt size in Figures 3.65 and 3.66:

1. For **small input prompts**, GEMM_1 remains the most time-consuming kernel, consistent with our earlier findings.
2. For **large input prompts**, SoftMax and element-wise operations dominate the execution time profile.

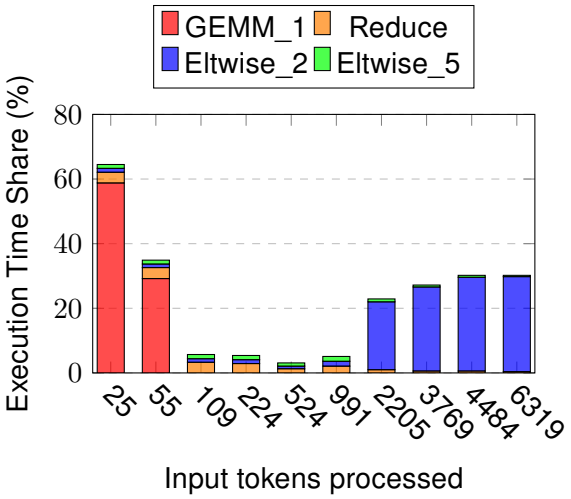


Figure 3.65: Percentage of total execution time for the four most time-consuming kernels during the Feed Forward Network layer, when most of the sequences is is composed by the input prompt.

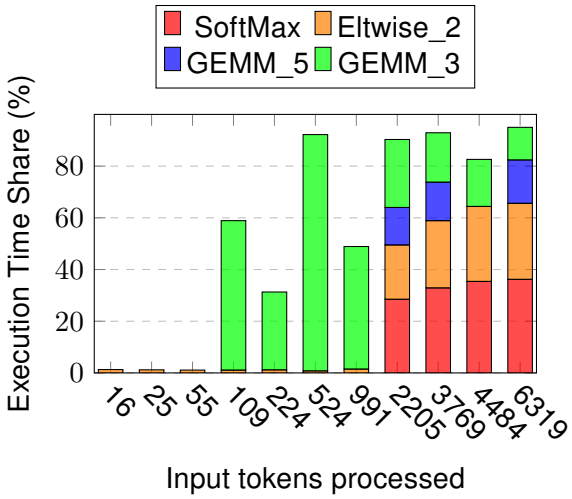


Figure 3.66: Percentage of total execution time for the four most time-consuming kernels during the Feed Forward Network layer, when most of the sequences is is composed by the input prompt.

Rotationary Embedding (Rot_emb) layer:

Once again, we observe the same behavior as when varying the input prompt size.

For small input prompts, element-wise kernels dominate the execution time, particularly Elementwise_1 and Elementwise_2. However, as the input size increases, GEMM operations begin to appear and contribute significantly to the overall execution time.

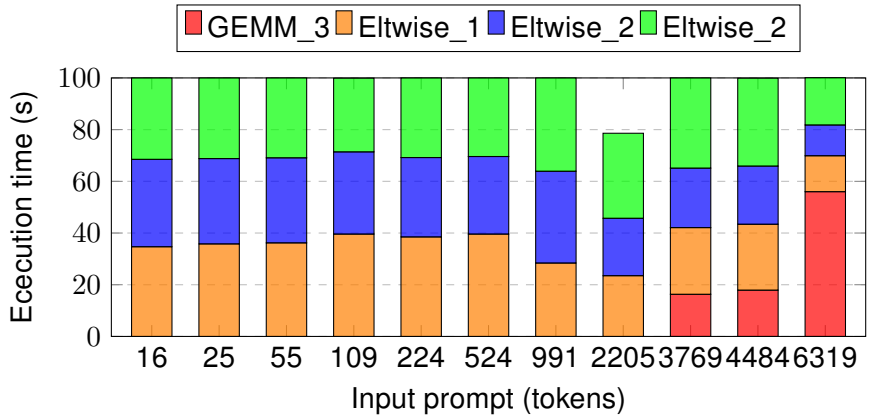


Figure 3.67: Percentage of total execution time for the four most time-consuming kernels during the Rot_emb layer, when most of the sequences is is composed by the input prompt.

SoftMax layer:

Finally, for the SoftMax layer, we once again observe the same behavior as when varying the input prompt size. As the input prompt increases, GEMM kernels begin to appear, and the SoftMax kernels are replaced by other PyTorch implementations. These alternative kernels have different GPU utilization patterns, as previously explained in the input prompt variation analysis.

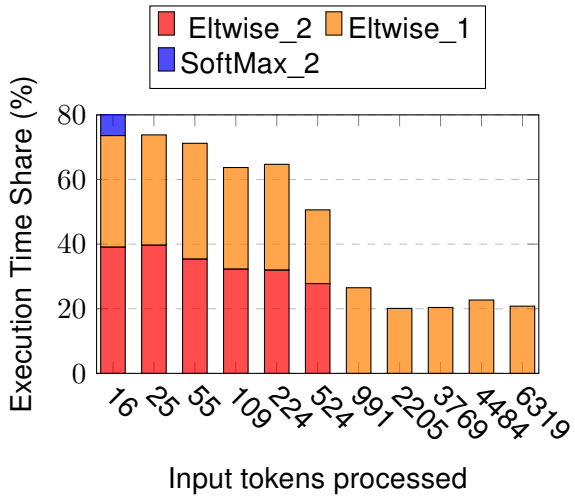


Figure 3.68: Percentage of total execution time for the four most time-consuming kernels during the SoftMax, when most of the sequences is is composed by the input prompt.

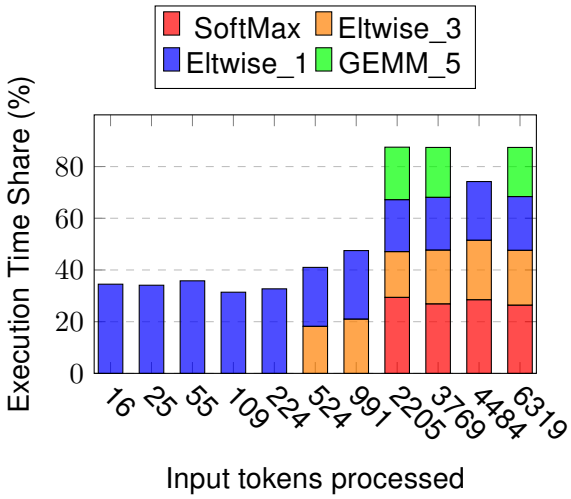


Figure 3.69: Percentage of total execution time for the four most time-consuming kernels during the SoftMax, when most of the sequences is is composed by the input prompt.

Takeouts:

1. When varying the number of output tokens, we observe consistent results across executions.

    - The attention layer is mainly composed of GEMM operations that saturate memory bandwidth.
    - The feed-forward layer is also dominated by GEMM operations with high memory bandwidth usage.
    - Rotary embedding consists mostly of element-wise operations that perform memory copy tasks and operations with low arithmetic intensity and bandwidth usage.
    - The SoftMax layer similarly relies on kernels with low arithmetic intensity and limited bandwidth usage.

2. When varying the input prompt or the ratio of input to output tokens, we observe higher variability in kernel usage and computational requirements. This variability is present in the Attention, Feed-Forward (FNN), and SoftMax layers, while the Rotary Embedding layer remains the most stable across executions.

3.5 Summary

From the experiments and analysis presented in this work, we can extract several important conclusions. Taking always into account that the model fits entirely within the GPU memory and all tests were conducted during inference using a batch size of 1.

First, regarding execution time, we observed that **generating longer output sequences increases the total execution time more than increasing the size of the input** prompt. This is mainly because the prefill stage, which processes the input, consistently takes less time and uses less memory compared to the decode stage, which generates the output. Among all layers, the attention layer is the most time-consuming, accounting for around 60% of the total execution time. This is followed by the Feed-Forward Network (FNN) at around 30%, the rotary embeddings at 10%, and finally the SoftMax layer, which takes approximately 4%.

In terms of **memory usage**, we can separate the total memory footprint into **two parts**. One part is **constant** and comes from the model parameters themselves. The other part is **variable** and depends on the specific execution parameters, such as the length of the input and output sequences. This variable portion can grow up to 7 GB. We also observed that the decode stage generally requires more memory than the prefill stage, and that the attention layer is the component with the highest memory usage. Overall, memory operations do not contribute significantly to execution time, even when processing long sequences.

When analyzing the kernels involved in execution, we found that **GEMM and GEMV operations are the most important when generating long output sequences**. On the other hand, when processing large input prompts, SoftMax and elementwise kernels become more relevant. This reflects the different computational requirements of each part of the model depending on the task.

Finally, the **different layers show specific kernel behavior**. The attention and FNN layers mainly use GEMM kernels, which put high pressure on the memory bandwidth. The rotary embedding layer relies mostly on elementwise kernels. The SoftMax layer uses specialized SoftMax and elementwise kernels, which do not fully use the computational or memory resources of the GPU. In general, although no kernel fully utilizes the GPU's compute capabilities, several of them—particularly GEMM, GEMV, and certain elementwise kernels—do stress the memory bandwidth significantly.

Chapter 4

Model offloading

As shown in Section 2.4, transformer models can have **big variations** in size and structure. The number of **parameters, layer configurations, and target applications** all **influence** their **performance** and resource demands. In many cases, parts of a large model simply will not fit in the memory of a single GPU. One common practice in these scenarios is offloading the model, which is the practice of moving selected model data or computations to a different hardware device—typically the host CPU or even disk storage—to overcome these limitations. By carefully choosing which layers or tensors to offload, we can reduce GPU memory pressure, lower deployment costs, enable more flexible system designs and/or improve overall hardware utilization.

One key motivation for **offloading** is the ever-increasing parameter count of modern LLMs. Models with hundreds of billions of parameters require many high-end GPUs to store their weights and intermediate data such as the key–value (KV) cache. For example, GPT-175B needs over 325 GB of device memory, forcing large clusters of A100 GPUs. Offloading some weights and cache entries into CPU RAM or onto SSDs bounds the GPU footprint and **makes it possible to run such models with fewer accelerators**. This approach not only reduces equipment cost but also simplifies system management [115, 116].

Offloading also exploits the comparatively bigger memory capacity of CPU's main memory and disks. Whereas a GPU might have 40–80 GB of HBM memory, a server CPU typically offers several hundred gigabytes, and local disks can provide terabytes of storage. Systems like FlexGen and RAGDoll demonstrate how mapping inactive weights or large activation buffers to these tiers dramatically increases the effective model size that can be served. In turn, this allows for larger batch sizes or longer input sequences, which directly boosts throughput in scenarios where latency is less critical [116, 117].

Another driver for offloading is the runtime growth of the KV cache during autoregressive decoding. As input length or batch size increases, the cache can grow by hundreds of gigabytes, quickly exceeding GPU limits. Offloading parts of the cache to host memory enables larger batches and longer contexts without requiring additional GPUs. This is especially important for applications such as document generation or summarization, where maintaining a long context window is crucial for output quality [115, 116].

In Retrieval-Augmented Generation (RAG) systems, offloading is even more important. A RAG system uses a large language model together with an external vector database, which can be hundreds of gigabytes in size. To run both the model and the database on limited hardware, we need to spread data and computation across GPU, CPU, and disk. By moving rarely used parts of the database—such as embedding tables or shards—into host memory or disk, systems like RAGDoll can fit the entire retrieval and generation pipeline on a single, ordinary GPU [115, 116, 117, 118].

Throughput-oriented workloads benefit when offloading allows larger effective batch sizes on the GPU. By moving memory-heavy layers or activations to CPU or disk, the GPU can process

more samples in parallel, achieving significant gains in tokens per second. FlexGen, for instance, reported over a hundred-fold throughput increase for a 175B-parameter model by combining offloading with compression and a block scheduling algorithm [117].

Finally, offloading can mitigate PCIe bandwidth bottlenecks by shifting computation for layers with low arithmetic intensity to the CPU. Instead of repeatedly moving large tensors across the PCIe link, those layers execute where their data already resides, reducing data transfer volume and improving overall latency. This cooperative CPU-GPU computing model can achieve better performance than either could alone by maximizing hardware utilization [115].

In this chapter, we apply our methodology to a real-world case: offloading a selected layer of the LLaMA 3 8B model to the host CPU.

4.1 Parameter and layer Selection

As demonstrated in Chapter 3, the model's execution time, kernel activity, and memory demands change significantly with different sequence lengths, prompt sizes, and output lengths. Therefore, before selecting any layer for offloading, we fix these execution parameters to a single, consistent configuration.

4.1.1 Selection of parameters for testing

Varying the input size significantly alters the requirements in terms of kernels executed by the model, as previously discussed in Chapter 3. Also, while varying this parameter the over all execution time effect are "small" and the results are less consistent as explained. In contrast, varying the output sequence size produces more consistent results. The model's arithmetic and memory needs remain stable, the kernels executed are consistent, and the execution time of each layer maintains a constant ratio relative to the total execution time. For this reason, in this chapter, we will test the model by varying the output size while keeping the input size fixed at a minimal value, consistent with the experiments conducted in Chapter 3 when varying the output.

4.1.2 Layer Selection

Once the execution parameters are set, we can choose the layer to offload. To select the best layer for offloading, we consider the following points:

- **Low Compute and Memory Bandwidth Needs**: The layer should not require much computation or memory bandwidth. This is important because the CPU is slower and has less memory bandwidth than the GPU. A layer with low demands is more suitable for the CPU.
- **Position in the Model**: The layer should not be in the middle of the model or called many times during each iteration. Transformer models run layers one after another, so each layer must finish before the next one starts. If a layer runs in every iteration, moving it to the CPU could slow down the model due to frequent data transfers between the GPU and CPU.

Following these rules, layers such as attention and feed-forward network (FFN) are not good choices for offloading. Even though they take a long time to run, they rely heavily on matrix multiplications, which need a lot of compute power and memory bandwidth.

In a standard transformer model, layers like the embedding or softmax layer might be better candidates, These layers have less compute and bandwidth requirement, as shown in Chapter 3. In particular, the rotary positional embedding (RoPE) layer shows the lowest compute and bandwidth usage. However, as explained in Section 3.3.2, RoPE is used to add positional information

to the query and key vectors in the attention mechanism. Because of this, RoPE runs every time the attention layer is executed. Offloading the RoPE layer may cause extra overhead due to frequent data movement between the CPU and GPU. Still, compared to the softmax layer, which also runs inside the attention layer but takes up a smaller portion of total execution time, we choose to offload the RoPE layer.

4.2 Embedding layer offloading results

As we can see in Figure 4.1, offloading the RoPE embedding to the CPU increases the total execution time of the full model by about 20% to 40%.

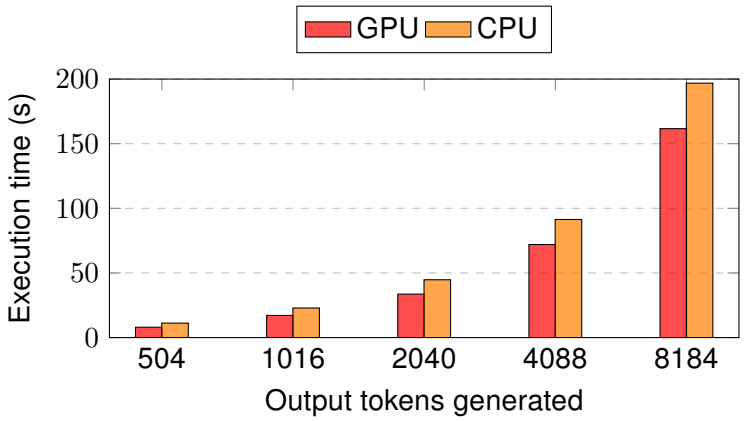


Figure 4.1: Full model execution time with and without offloading the RoPE layer to the CPU.

If we analyze the execution time layer by layer, we can observe how since the RoPE embedding is part of the attention layer, offloading it affects the execution times of both layers as shown in Figures 4.2 and 4.3. The attention layer execution time increases by about 69–80%, while the RoPE layer time increases by about 370–425%. The feed-forward network (FFN) and softmax layers remain the same.

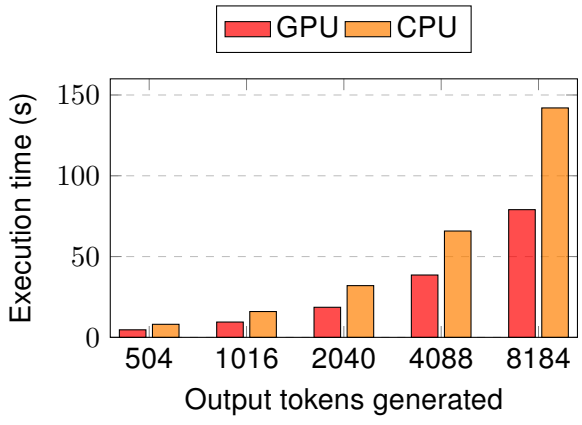


Figure 4.2: Attention layer execution time with and without offloading the RoPE layer to the CPU.

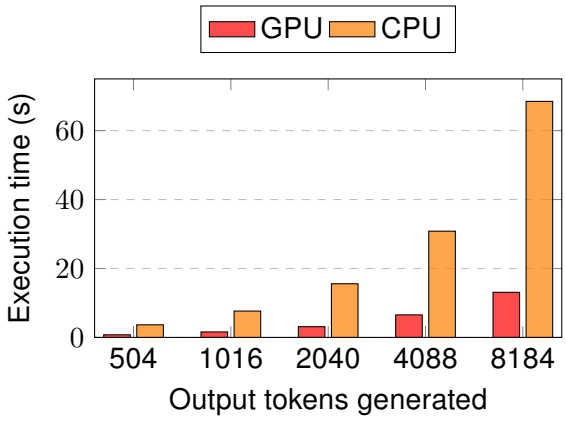


Figure 4.3: RoPE layer execution time executed in CPU and GPU.

It is important to note that the RoPE layer is part of the attention layer. This means that the execution time of the RoPE layer is already included in the attention layer time. These times do not add up; instead, they overlap.

To sum up, because the RoPE layer is contained within the attention layer, each time the attention layer is executed, the input data must first be transferred from the GPU to the CPU through the PCIe bus. After computation, the output data must be sent back to the GPU. This data transfer happens during every iteration of the transformer block. As a result, it increases the overhead of the attention layer. This added overhead leads to longer execution time for both the attention layer and the full model, as shown in the figures. It is also important to highlight that the RoPE function uses GPU-optimized operations such as element-wise multiplications. This way, running them on the CPU can be also contributing to the overall performance loss.

4.3 Summary

We can see that offloading the RoPE layer to the CPU leads to a significant increase in execution time. This is mainly caused by the need to transfer data between the GPU and CPU, which adds overhead to both the attention layer and the overall model.

Nevertheless, by applying the methodology described, we were able to identify the RoPE layer as a potential candidate for offloading and clearly compare the parts of the model affected by this change. This helps us make informed decisions when trying to optimize the model's performance. This chapter shows a practical use of the profiling methodology presented in Chapter 3. In this case, since the offloadable layers are part of larger layers that must run in every iteration of the model, offloading is not an efficient option.

However, a possible improvement could be to run the model on hardware with more suitable capabilities. This would help avoid underutilization and better support efficient layer execution.

Chapter 5

Sustainability analysis and ethical implications

As previously explained, the rapid advancements in Artificial Intelligence (AI) has increase our problem-solving capabilities, enabling machines to learn from data and make intelligent decisions across diverse industries. However, this progress is accompanied by significant challenges, concerning environmental sustainability and ethical implications. The computational demands of modern AI models, such as explained Deep Neural Networks (DNNs), Deep Learning Recommendation Systems (DLRMs), Recurrent Neural Networks (RNNs), and Transformers, contribute to substantial energy consumption and carbon emissions. Simultaneously, ethical concerns arise from the concentration of AI development resources in a few large organizations and the lack of transparency in AI decision-making processes.

The thesis introduces FAiNDER, an interactive web-based database that provides structured access to detailed information about six major AI models, including their architectures and hardware requirements. Additionally, the standardized profiling methodology is proposed to enable reproducible and extensible performance analyses, facilitating obtaining organized insights. These contributions aim to address both the environmental footprint of AI and its ethical challenges.

5.1 Environmental Sustainability

5.1.1 The Ecological Footprint of AI Models

The environmental impact of AI has gain relevance due to the resource-intensive nature of modern models. Training large language models (LLMs), such as Transformers, can emit hundreds of tons of carbon dioxide, comparable to the lifetime emissions of multiple vehicles. Since 2012, the computational power required for training state-of-the-art AI models has doubled approximately every 3.4 months, leading to a significant increase in greenhouse gas emissions [119]. Moreover, the deployment of these models contributes to substantial water usage for data center cooling and generates electronic waste.

For instance, training a model like GPT-4 demands thousands of megawatt-hours of electricity, straining energy grids and contributing to carbon emissions [120].

On the other hand AI inference, the process of using trained models to make predictions or decisions, is also a significant contributor to the environmental impact of AI systems. Unlike training, which is a one-time process, inference occurs continuously in real-world applications, such as chatbots, recommendation systems, and autonomous vehicles, leading to cumulative energy consumption. For instance, deploying a 176-billion-parameter model like BLOOM for inference in the cloud consumes approximately 40.3 kWh daily, emitting around 19 kg of CO_2 equivalent per day

[121]. With millions of inference queries processed globally, the carbon footprint of AI inference is scaling rapidly in the last years [120].

Large-scale inference systems, such as those used by recommendation platforms like Netflix or Amazon, rely on Deep Learning Recommendation Systems (DLRMs). These models demand high memory bandwidth and computational power, often requiring specialized hardware like GPUs or TPUs, which are energy-intensive [122]. The continuous operation of data centers hosting these models also contributes to significant water usage for cooling and electronic waste from hardware lifecycles.

5.1.2 Thesis Contributions to Sustainability

This thesis directly addresses AI's environmental challenges by focusing on improving resource utilization and system performance. The profiling methodology applied to the Llama3-8B model reveals critical insights into hardware demands and inefficiencies. For example, the analysis indicates that while certain kernels (e.g., GEMM, GEMV) stress memory bandwidth, none fully saturate GPU compute capabilities, suggesting opportunities for energy optimization. By identifying such inefficiencies, the methodology enables developers to optimize model execution, reducing energy waste.

The development of proprietary accelerators, such as Meta's Training and Inference Accelerator (MTIA), represents another significant step toward sustainability. MTIA v2, with its enhanced architecture and increased memory capacity, is designed to reduce power consumption while supporting Meta's AI workloads, particularly for inference tasks [123]. This aligns with industry efforts to transition to energy-efficient hardware, reducing reliance on power-intensive GPUs.

Additionally, the thesis explores model offloading techniques, which involve transferring selected model data or computations to alternative platforms, such as CPUs or disk storage, to alleviate GPU memory pressure. Offloading can lower deployment costs and enable the execution of large models on fewer accelerators, potentially reducing energy consumption [124]. Although practical experiments, such as offloading the Rotary Positional Embedding (RoPE) layer in Llama3, showed increased execution time due to data transfer overhead, the principle of optimizing resource distribution remains promising for sustainable AI operations.

Table 5.1 summarizes the environmental impacts of AI and the thesis's contributions to mitigating them.

Aspect	**Environmental Impact**	**Thesis Contribution**
Energy Consumption	High energy use for training and inference, e.g., hundreds of tons of CO2 for LLMs.	Profiling methodology identifies inefficiencies for optimization [125].
Hardware Demands	Reliance on power-intensive GPUs increases carbon footprint [119].	Detect which more energy-efficient accelerators like MTIA could be used [123].
Resource Utilization	Inefficient resource use leads to energy waste.	Offloading techniques reduce GPU memory pressure [124].

Table 5.1: Environmental Impacts of AI and Thesis Contributions

5.2 Ethical Implications

5.2.1 Accessibility and Equity in AI Development

The computational resources required for state-of-the-art AI models create significant barriers to entry, concentrating development capabilities in a few large organizations. For example, training a model like GPT-175B requires over 325 GB of device memory and clusters of A100 GPUs, resources that are often inaccessible to smaller research institutions, startups, or regions with limited infrastructure [126]. This concentration risks increasing inequalities and centralizing decision-making power in AI development, raising ethical questions about fairness and inclusivity.

5.2.2 Transparency and Trust in AI Systems

Transparency is essential for ethical AI, but many AI systems are “black boxes” whose decisions are hard to understand. This lack of clarity can reduce trust and lead to misuse, especially in sensitive areas like recommendation systems [127]. Ethical guidelines highlights the importance of making AI systems explainable to ensure fairness and trust [126].

5.2.3 Thesis Contributions to Ethical AI

The thesis addresses these ethical concerns through its focus on accessibility and transparency. By analyzing model structures and hardware requirements, the research enables the deployment of large models on less specialized hardware through techniques like offloading, potentially allowing access to AI capabilities [124]. The selection of Llama3, an open-source model, for detailed profiling promotes transparency and reproducibility, as its architecture and performance characteristics are publicly accessible [125].

The FAiNDER platform is a key contribution, offering a live and open web tool that provides structured and interactive access to model details and hardware demands. FAiNDER allows researchers and developers to explore model architectures, compare performance characteristics, and identify optimization opportunities based on reliable data [128]. By addressing the lack of standardization in prior profiling studies, the thesis’s standardized methodology enhances the comparability and transparency of AI research, which is essential for mitigating ethical issues like algorithmic bias and ensuring equitable outcomes in real-world applications.

Table 5.2 outlines the ethical challenges and the thesis’s contributions to addressing them.

Ethical Issue	Challenge	Thesis Contribution
Accessibility	High computational barriers limit access to AI development [126].	Offloading enables deployment on less specialized hardware [124].
Transparency	Opaque AI systems erode trust [127].	Open-source Llama3 and FAiNDER promote transparency [125, 128].
Bias and Fairness	Lack of standardization complicates bias detection [126].	Standardized profiling methodology enhances comparability [128].

Table 5.2: Ethical Challenges in AI and Thesis Contributions

5.3 Summary

This chapter underscores the need to address sustainability and ethics in AI development. The environmental footprint of AI models, driven by their energy-intensive training and deployment, poses significant challenges to global sustainability goals. Concurrently, ethical issues surrounding accessibility and transparency highlight the importance of inclusive and accountable AI ecosystems. The thesis's contributions—FAiNDER, the standardized profiling methodology, and explorations into offloading and energy-efficient hardware—provide a robust framework for mitigating these challenges.

The insights gained from **profiling** Llama3-8B and developing FAiNDER are not static; they serve as a foundation for future research. Extending the profiling methodology to larger models, such as LLaMA 70B or 405B, could uncover new optimization opportunities. Additionally, incorporating detailed measurements, such as bandwidth usage and memory allocations over time, would **enhance the understanding** of model performance under diverse workloads. Exploring inter-GPU communication in multi-GPU setups could further improve resource utilization, contributing to both sustainability and accessibility.

In the end, **this research helps create AI systems** that are not only fast and efficient but also **better for the environment and more responsible**. By providing useful information for choosing hardware, improving models, and planning how to run them, this thesis supports a the development of AI models always with sustainability in mind.

Chapter 6

Related work

The rapid growth of AI research and the increasing complexity of models have led to a surge in studies analyzing various aspects of AI systems. Existing research includes surveys that classify models by their architectural features, task-specific applications, or operational traits, alongside profiling studies that reveal runtime behaviors Moreover, as AI models grow increasingly large and complex, offloading techniques have emerged to optimize their deployment by distributing computations across diverse hardware platforms.

6.1 Models surveys

The increment of the number of AI research and models has also provoke the grow of surveys analyzing various aspects of AI models, each focusing on different architectural characteristics and implementation requirements. These studies can be categorize in **three groups**: model-focus analyses, task-oriented examinations, and specific-architecture investigations.

Model-focus surveys provide detailed breakdowns of **specific model families**. For Transformers, some reviews analyze the model variants and their success in natural language processing and computer vision applications [129]. Similarly, recurrent neural networks (RNNs) and their Long Short-Term Memory (LSTM) variants have been documented, explaining their temporal processing capabilities and solutions to challenges like gradient vanishing [64]. In the recommendation systems domain, some surveys analyze their different implementations and map how this architectures address challenges like information overload [130].

Task-oriented analyses study **specific usages** of models. For example, surveys on Neural Machine Translation compare RNN and CNN-based encoder-decoder models, showing how attention mechanisms help manage long sequences for machine translation [131]. Other studies explain how transformers are now used for image tasks using their self-attention mechanism, and their challenges given their high computation requirements [132]. Some of these studies may only focus on either the model's inference or training characteristics. For example, some surveys examine how Transformers perform during inference, with special attention to optimizing matrix operations and memory usage [112].

Specific-architecture surveys look at similarities and differences of how **specific layers and operations** are built and used across different models. An example is, one large study that groups over seventy types of dropout methods by where they are applied, such as in model layers, embeddings, or inputs. It shows that these methods work well in models like recommendation systems with different types of data [62]. Other studies focus on the backpropagation algorithm, which is used in almost all AI models during training, and examine new alternatives that have been developed in recent years [133].

This way we can see a wide range of research however, there are still three main challenges.

First, different model types use different terminology and designs, making it hard to compare them directly. Second, AI models change quickly, often faster than surveys can cover. Third, most surveys study one model type or task in detail, but they do not compare hardware needs across different models. This makes it harder to understand the resource demands of new AI systems that mix several types of models.

6.2 Profiling studies

To understand the requirements of different AI models in their specific implementations, numerous studies have conducted profiling with various objectives. Profiling enables the analysis of **model behavior, the identification of performance bottlenecks**, and the optimization for enhanced hardware and software utilization. Nowadays there exist several research papers that profile AI models in diverse manners.

One study made by **Purdue University and NVIDIA** profiles the BERT-Large model during inference to comprehend the runtime characteristics of operations within Transformer networks. The hardware utilized is a Nvidia Volta GPU. The model comprises approximately 340 million parameters. Although the specific profiler tool is not disclosed, the study measures the execution time of various operations, with a particular emphasis on the softmax operation. By varying the sequence length, the researchers observe that the softmax operation consumes a significant portion of the runtime, particularly for longer sequences. This contrasts with other deep neural networks where the softmax operation is relatively negligible. The findings indicate that the softmax operation constitutes a major performance bottleneck in Transformer implementations on GPU's hardware. Consequently, the study proposes Softermax as a hardware-friendly alternative to the softmax function to address this identified bottleneck [134].

Another paper made by Stevens Institute of Technology, **University of Connecticut and University of Delaware profiles Transformer** model inference on NVIDIA V100S GPUs. The researchers evaluate a method called E.T. against established tools such as TensorRT and Faster-Transformer (libraries designed to speed up and optimize transformer models). The models examined include BERT-Base with 110 million parameters and a general Transformer model. Parameters varied in the study include sequence length (ranging from 64 to 256 tokens), sparsity (from 0% to 95%), number of heads, and model dimension. Input sequences of 128 tokens are utilized for certain tests. The profiling tool employed is nvprof, which measures metrics such as global memory load and store transactions, Streaming Multiprocessor (SM) efficiency, and Instructions Per Cycle (IPC). The results demonstrate that E.T.'s attention mechanism increases memory loads but reduces stores, leading to improved GPU's SM efficiency and IPC compared to TensorRT library. Additionally, E.T. achieves faster execution times, with speedups of up to 3.4 times over TensorRT across various configurations. The study concludes that E.T.'s design and pruning techniques effectively accelerate Transformer inference on GPUs [135].

A separate study made by **Universidade de California, Berkeley** and **NVIDIA** profiles Transformer inference on an Intel Gold 6242 CPU. The models analyzed are the 12-layer BERT-Base (110 million parameters), the 24-layer BERT-Large (340 million parameters), and the 12-layer GPT-2 (comparable in size to BERT-Base). The primary parameter varied is the sequence length, ranging from 128 to 4096 tokens. The study measures the latency breakdown, examining the time allocated to multi-head attention (MHA) projections, matrix multiplications, feed-forward network (FFN) projections, and other operations. Although the specific profiler tool is not mentioned, the results indicate that for shorter sequences, projection layers dominate the latency, whereas for longer sequences, matrix multiplications become the primary bottleneck due to their quadratic scaling. Furthermore, GPT-2 exhibits higher latency than BERT-Base, attributed to its lower arithmetic intensity. The study concludes that as sequence lengths increase, decoder inference becomes constrained by memory bandwidth rather than computational capacity [112].

Another paper made by The **Pennsylvania State University, University of Virginia, University Park and Intel** concentrates on profiling Deep Learning Recommendation Systems (DLRM) inference on an Intel Cascade Lake 6240R CPU. The models tested include RM2_1 (28.6GB), RM2_2, RM2_3, and RM_1 from the RMC1 and RMC2 classes, with variations in embedding dimensions and the number of tables. A fixed batch size of 64 is used, and the study examines different input dataset hotness levels (Low, Medium, High) and core counts. The Intel VTune Profiler is employed to measure execution time breakdown, batch latency, average load latency, cache hit rates (L1D, L2, L3), memory bandwidth, reuse distance, and cold misses. The results reveal that irregular memory accesses and large reuse distances in the embedding stage lead to inefficient cache utilization. The study concludes that off-chip memory access and limited memory bandwidth are the principal performance bottlenecks for DLRM inference on CPUs [56].

One study made by **Facebook** profiles inference for Transformer (NLP), CNN (CV), and DLRM (Recommendation) models on a custom accelerator system comprising six low-power cards and a host CPU. The models range from tens of millions to over 100 billion parameters. For NLP models, sequence length is varied, while for other models, batch size is adjusted. Although a specific profiler tool is not named, the study utilizes their software stack and vendor tools, to measure end-to-end latency, throughput (QPS), execution time breakdown of operations (such as FC, SLS, MatMul, Convolutions), memory bandwidth, and accelerator core utilization. The results indicate that NLP and Recommendation models are memory bandwidth-bound, whereas other models are compute-bound. The study emphasizes that profiling is crucial for identifying performance bottlenecks and optimizing the model, card, and system levels to efficiently execute complex models within latency constraints [59].

Another effort made by **Facebook** profiles DLRM inference on CPUs and custom accelerators using Facebook's production models, which possess tens to hundreds of billions of parameters and large embedding tables. The study varies batch size and model configurations. Profiling is conducted through hardware profiling, performance modeling tools, and custom accuracy monitoring tools, measuring end-to-end latency, throughput, execution time of operators (SparseLengthsSum, FullyConnected), memory bandwidth, and quantization errors. The results show that embedding lookups are memory bandwidth-bound, while standard neural networks can be either memory or compute-bound depending on the batch size. These insights facilitate the development of low-precision optimization strategies to deploy large models efficiently while maintaining accuracy [136].

A study made by **Harvard University and Facebook** on recommendation model inference profiles models such as DLRM-RMC1, RMC2, RMC3, NCF, WnD, MT-WnD, DIN, and DIEN, which feature tens to hundreds of billions of parameters and large embedding tables. Parameters varied include query size, arrival patterns, and batch size, with inputs consisting of dense and sparse features and outputs being click-through rate (CTR) probabilities. Profiling is performed on Intel Broadwell and Skylake CPUs and an NVIDIA GTX 1080Ti GPU using DeepRecInfra and custom tools. Measurements include end-to-end latency, throughput (QPS), and execution time of operations like SparseLengthsSum and FullyConnected. The results indicate that embedding lookups are memory bandwidth-bound, while standard neural networks can be compute or memory-bound. The study concludes that optimizing performance requires balancing request and batch parallelism and strategically offloading queries to accelerators [57].

Another study made by **Samsung** shows the PIM-HBM architecture, which is profiled measure its performance during inference of memory-bound DNNs, including DeepSpeech2, RNN Transducer, GNMT, AlexNet, and ResNet-50. The system integrates a commercial processor with PIM-HBM devices, benchmarked against commodity HBM. Parameters varied are batch size (1, 2, 4) and input data dimensions. Measurements encompass execution time, energy efficiency, power consumption, and LLC miss rates. The results demonstrate up to 11.2 times improvement in performance for memory-bound kernels and 3.2 times higher energy efficiency compared to

the HBM baseline. The study concludes that PIM-HBM is highly effective for data-intensive AI workloads [137].

A study made by **Facebook** profiles inference of recommendation models RMC1, RMC2, and RMC3 in data centers on Intel Haswell, Broadwell, and Skylake CPUs. Parameters varied include batch size (1 to 256) and the number of co-located inference jobs. Inputs are dense and sparse features, with outputs being CTR. Profiling is conducted using Caffe2 with Intel MKL, measuring inference latency, execution time breakdown (FC, SparseLengthsSum), throughput, cache miss rates, SIMD utilization, and latency variability. The results show that latency varies with model and hardware, and while batching enhances throughput, it introduces latency degradation. Bottlenecks shift between embedding lookups and FC layers. The study concludes that recommendation workloads necessitate tailored hardware and scheduling optimizations [47].

Lastly, a study from **Georgia Institute of Technology** profiles inference of the Swin-T model (8.6 million parameters, expanded to 28.3 million) for Vision Transformer MHSA workloads on a 5-tier 3D integrated design with RRAM and SRAM. Tools used include DNN+NeuroSim, SPICE, and Ansys for measuring power, performance (frequency, throughput), area, inference accuracy on ImageNet-1k, signaling, and thermal profiles. The results indicate that the H3DAtten design achieves 8.4 times higher compute density and comparable accuracy (81.19%) with a smaller footprint than 2D baselines. The study concludes that heterogeneous 3D integration is beneficial for memory-intensive MHSA inference [138].

These way we can see how there exist multiple studies that investigate AI models behavior on inference or training, varying parameters such as sequence length, input prompts, or generated outputs, utilize different models and sizes, employ various profiling tools, and operate on different hardware platforms. They measure metrics including memory usage, execution time of layers or operations, and other relevant factors. The **variability in profiling methodologies** across these studies presents a challenge: the **lack of standardization** complicates the comparison and comprehensive understanding of the results across models. The examination of these studies reveals substantial diversity in profiling methodologies. While some research profiles the entire model comprehensively, others concentrate on specific components, such as attention mechanisms or embeddings, to evaluate their proposed solutions. Furthermore, the parameters varied and the hardware platforms employed differ across studies.

6.3 Offloading Techniques

Other research paths focus more on understanding the requirements of the model and how to offload part of it to another hardware platform. Specially in relation to Transformers models there is a wide range of research.

One of this works explores the challenge of running extremely large language models, which now often contain hundreds of billions of parameters, on limited GPU resources. The authors begin by demonstrating that simply storing model weights and intermediate data like the key–value cache in CPU memory during inference—and shuttling data back and forth over the PCIe bus—becomes a severe performance bottleneck. Even when only thirty percent of the weights are kept in CPU memory, over ninety percent of inference time is consumed by PCIe transfers for a 30-billion-parameter model. To alleviate this, they propose a cooperative CPU–GPU inference strategy. This design leverages **Intel's Advanced Matrix Extensions (AMX)** available on modern server CPUs to perform parts of the matrix multiplications directly on the host CPU. The paper introduces a dynamic layer-partitioning policy: it measures each neural network layer's arithmetic intensity (the ratio of compute to data movement) and compares expected CPU versus GPU execution times to decide where each layer should run. In experiments with the OPT-30B model on a system featuring two Sapphire Rapids CPUs (with AMX) alongside an NVIDIA A100 GPU, the CPU–GPU cooperative approach yields up to a twelvefold reduction in latency for single-token generation, compared

to GPU-only inference, and nearly doubling performance over CPU-only execution in latency-sensitive cases. For batched workloads optimized for throughput, the hybrid system achieves over five times the maximum token throughput of a GPU-only baseline, demonstrating clear benefits of using large host memory plus AMX compute to assist GPU acceleration [115].

Another proposal is **FlexGen** which addresses a related but distinct problem: how to run very large language models on a single, commodity GPU by exploiting a full memory hierarchy comprising GPU RAM, host DRAM, and disk storage. The authors formalize the offloading problem as a search over where to place model weights, activations, and the growing key–value cache at each stage of inference, with the goal of minimizing end-to-end latency for batched generation tasks. A key innovation is the zig-zag block scheduling algorithm, which processes multiple input samples in blocks. This schedule reuses weights already resident on the GPU while overlapping computation with I/O. Furthermore, FlexGen introduces group-wise 4-bit quantization of both weights and cache entries, reducing memory footprint by up to 75% without significant accuracy loss. When evaluated on a single NVIDIA T4 GPU with 16 GB of memory, augmented by 208 GB of host DRAM and a 1.5 TB SSD, FlexGen runs the 175-billion-parameter OPT model with effective batch sizes beyond 100, reaching a sustained throughput of more than one token per second, hundreds of times higher than prior offloading systems such as DeepSpeed ZeRO-Inference or Hugging Face Accelerate [116].

Other proposal is the **H2M2** system which extends the offloading concept into custom hardware by co-designing memory and compute units in a heterogeneous architecture. Unlike conventional designs that strictly stack high-bandwidth memory on top of slower capacity memory, H2M2 attaches dedicated accelerators directly to both memory types—such as HBM and LPDDR—arranged in parallel. The paper details a head-aware mapping scheme that partitions each attention head and fully connected kernel across the two memory–compute islands, according to their bandwidth needs and the dynamic size of the key–value cache during decoding. A runtime solver, guided by an analytical cost model, balances compute loads across HBM and LPDDR, while a hardware-supported page-table mechanism virtualizes physical memory, enabling the system to allocate contiguous logical addresses for growing caches without costly data movement. In extensive evaluations on GPT-3 (175B), Chinchilla (70B), and Llama2-70B models, H2M2 achieves 1.5× to nearly 3× speedups over CPU-backed offloading schemes and outperforms strict hierarchical systems by over 30%. Its hardware-assisted memory events incur less than 1.4% overhead [118].

Finally, **RAGDoll** addresses the challenge of deploying Retrieval-Augmented Generation systems on a single GPU by combining offloading with parallelization of retrieval and generation stages. Traditional RAG setups execute retrieval first on a CPU-based vector index and then generation on a GPU, leading to idle periods and suboptimal resource use. RAGDoll decouples these stages into concurrent pipelines: retrieval batches run on the CPU while the GPU performs model inference. The system integrates LLM tensor offloading (leveraging techniques like those in FlexGen) and vector database partitioning into a unified memory management framework. A dedicated prefetching module enqueues upcoming layers and database partitions based on current memory availability, using separate threads and CUDA streams to hide transfer latency. An adaptive controller tunes batch sizes and memory ratios for weights, key–value cache, and index partitions by profiling system performance offline and adjusting dynamically at runtime. On consumer-grade GPUs with 12–24 GB VRAM and 176–256 GB main memory, RAGDoll delivers up to 11.7× lower end-to-end latency than serial RAG baselines like vLLM, and up to 3.6× speedups over simpler parallel designs. Ablation studies confirm that both pipelining and dynamic offloading are critical: disabling either reduces performance by over 50%, highlighting the importance of joint resource management for efficient end-to-end RAG on constrained hardware [117].

Chapter 7

Conclusions

7.1 Thesis contributions

This thesis addresses the challenge of comparing and understanding AI model performance across different platforms.

First, we studied and **analyzed common AI models to understand their differences and similarities** in structure, resource requirements, and usage patterns. We observed that many of these models share similar components. For example, transformer models use embedding layers similar to those in recommendation systems, and feed-forward networks that are also used in deep neural networks (DNNs). Furthermore, transformers have largely replaced recurrent neural networks (RNNs) in natural language tasks such as text recognition, translation, and summarization. This analysis led us to focus on transformer models as the main target of our study. This comparison helps users understand how different models can be applied in various scenarios and how they can be optimized for specific tasks. All of **this work is presented in *FAiNDER* [1]**, a live web platform that offers an accessible and structured overview of widely used AI models in the industry. The platform includes detailed information about model architectures, layer implementations, and hardware requirements. It serves as a useful tool for both researchers and professionals who want to explore, compare, and better understand AI models. FAiNDER has received the HiPEAC Technology Transfer Award for its contribution to power technologies, which will support the development of the next generation of high-performance AI applications.

Next, we **proposed a clear and consistent methodology for measuring compute and memory usage** at different levels of detail and under various workloads. This helps researchers better understand how models behave and allows for more reliable performance comparisons across platforms. We then applied this methodology to transformer models. In particular, we **profiled the LLaMA 3 8B model** to demonstrate the methodology in action. We found that increasing the length of the generated output has a bigger impact on execution time than increasing the input prompt size. This is because the prefill stage, which processes the input, is faster and uses less memory than the decode stage, which generates the output. Among all layers, the attention layer is the most time-consuming, using around 60% of the total time, followed by the feed-forward network (FNN) at 30%, rotary embeddings at 10%, and the SoftMax layer at about 4%. In terms of memory usage, we saw that the total memory footprint consists of a fixed part (model parameters) and a variable part (depending on input and output length), which can grow up to 7 GB. The decode stage needs more memory than the prefill stage, with the attention layer being the most memory-intensive. However, memory operations have a limited impact on execution time, even with long sequences. Regarding kernel usage, GEMM and GEMV kernels dominate when generating long outputs, while SoftMax and elementwise kernels are more relevant for large input prompts. This shows how the model's computational needs change depending on the task. The attention and FNN layers mainly rely on GEMM kernels, which put pressure on memory band-

width. The RoPE layer mainly uses elementwise operations, and the SoftMax layer uses dedicated kernels that do not fully utilize the GPU. In general, no kernel fully uses the GPU's compute capabilities, but many—especially GEMM, GEMV, and certain elementwise kernels—put significant pressure on memory bandwidth.

Finally, we applied the methodology in a real-world scenario by **offloading one model layer to the CPU**. This example showed how profiling results can guide performance improvements. We selected the RoPE layer because of its low compute and memory needs. However, offloading this layer led to a 20% to 40% increase in overall model execution time. This result demonstrates the trade-offs of offloading and the importance of selecting layers carefully based on where and how they are used in the model.

Nevertheless, this work helped us understand the hardware requirements of the model. With this knowledge, we can better select the hardware that matches the model's needs, avoiding underutilization and improving performance.

7.2 Future work

Thanks to the flexibility of the proposed methodology, we can continue to extend and improve the analysis in several directions. One possible extension is to **include more detailed measurements**. For example, we could track **bandwidth usage over time** during the execution of each layer to identify when peak usage occurs. Similarly, we could monitor memory allocations over time to better understand when and how memory is used during model execution. We could also apply the methodology to **different execution setups**. For instance, testing with **batching**, where multiple input sequences are processed at the same time, would help us understand its effect on bandwidth, memory usage and kernel's hardware utilization. In addition, the methodology can be expanded to other models. Within the LLaMA family, we can **analyze larger models** like LLaMA 70B and LLaMA 405B. These models introduce new challenges due to the increased number of parameters and the need for multi-GPU execution. In such cases, studying the communication between GPUs becomes an important aspect of performance analysis.

These future directions would allow us to further validate and improve the methodology, while gaining deeper insights into model behavior under different workloads and hardware configurations.

Kernel glossary

Kernel	Description
GEMM_1	**Name:** `sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_tilesize64x64x64_warpgroupsize1x1x1_execute_segment_k_off_kernel__5x_cublas` **Operation:** General Matrix-Matrix Multiplication **Tile size:** $64 \times 64 \times 64$ **Data types:** BF16 (compute), FP32 (results) **Cores used:** Tensor Cores and FP32 cores.
GEMM_2	**Name:** `sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_tilesize64x128x64_warpgroupsize1x1x1_execute_segment_k_off_kernel__5x_cublas` **Operation:** General Matrix-Matrix Multiplication **Tile size:** $64 \times 128 \times 64$ **Data types:** BF16 (compute), FP32 (results) **Cores used:** Tensor Cores and FP32 cores.
GEMM_3	**Name:** `sm90_xmma_gemm_bf16bf16_bf16f32_f32_tn_n_tilesize128x128x64_warpgroupsize1x1x1_execute_segment_k_off_kernel__5x_cublas` **Operation:** General Matrix-Matrix Multiplication **Tile size:** $128 \times 128 \times 64$ **Data types:** BF16 (compute), FP32 (results) **Cores used:** Tensor Cores and FP32 cores.
GEMM_4	**Name:** `void cutlass::Kernel<cutlass_80_tensorop_s16816gemm_bf16_64x64_32x6_tn_align8>(T1::Params)` **Operation:** General Matrix-Matrix Multiplication **Tile size:** — **Data types:** BF16, FP32 **Cores used:** Tensor Cores and FP32 cores.
GEMM_5	**Name:** `void cutlass::Kernel<cutlass_75_tensorop_bf16_s1688gemm_bf16_128x128_tn_align1>(T1::Params)` **Operation:** General Matrix-Matrix Multiplication **Tile size:** — **Data types:** BF16, FP32 **Cores used:** Tensor cores and FP32 cores.

Table 7.1: GEMMs kernel glossary.

Kernel	Description
GEMV_1	**Name:** `void gemv2T_kernel_val<int, int, __nv_bfloat16, __nv_bfloat16, __nv_bfloat16, float, (int)128, (int)16, (int)4, (int)4, (bool)0, (bool)0, cublasGemvParamsEx<...»(T13, T6, T6)` **Operation:** General Matrix-Vector Multiplication. **Data types:** BF16 (compute), FP32 (results). **Cores used:** FP32 cores.
GEMV_2	**Name:** `std::enable_if<!T7, void>::type internal::gemvx::kernel<int, int, __nv_bfloat16, __nv_bfloat16, __nv_bfloat16, float, (bool)0, (bool)1, (bool)1, (bool)0, (int)7, (bool)0, cublasGemvParamsEx<...»(T13)` **Operation:** General Matrix-Vector Multiplication **Data types:** BF16 (compute), FP32 (results). **Cores used:** Tensor Cores and FP32 cores.
GEMV_3	**Name:** `std::enable_if<!T7, void>::type internal::gemvx::kernel<int, int, __nv_bfloat16, __nv_bfloat16, __nv_bfloat16, float, (bool)0, (bool)1, (bool)0, (bool)0, (int)9, (bool)0, cublasGemvParamsEx<int, cublasGemvTensorStridedBatched<const __nv_bfloat16>, cublasGemvTensorStridedBatched<const __nv_bfloat16>, cublasGemvTensorStridedBatched<__nv_bfloat16>, float»(T13)` **Operation:** General Matrix-Vector Multiplication **Data types:** FP32. **Cores used:** Tensor Cores and FP32 cores.

Table 7.2: GEMVs kernel glossary.

Kernel	Description
Elementwise_1	**Name:** `void at::native::unrolled_elementwise_kernel<at::native ::direct_ copy_kernel_cuda(at::TensorIteratorBase &)::[lambda() (instance 3)]::operator ()() const::[lambda() (instance 7)]::operator ()() const::[lambda(float) (instance 1)], at::detail::Array<char *, (int)2>, TrivialOffsetCalculator<(int)1, unsigned int>, TrivialOffsetCalculator<(int)1, unsigned int>, at::native::memory::LoadWithCast<(int)1>, at::native::memory::StoreWithCast <(int)1»(int, T1, T2, T3, T4, T5, T6)` **Operation:** Tensor copy operation. **Data types:** — **Cores used:** —
Elementwise_2	**Name:** `void at::native::unrolled_elementwise_kernel<at::native ::direct_ copy_kernel_cuda(at::TensorIteratorBase &)::[lambda() (instance 3)]::operator ()() const::[lambda() (instance 12)]::operator ()() const::[lambda(c10::BFloat16) (instance 1)], at::detail::Array<char *, (int)2>, TrivialOffsetCalculator<(int)1, unsigned int>, TrivialOffsetCalculator<(int)1, unsigned int>, at::native::memory::LoadWithCast<(int)1>, at::native::memory:: StoreWithCast<(int)1»(int, T1, T2, T3, T4, T5, T6)` **Operation:** Tensor copy operation. **Data types:** — **Cores used:** —
Elementwise_3	**Name:** `void at::native::elementwise_kernel<(int)128, (int)4, void at::native::gpu_kernel_impl_nocast<at::native::CUDAFunctor_add <c10::BFloat16> > (at::TensorIteratorBase &, const T1 &)::[lambda(int) (instance 1)]>(int, T3)` **Operation:** — **Data types:** — **Cores used:** Tensor Cores and FP32 cores.
Elementwise_4	**Name:** `void at::native::elementwise_kernel<(int)128, (int)4, void at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel _cuda(at::TensorIteratorBase &)::[lambda() (instance 3)]::operator ()() const::[lambda() (instance 12)]::operator ()() const::[lambda(c10::BFloat16) (instance 1)]>(at::TensorIteratorBase &, const T1 &)::[lambda(int) (instance 1)]>(int, T3)` **Operation:** Data copy and data transformation. **Data types:** — **Cores used:** Tensor Cores and FP32 cores.

Table 7.3: Elementwise kernels glossary.

Elementwise_5	**Name:** `void at::native::elementwise_kernel<(int)128, (int)4, void at::native::gpu_kernel_impl_nocast<at::native::BinaryFunctor<c10 ::BFloat16, c10::BFloat16, c10::BFloat16, at::native::binary_internal::MulFunctor<float»>(at:: TensorIterator Base &, const T1 &)::[lambda(int) (instance 1)]>(int, T3)` **Operation:** — **Data types:** BF16, FP32. **Cores used:** Tensor Cores and FP32 cores.
Elementwise_6	**Name:** `void at::native::elementwise_kernel<(int)128, (int)2, void at::native::gpu_kernel_impl_nocast<at::native::BinaryFunctor<c10 ::complex<float>, c10::complex<float>, c10::complex<float>, at::native::binary_internal::MulFunctor<c10::complex<float»»(at:: TensorIteratorBase &, const T1 &)::[lambda(int) (instance 1)]>(int, T3)` **Operation:** — **Data types:** BF16, FP32. **Cores used:** Tensor Cores and FP32 cores.

Table 7.4: Elementwise kernels glossary 2.

Kernel	Description
SoftMax	**Name:** `void at::native::<unnamed>::cunn_SoftMaxForward<(int)4, float, float, float, at::native::<unnamed>::SoftMaxForwardEpilogue>(T4 *, const T2 *, int)` **Operation:** SoftMax **Data types:** — **Cores used:** Tensor cores and FP32 cores.
SoftMax_2	**Name:** `void <unnamed>::softmax_warp_forward<float, float, float, (int)4, (bool)0, (bool)0>(T2 *, const T1 *, int, int, int, const bool *, int, bool)` **Operation:** SoftMax **Data types:** FP32. **Cores used:** FP32 cores.
Reduce	**Name:** `void at::native::reduce_kernel<(int)512, (int)1, at::native::ReduceOp<float, at::native::MeanOps<float, float, float, float>, unsigned int, float, (int)4»(T3)` **Operation:** Reduction operation over a vector. **Data types:** — **Cores used:** Tensor cores and FP32 cores.

Table 7.5: Other kernels glossary.

Bibliography

[1] Fainder: Find comprehensive ai data and insights. https://fainder.eu/. Accessed: 2025-05-01.

[2] Apple Inc. Face detection, n.d. Accessed: 2025-03-03.

[3] Amazon Science. Amazon scientists: Applying deep neural networks to custom skills. *Amazon Science Blog*, Year.

[4] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '18, page 1059–1068, New York, NY, USA, 2018. Association for Computing Machinery.

[5] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. SC '22. IEEE Press, 2022.

[6] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie Amy Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, KR Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. Software-hardware co-design for fast and scalable training of deep learning recommendation models, 2023.

[7] Youngeun Kwon and Minsoo Rhu. Training personalized recommendation systems from (gpu) scratch: Look forward not backwards, 2022.

[8] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '52, page 740–753, New York, NY, USA, 2019. Association for Computing Machinery.

[9] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, RecSys '16, page 191–198, New York, NY, USA, 2016. Association for Computing Machinery.

[10] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024.

[11] Antonio, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences, 2023.

[12] Yanzhang He, Tara N. Sainath, Rohit Prabhavalkar, Ian McGraw, Raziel Alvarez, Ding Zhao, David Rybach, Anjuli Kannan, Yonghui Wu, Ruoming Pang, Qiao Liang, Deepti Bhatia, Yuan Shangguan, Bo Li, Golan Pundak, Khe Chai Sim, Tom Bagby, Shuo yiin Chang, Kanishka Rao, and Alexander Gruenstein. Streaming end-to-end speech recognition for mobile devices, 2018.

[13] Eunjoon Cho and Shankar Kumar. A conversational neural language model for speech recognition in digital assistants. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5784–5788, 2018.

[14] Martin Radfar, Rohit Barnwal, Rupak Vignesh Swaminathan, Feng-Ju Chang, Grant P. Strimel, Nathan Susanj, and Athanasios Mouchtaris. Convrnn-t: Convolutional augmented recurrent neural network transducers for streaming speech recognition, 2022.

[15] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. *arXiv preprint arXiv:1512.02595*, 2016.

[16] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tara Shaked, Tushar Chandra, Hrishi Aradhye, Greg Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Jain, Xiaobing Liu, Hemal Shah, Eric Yao, Aditya Jeyapaul, et al. Wide & deep learning for recommender systems. *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10, 2016.

[17] Wenke Lee, Wen Liu, Donghong Luo, Junfei Zhang, and Wei Gong. A deep learning approach for network intrusion detection system. *Computers & Security*, 76:276–289, 2018.

[18] Liang Chen, Peter Bentley, Kensaku Mori, Kazunari Misawa, Michitaka Fujiwara, and Daniel Rueckert. Deep learning for medical image analysis. *Frontiers in Medical Imaging and Health Informatics*, 21:1–20, 2017.

[19] Matthew D. Zeiler and Rob Fergus. Deep neural networks for object detection. *Advances in Neural Information Processing Systems*, pages 2553–2561, 2014.

[20] Google. The neural networks behind google voice transcription, 2015.

[21] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Springer, 2018.

[22] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530*, 2017.

[23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[24] Alice Johnson, Bob Smith, and Carol Lee. Hardware design exploration of fully-connected deep neural network with binary parameters. In *Proceedings of the 2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 123–126, Tokyo, Japan, May 2024. IEEE.

[25] Cheng Yang, Xiaobo Hu, Yi Wang, and Zhipeng Liu. Accelerating cnn inference on asics: A survey. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 68(4):1327–1339, 2021.

[26] Bahareh Khabbazan, Marc Riera, and Antonio González. An energy-efficient near-data processing accelerator for dnns that optimizes data accesses. *arXiv preprint arXiv:2310.18181*, 2023.

[27] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian Stich, and Martin Jaggi. Error feedback fixes signsgd and other gradient compression schemes. In *International Conference on Machine Learning*, pages 3252–3261. PMLR, 2019.

[28] Shane Cook and Joshua Patterson. Speeding up deep learning inference using tensorrt. https://developer.nvidia.com/blog/speeding-up-deep-learning-inference-using-tensorrt-updated/, 2018. Accessed: 2025-02-23.

[29] T Dettmers. 8-bit approximations for parallelism in deep learning. arxiv 2015. *arXiv preprint arXiv:1511.04561*.

[30] Wei Wen, Cong Xu, Feng Yan, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. Terngrad: Ternary gradients to reduce communication in distributed deep learning. *Advances in neural information processing systems*, 30, 2017.

[31] NVIDIA Corporation. NVIDIA Volta Architecture White Paper. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/volta-architecture/NVIDIA-Volta-Architecture-Whitepaper.pdf, 2017. Accessed: 2025-04-29.

[32] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: A system for large-scale machine learning. *OSDI*, 16:265–283, 2016.

[33] Pytorch: An open source deep learning platform. https://pytorch.org/. Accessed: 2025-04-29.

[34] NVIDIA Corporation. Nvidia tensorrt. 2017. Accessed: 2025-04-29.

[35] Pablo Prieto, Pablo Abad, Jose Angel Gregorio, and Valentin Puente. Performance characterization of popular dnn models on out-of-order cpus. In *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 199–210. IEEE, 2023.

[36] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

[37] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

[38] Daniel Justus, John Brennan, Stephen Bonner, and Andrew Stephen McGough. Predicting the computational cost of deep learning models. In *2018 IEEE international conference on big data (Big Data)*, pages 3873–3882. IEEE, 2018.

[39] Robert A Jacobs. Increased rates of convergence through learning rate adaptation. *Neural networks*, 1(4):295–307, 1988.

[40] Chi-Chung Chen, Chia-Lin Yang, and Hsiang-Yun Cheng. Efficient and robust parallel dnn training through model parallelism on multi-gpu platform. *arXiv preprint arXiv:1809.02839*, 2018.

[41] Linyan Mei, Koen Goetschalckx, Arne Symons, and Marian Verhelst. Defines: Enabling fast exploration of the depth-first scheduling space for dnn accelerators through analytical modeling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 570–583. IEEE, 2023.

[42] Netflix Technology Blog. For your eyes only: Improving netflix video quality with neural networks, 2023. Accessed: 2025-05-01.

[43] Jongse Park Park, Dooyoung Jeong, Kyung Ki Min, and Myung-Chul Lee. An energy-efficient near-data processing accelerator for dnns that optimizes data accesses. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 28(6):1474–1487, 2020.

[44] Balint Jacob, Anirudh Kora, Pavan Balaji, Thomas Carlin, Priyank Gupta, and Thomas R. Markham. Leveraging the bfloat16 artificial intelligence datatype for higher-precision computations. *IEEE Micro*, 40(4):21–29, 2020.

[45] NVIDIA. Mixed precision training. https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html, 2022. Accessed: 2025-02-23.

[46] Weizheng Xu, Youtao Zhang, and Xulong Tang. Parallelizing dnn training on gpus: Challenges and opportunities. In *Companion Proceedings of the Web Conference 2021*, pages 174–178, 2021.

[47] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, Hsien-Hsin S. Lee, Andrey Malevich, Dheevatsa Mudigere, Mikhail Smelyanskiy, Liang Xiong, and Xuan Zhang. The architectural implications of facebook's dnn-based personalized recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 488–501, 2020.

[48] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. Deep interest evolution network for click-through rate prediction. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI'19/IAAI'19/EAAI'19. AAAI Press, 2019.

[49] Zhibo Xiao, Luwei Yang, Tao Zhang, Wen Jiang, Wei Ning, and Yujiu Yang. Deep evolutional instant interest network for ctr prediction in trigger-induced recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, WSDM '24, page 846–854, New York, NY, USA, 2024. Association for Computing Machinery.

[50] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, Adam Hutchin,

Utku Diril, Krishnakumar Nair, Ehsan K. Aredestani, Martin Schatz, Yuchen Hao, Rakesh Komuravelli, Kunming Ho, Sameer Abu Asal, Joe Shajrawi, Kevin Quinn, Nagesh Sreedhara, Pankaj Kansal, Willie Wei, Dheepak Jayaraman, Linda Cheng, Pritam Chopda, Eric Wang, Ajay Bikumandla, Arun Karthik Sengottuvel, Krishna Thottempudi, Ashwin Narasimha, Brian Dodds, Cao Gao, Jiyuan Zhang, Mohammed Al-Sanabani, Ana Zehtabioskuie, Jordan Fix, Hangchen Yu, Richard Li, Kaustubh Gondkar, Jack Montgomery, Mike Tsai, Saritha Dwarakapuram, Sanjay Desai, Nili Avidan, Poorvaja Ramani, Karthik Narayanan, Ajit Mathews, Sethu Gopal, Maxim Naumov, Vijay Rao, Krishna Noru, Harikrishna Reddy, Prahlad Venkatapuram, and Alexis Bjorlin. Mtia: First generation silicon targeting meta's recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, New York, NY, USA, 2023. Association for Computing Machinery.

[51] Jongsoo Park, Maxim Naumov, Protonu Basu, Summer Deng, Aravind Kalaiah, Daya Khudia, James Law, Parth Malani, Andrey Malevich, Satish Nadathur, Juan Pino, Martin Schatz, Alexander Sidorov, Viswanath Sivakumar, Andrew Tulloch, Xiaodong Wang, Yiming Wu, Hector Yuen, Utku Diril, Dmytro Dzhulgakov, Kim Hazelwood, Bill Jia, Yangqing Jia, Lin Qiao, Vijay Rao, Nadav Rotem, Sungjoo Yoo, and Mikhail Smelyanskiy. Deep learning inference in facebook data centers: Characterization, performance optimizations and hardware implications, 2018.

[52] Maxim Naumov, John Kim, Dheevatsa Mudigere, Srinivas Sridharan, Xiaodong Wang, Whitney Zhao, Serhat Yilmaz, Changkyu Kim, Hector Yuen, Mustafa Ozdal, Krishnakumar Nair, Isabel Gao, Bor-Yiing Su, Jiyan Yang, and Mikhail Smelyanskiy. Deep learning training in facebook data centers: Design of scale-up and scale-out systems, 2020.

[53] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, DLRS 2016, page 7–10, New York, NY, USA, 2016. Association for Computing Machinery.

[54] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering, 2017.

[55] Liu Ke, Udit Gupta, Carole-Jean Wu, Benjamin Y. Cho, Mark Hempstead, Brandon Reagen, Xuan Zhang, David M. Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim M. Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Mengxing Li, Bertrand A. Maher, Dheevatsa Mudigere, Maxim Naumov, Martin D. Schatz, Mikhail Smelyanskiy, and Xiaodong Wang. RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing. *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 790–803, 2020.

[56] Rishabh Jain, Scott Cheng, Vishwas Kalagi, Vrushabh Sanghavi, Samvit Kaul, Meena Arunachalam, Kiwan Maeng, Adwait Jog, Anand Sivasubramaniam, Mahmut Taylan Kandemir, and Chita R. Das. Optimizing cpu performance for recommendation systems at-scale. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, New York, NY, USA, 2023. Association for Computing Machinery.

[57] Udit Gupta, Samuel Hsia, Vikram Saraph, Xiaodong Wang, Brandon Reagen, Gu-Yeon Wei, Hsien-Hsin S. Lee, David Brooks, and Carole-Jean Wu. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 982–995, 2020.

[58] Bilge Acun, Matthew Murphy, Xiaodong Wang, Jade Nie, Carole-Jean Wu, and Kim Hazelwood. Understanding training efficiency of deep learning recommendation models at scale. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 802–814, 2021.

[59] Michael Anderson, Benny Chen, Stephen Chen, Summer Deng, Jordan Fix, Michael Gschwind, Aravind Kalaiah, Changkyu Kim, Jaewon Lee, Jason Liang, Haixin Liu, Yinghai Lu, Jack Montgomery, Arun Moorthy, Satish Nadathur, Sam Naghshineh, Avinash Nayak, Jongsoo Park, Chris Petersen, Martin Schatz, Narayanan Sundaram, Bangsheng Tang, Peter Tang, Amy Yang, Jiecao Yu, Hector Yuen, Ying Zhang, Aravind Anbudurai, Vandana Balan, Harsha Bojja, Joe Boyd, Matthew Breitbach, Claudio Caldato, Anna Calvo, Garret Catron, Sneh Chandwani, Panos Christeas, Brad Cottel, Brian Coutinho, Arun Dalli, Abhishek Dhanotia, Oniel Duncan, Roman Dzhabarov, Simon Elmir, Chunli Fu, Wenyin Fu, Michael Fulthorp, Adi Gangidi, Nick Gibson, Sean Gordon, Beatriz Padilla Hernandez, Daniel Ho, Yu-Cheng Huang, Olof Johansson, Shishir Juluri, Shobhit Kanaujia, Manali Kesarkar, Jonathan Killinger, Ben Kim, Rohan Kulkarni, Meghan Lele, Huayu Li, Huamin Li, Yueming Li, Cynthia Liu, Jerry Liu, Bert Maher, Chandra Mallipedi, Seema Mangla, Kiran Kumar Matam, Jubin Mehta, Shobhit Mehta, Christopher Mitchell, Bharath Muthiah, Nitin Nagarkatte, Ashwin Narasimha, Bernard Nguyen, Thiara Ortiz, Soumya Padmanabha, Deng Pan, Ashwin Poojary, Ye, Qi, Olivier Raginel, Dwarak Rajagopal, Tristan Rice, Craig Ross, Nadav Rotem, Scott Russ, Kushal Shah, Baohua Shan, Hao Shen, Pavan Shetty, Krish Skandakumaran, Kutta Srinivasan, Roshan Sumbaly, Michael Tauberg, Mor Tzur, Sidharth Verma, Hao Wang, Man Wang, Ben Wei, Alex Xia, Chenyu Xu, Martin Yang, Kai Zhang, Ruoxi Zhang, Ming Zhao, Whitney Zhao, Rui Zhu, Ajit Mathews, Lin Qiao, Misha Smelyanskiy, Bill Jia, and Vijay Rao. First-generation inference accelerator deployment at facebook, 2021.

[60] Assaf Eisenman, Maxim Naumov, Darryl Gardner, Misha Smelyanskiy, Sergey Pupyrev, Kim Hazelwood, Asaf Cidon, and Sachin Katti. Bandana: Using non-volatile memory for storing deep learning models, 2018.

[61] Zehuan Wang, Yingcan Wei, Minseok Lee, Matthias Langer, Fan Yu, Jie Liu, Shijie Liu, Daniel G. Abel, Xu Guo, Jianbing Dong, Ji Shi, and Kunlun Li. Merlin hugectr: Gpu-accelerated recommender system training and inference. In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys '22, page 534–537, New York, NY, USA, 2022. Association for Computing Machinery.

[62] Yangkun Li, Weizhi Ma, Chong Chen, Min Zhang, Yiqun Liu, Shaoping Ma, and Yuekui Yang. A survey on dropout methods and experimental verification in recommendation. *IEEE Trans. on Knowl. and Data Eng.*, 35(7):6595–6615, jul 2023.

[63] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, Elton Zhang, Rewon Child, Reza Yazdani Aminabadi, Julie Bernauer, Xia Song, Mohammad Shoeybi, Yuxiong He, Michael Houston, Saurabh Tiwary, and Bryan Catanzaro. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model, 2022.

[64] Benyamin Ghojogh and Ali Ghodsi. Recurrent neural networks and long short-term memory networks: Tutorial and survey, 2023.

[65] Afshine Amidi and Shervine Amidi. Recurrent neural networks. Available at https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks.

[66] Mourad Gridach. Character-level neural network for biomedical named entity recognition. *Journal of biomedical informatics*, 70:85–91, 2017.

[67] Bo Peng, Eric Alcaide, Quentin Gregory Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Nguyen Chung, Leon Derczynski, Xingjian Du, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Jiaju Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Johan S. Wind, Stanisław Woźniak, Zhenyuan Zhang, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. RWKV: Reinventing RNNs for the transformer era. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[68] Hainan Xu, Fei Jia, Somshubra Majumdar, Shinji Watanabe, and Boris Ginsburg. Multi-blank transducers for speech recognition. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2023.

[69] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units, 2016.

[70] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023.

[71] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. *arXiv preprint arXiv:1512.02595*, 2016.

[72] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation, 2016.

[73] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024.

[74] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time, 2017.

[75] Aleksandar Botev, Soham De, Samuel L Smith, Anushan Fernando, George-Cristian Muraru, Ruba Haroun, Leonard Berrada, Razvan Pascanu, Pier Giuseppe Sessa, Robert Dadashi, Léonard Hussenot, Johan Ferret, Sertan Girgin, Olivier Bachem, Alek Andreev, Kathleen Kenealy, Thomas Mesnard, Cassidy Hardin, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Armand Joulin, Noah Fiedel, Evan Senter, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, David Budden, Arnaud Doucet, Sharad Vikram, Adam Paszke, Trevor Gale, Sebastian Borgeaud, Charlie Chen, Andy Brock, Antonia Paterson, Jenny Brennan, Meg Risdal, Raj Gundluru, Nesh Devanathan, Paul Mooney, Nilay Chauhan, Phil Culliton, Luiz GUStavo Martins, Elisa Bandy, David Huntsperger, Glenn Cameron, Arthur Zucker, Tris Warkentin, Ludovic Peran, Minh Giang, Zoubin Ghahramani, Clément Farabet, Koray Kavukcuoglu, Demis Hassabis, Raia Hadsell, Yee Whye Teh, and Nando de Frietas. Recurrentgemma: Moving past transformers for efficient open language models, 2024.

[76] Heiga Zen, Yannis Agiomyrgiannakis, Niels Egberts, Fergus Henderson, and Przemysław Szczepaniak. Fast, compact, and high quality lstm-rnn based statistical parametric speech synthesizers for mobile devices, 2016.

[77] Johan Schalkwyk. An all-neural on-device speech recognizer, 2019. Available at: https://research.google/blog/an-all-neural-on-device-speech-recognizer/.

[78] Suwa Xu, Jinwon Lee, and Jim Steele. Psvd: Post-training compression of lstm-based rnn-t models. In *ASRU 2021*, 2021.

[79] P.J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[80] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, and Katie Millican et al. Gemini1, 2023.

[81] GemmaTeam, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy, Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikuła, Mateo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Pier Giuseppe Sessa, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimenko, Tom Hennigan, Vlad Feinberg, Wojciech Stokowiec, Yu hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. Gemma: Open models based on gemini research and technology, 2024.

[82] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[83] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. Available at https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[84] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018. Available at https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_paper.pdf.

[85] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020.

[86] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, and et. al. Gpt-4 technical report, 2024.

[87] James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, Wesam Manassra, Prafulla Dhariwal, Casey Chu, Yunxin Jiao, and Aditya Ramesh. Improving image generation with better captions. *OpenAI*, 2023.

[88] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.

[89] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.

[90] Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, and Eric Chu et al. Palm 2 technical report, 2023.

[91] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Improving image generation with better captions, 2022.

[92] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al.

Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, 2021.

[93] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.

[94] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[95] AI@Meta. Llama 3 model card, 2024.

[96] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.

[97] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019.

[98] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

[99] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023.

[100] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.

[101] Google Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024. Available at https://storage.googleapis.com/deepmind-media/gemini/gemini_v1_5_report.pdf.

[102] Shrihari Sridharan, Jacob R. Stevens, Kaushik Roy, and Anand Raghunathan. X-former: In-memory acceleration of transformers, 2023.

[103] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models, 2022.

[104] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference, 2022.

[105] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery.

[106] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models, 2022.

[107] Meta AI. Llama 3. https://github.com/meta-llama/llama3, 2024. GitHub repository.

[108] Barcelona Supercomputing Center. Marenostrum5 overview, 2023.

[109] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019.

[110] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

[111] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

[112] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, Yakun Sophia Shao, and Amir Gholami. Full stack optimization of transformer inference: a survey, 2023.

[113] Pouya Esmaili-Dokht, Francesco Sgherzi, Valéria Soldera Girelli, Isaac Boixaderas, Mariana Carmin, Alireza Monemi, Adrià Armejach, Estanislao Mercadal, Germán Llort, Petar Radojković, Miquel Moreto, Judit Giménez, Xavier Martorell, Eduard Ayguadé, Jesus Labarta, Emanuele Confalonieri, Rishabh Dubey, and Jason Adlard. A mess of memory system benchmarking, simulation and application profiling. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 136–152, 2024.

[114] NVIDIA Corporation. Nvidia nsight compute documentation, 2025.

[115] Hyungyo Kim, Gaohan Ye, Nachuan Wang, Amir Yazdanbakhsh, and Nam Sung Kim. Exploiting intel advanced matrix extensions (amx) for large language model inference. *IEEE Computer Architecture Letters*, 23(1):117–120, 2024.

[116] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: high-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

[117] Weiping Yu, Ningyi Liao, Siqiang Luo, and Junfeng Liu. Ragdoll: Efficient offloading-based online rag system on a single gpu, 2025.

[118] Soojin Hwang, Jungwoo Kim, Sanghyeon Lee, Hongbeen Kim, and Jaehyuk Huh. Hardware-based heterogeneous memory management for large language model inference, 2025.

[119] MIT Sloan Management Review. Tackling AI's climate change problem. *MIT Sloan Management Review*, 2023.

[120] MIT News. Explained: Generative AI's environmental impact, January 2025.

[121] Hugging Face. The environmental impacts of AI – primer, 2024.

[122] C. Wu et al. Sustainable AI: Environmental implications, challenges and opportunities. 2023.

[123] Meta. Introducing our next generation infrastructure for AI, April 2024.

[124] A. Elgamal et al. Adaptive AI-enhanced computation offloading with machine learning for QoE optimization and energy-efficient mobile edge systems. *Nature Scientific Reports*, 2025.

[125] H. Touvron et al. The Llama 3 herd of models. 2024.

[126] UNESCO. Ethics of artificial intelligence, 2024.

[127] A. Adadi et al. Transparency and explainability of AI systems: From ethical guidelines to requirements. *ScienceDirect*, 2023.

[128] [Your Name]. *[Your Thesis Title]*. PhD thesis, [Your University], 2025. This thesis.

[129] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers, 2021.

[130] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. Deep learning based recommender system: A survey and new perspectives. *ACM Comput. Surv.*, 52(1), February 2019.

[131] Shuoheng Yang, Yuxin Wang, and Xiaowen Chu. A survey of deep learning techniques for neural machine translation, 2020.

[132] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in vision: A survey. *ACM Comput. Surv.*, 54(10s), September 2022.

[133] John Waldo. A comparative study of back propagation and its alternatives on multilayer perceptrons, 2022.

[134] Jacob R. Stevens, Rangharajan Venkatesan, Steve Dai, Brucek Khailany, and Anand Raghunathan. Softermax: Hardware/software co-design of an efficient softmax for transformers. In *Proceedings of the 58th Annual ACM/IEEE Design Automation Conference*, DAC '21, page 469–474. IEEE Press, 2022.

[135] Shiyang Chen, Shaoyi Huang, Santosh Pandey, Bingbing Li, Guang R. Gao, Long Zheng, Caiwen Ding, and Hang Liu. E.t.: re-thinking self-attention for transformer models on gpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery.

[136] Zhaoxia Deng, Jongsoo Park, Ping Tak Peter Tang, Haixin Liu, Jie Yang, Hector Yuen, Jianyu Huang, Daya Khudia, Xiaohan Wei, Ellie Wen, Dhruv Choudhary, Raghuraman Krishnamoorthi, Carole-Jean Wu, Satish Nadathur, Changkyu Kim, Maxim Naumov, Sam Naghshineh, and Mikhail Smelyanskiy. Low-precision hardware architectures meet recommendation model inference at scale. *IEEE Micro*, 41(5):93–100, 2021.

[137] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, Seongil O, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. Hardware architecture and software stack for pim based on commercial dram technology. In *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ISCA '21, page 43–56. IEEE Press, 2021.

[138] Wantong Li, Madison Manley, James Read, Ankit Kaul, Muhannad S. Bakir, and Shimeng Yu. H3datten: Heterogeneous 3-d integrated hybrid analog and digital compute-in-memory accelerator for vision transformer self-attention. *IEEE Trans. Very Large Scale Integr. Syst.*, 31(10):1592–1602, October 2023.

List of Figures

List of Tables

Listings